



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验零：环境搭建与实验准备

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. Rust 学习和巩固，了解标准库提供的基本数据结构和功能。
2. QEMU 与 Rust 环境搭建，尝试使用 QEMU 启动 UEFI Shell。
3. 了解 x86 汇编、计算机的启动过程，UEFI 的启动过程，实现 UEFI 下的 Hello, world!。

二. 实验内容

1. 配置实验环境

2. 尝试使用 Rust 进行编程：五个基础任务

2.1. 使用 Rust 编写一个程序,完成以下任务:

a. 创建一个函数 `count_down(seconds: u64)`

该函数接收一个 `u64` 类型的参数，表示倒计时的秒数。

函数应该每秒输出剩余的秒数，直到倒计时结束，然后输出 `Countdown finished!`。

b. 创建一个函数 `read_and_print(file_path: &str)`

该函数接收一个字符串参数，表示文件的路径。

函数应该尝试读取并输出文件的内容。如果文件不存在，函数应该使用 `expect` 方法主动 panic,并输出 `File not found!`。

c. 创建一个函数 `file_size(file_path: &str) -> Result<u64, &str>`

该函数接收一个字符串参数，表示文件的路径，并返回一个 `Result`。

函数应该尝试打开文件，并在 `Result` 中返回文件大小。如果文件不存在，函数应该返回一个包含 `File not found!` 字符串的 `Err`。

d. 在 `main` 函数中，按照如下顺序调用上述函数：

首先调用 `count_down(5)` 函数进行倒计时

然后调用 `read_and_print("/etc/hosts")` 函数尝试读取并输出文件内容

最后使用 `std::io` 获取几个用户输入的路径，并调用 `file_size` 函数尝试获取文件大小，并处理可能的错误。

2.2. 实现一个进行字节数转换的函数，并格式化输出：

a. 实现函数 `humanized_size(size: u64) -> (f64, &'static str)` 将字节数转换为人类可读的大小和单位

使用 1024 进制，并使用二进制前缀（B, KiB, MiB, GiB）作为单位

b. 补全格式化代码，使得你的实现能够通过如下测试：

```
#[test]
fn test_humanized_size() {
    let byte_size = 1554056;
    let (size, unit) = humanized_size(byte_size);
    assert_eq!("Size : 1.4821 MiB", format!("{}", FIXME));
}
```

2.3. 自行搜索学习如何利用现有的 crate 在终端中输出彩色的文字

输出一些带有颜色的字符串，并尝试直接使用 `print!` 宏输出一到两个相同的效果。

尝试输出如下格式和内容： - INFO: Hello, world!，其中 INFO: 为绿色，后续内容为白色
- WARNING: I'm a teapot!，颜色为黄色，加粗，并为 WARNING 添加下划线 - ERROR: KERNEL PANIC!!!，颜色为红色，加粗，并尝试让这一行在控制行窗口居中 - 一些你想尝试的其他效果和 内容.....

2.4. 使用 enum 对类型实现同一化

实现一个名为 Shape 的枚举，并为它实现 `pub fn area(&self) -> f64` 方法，用于计算不同形状的面积。 - 你可能需要使用模式匹配来达到相应的功能 - 请实现 Rectangle 和 Circle 两种 Shape，并使得 area 函数能够正确计算它们的面积 - 使得你的实现能够通过如下测试：

```
#[test]
fn test_area() {
    let rectangle = Shape::Rectangle {
        width: 10.0,
        height: 20.0,
    };
    let circle = Shape::Circle { radius: 10.0 };

    assert_eq!(rectangle.area(), 200.0);
    assert_eq!(circle.area(), 314.1592653589793);
}

!!! note "可以使用标准库提供的 `std::f64::consts::PI`"
```

2.5. 实现一个元组结构体 UniqueId(u16)

使得每次调用 `UniqueId::new()` 时总会得到一个新的不重复的 `UniqueId`。 - 你可以在函数体中定义 `static` 变量来存储一些全局状态 - 你可以尝试使用 `std::sync::atomic::AtomicU16` 来确保多线程下的正确性（无需进行验证，相关原理将在 Lab 5 介绍，此处不做要求） - 使得你的实现能够通过如下测试：

```
#[test]
fn test_unique_id() {
    let id1 = UniqueId::new();
    let id2 = UniqueId::new();
    assert_ne!(id1, id2);
}
```

3. 运行 UEFI Shell：初始化仓库，使用 QEMU 启动 UEFI Shell

4. YYOS 启动：配置 Rust ToolChain，运行第一个 UEFI 程序

三. 实验过程

1. 配置实验环境

1.1. 安装项目开发环境

1. 安装 WSL2

```
wsl --install -d Ubuntu
```

2. 使用系统包管理器安装依赖

```

je-suis-un-chat@LAPTOP-MAGCR3QA:~$ qemu-system-x86_64 --version
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.12)
Copyright (c) 2003-2023 Fabrice Bellard and the QEMU Project develop
ers
je-suis-un-chat@LAPTOP-MAGCR3QA:~$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There
is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR P
URPOSE.

je-suis-un-chat@LAPTOP-MAGCR3QA:~$ gdb --version
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/
gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

```

图 1 验证相关软件包的版本

3. 安装 Rust 开发环境与工具链

1. 安装 Rustup

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

2. 安装 Rust Toolchain

```
rustup toolchain install stable-x86_64-unknown-uefi
```

3. 安装 Rust 组件

```
rustup component add rust-src --toolchain stable-x86_64-unknown-uefi
```

```
rustup component add llvm-tools-preview --toolchain stable-x86_64-unknown-uefi
```

4. 检查编译器版本

```
rustc --version
```

```
cargo --version
```

```

je-suis-un-chat@LAPTOP-MAGCR3QA:~$ rustc --version
rustc 1.93.1 (01f6ddf75 2026-02-11)
je-suis-un-chat@LAPTOP-MAGCR3QA:~$ rustc +nightly --version
rustc 1.96.0-nightly (ec818fda3 2026-03-02)
je-suis-un-chat@LAPTOP-MAGCR3QA:~$

```

2. 尝试使用 Rust 进行编程

这部分要完成五份代码，我通过学习文档和与大模型对话掌握了 rust 的基本语法并完成了这部分任务。

通过任务一，我学会了用 cargo 构建一个 rust 项目，并熟悉了一个标准 rust 项目的代码文件和配置文件以及它们之间的关系。

通过任务二，我了解了 cargo 提供的良好的单元测试、集成测试，掌握了和 C、C++ 不太一样的调试流程。

通过任务三，我掌握了如何在终端中输出彩色的文字，在完成这部分任务的时候，我还遇到了一个问题：在配置 colored 依赖包并运行 cargo run 时，我一直遭遇网络报错：SSL connect error (unexpected eof while reading)。无论重试多少次，Cargo 都无法成功下载依赖的索引文件。出现这个问题主要是因为 WSL 环境下，网络拦截的情况比较复杂。为了排查与解决问题，我尝试了几个常见的修复手段：切换清华大学镜像源、更改拉取方式、关闭多路复用，都尝试失败。最终我运行了以下命令来检查网络：

```
curl -v https://index.crates.io/config.json
```

结果显示：

```
e-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/colored_output$ curl -v https://index.crates.io/config.json
* Uses proxy env variable no_proxy == '192.168.*,172.31.*,172.30.*,172.29.*,172.28.*,172.27.*,172.26.*,172.25.*,172.24*,
172.23.*,172.22.*,172.21.*,172.20.*,172.19.*,172.18.*,172.17.*,172.16*,10*,127*,
localhost,<local>'
* Uses proxy env variable https_proxy == 'https://127.0.0.1:7890'
* Trying 127.0.0.1:7890...
* Connected to 127.0.0.1 (127.0.0.1) port 7890
* ALPN: curl offers http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* CAfile: /etc/ssl/certs/ca-certificates.crt
* CApath: /etc/ssl/certs
* OpenSSL SSL_connect: SSL_ERROR_SYSCALL in connection to 127.0.0.1:7890
* Closing connection
curl: (35) OpenSSL SSL_connect: SSL_ERROR_SYSCALL in connection to 127.0.0.1:7890
je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/colored_output$
```

意味着根本问题出在我的终端代理环境变量格式写错了。

最终，我用 `unset https_proxy`、`unset http_proxy` 等命令，清除了当前环境中错误的代理变量。并重新以正确的格式声明了代理：将 `https_proxy` 和 `http_proxy` 的值都设置为了 `http://127.0.0.1:7890`。

修改完成之后，网络瞬间打通，我的彩色输出也顺利跑起来了。

通过任务四，我学会了 rust 语句中的两个重要概念：携带数据的枚举和模式匹配。在 rust 中，枚举的每一个变体都可以独立携带属于自己的数据结构，而 match 可以在需要时安全地把结构解构并分类处理。

通过任务五，我了解了 rust 的元组结构体——只有一个类型包裹着的结构体，和全局状态量 static——储存在程序固定的内存区域，贯穿程序整个生命周期。而 AtomicU16 可以保证每一次获取的值都各不相同。

3. 运行 UEFI Shell

3.1. 概念学习

由于这部分内容涉及到很多不认识没见过的名词，所以在任务开始前，我先通过网络搜索和于大模型对话的方式，了解了一些基本概念，笔记如下：

3.1.1. UEFI

UEFI, 即统一可扩展固件接口，是操作系统和固件之间的接口，它提供了启动操作系统或运行预启动程序的标准环境。它支持多种操作系统的加载，如 Windows、Linux 等。在实验文档的上下文中，UEFI Shell 是一个基于 UEFI 的命令行工具，它允许我们在 UEFI 环境中执行一些简单的操作，例如查看硬件信息、启动操作系统等。

3.1.2. QEMU

QEMU 是一个开源的托管虚拟机软件，它可以模拟各种硬件平台和设备，使得一台电脑能模拟运行另一台完全不同的电脑系统，它主要有以下两大核心功能：

1. 全系统模拟：

在这个模式下，QEMU 会模拟一整套硬件，包括 CPU、内存、硬盘和网卡。这使得用户可以在一个物理机上运行另一个操作系统，就像在真实硬件上一样。

2. 用户模式模拟：

在这个模式下，QEMU 只模拟用户空间的程序，而不模拟整个系统。这使得用户可以在一个平台上运行另一个平台的应用程序，例如在 x86 上运行 ARM 程序。

3.1.3. OVMF

OVMF 是一个开源的 UEFI 固件实现，它允许我们在 QEMU 中模拟 UEFI 环境。通过使用 OVMF，我们可以在 QEMU 中启动 UEFI Shell，并进行一些基本的操作，例如查看硬件信息、启动操作系统等。

3.2. 初始化你的仓库

首先将实验仓库克隆到本地，然后参考实验 0x00 代码的文件结构，初始化我的仓库。

3.3. 使用 QEMU 启动 UEFI Shell

UEFI Shell 是一个基于 UEFI 的命令行工具，它可以让我们在 UEFI 环境下进行一些简单的操作。

在不挂载任何硬盘的情况下，我们可以使用如下命令启动 UEFI Shell：

```
qemu-system-x86_64 -bios ./assets/OVMF.fd -net none -nographic
```

命令参数解释：

`qemu-system-x86_64`：指定使用 QEMU 模拟 x86_64 架构的系统。

`-bios ./assets/OVMF.fd`：指定使用 OVMF 固件文件作为 BIOS，这个文件提供了 UEFI 环境。

`-net none`：禁用网络功能。

`-nographic`：禁用图形输出，使用命令行界面。

4. YYOS 启动！

4.1. 配置 Rust ToolChain

由于仓库提供的 `rust-toolchain.toml` 已经指定了需要使用的 Rust 工具链版本，只需要将这个文件放在项目根目录下，运行 `cargo build` 就会自动下载并使用指定的工具链。

4.2. 运行第一个 UEFI 程序

在 `crates/boot/src/main.rs` 中，完善示例代码，修改注释部分，完整代码如下：


```

#![no_std]
#![no_main]

#[macro_use]
extern crate log;
extern crate alloc;

use core::arch::asm;
use uefi::{Status, entry};

#[entry]
fn efi_main() -> Status {
    uefi::helpers::init().expect("Failed to initialize utilities");
    log::set_max_level(log::LevelFilter::Info);

    let std_num = 24312063;

    loop {
        info!("Hello World from UEFI bootloader! @ {}", std_num);

        for _ in 0..0x10000000 {
            unsafe {
                asm!("nop");
            }
        }
    }
}

```

由于本人初学 rust，所以需要这份简单代码进行分析，笔记如下：

1. `#![no_std]` 和 `#![no_main]` 是 Rust 的属性宏，分别表示不使用标准库和不使用默认的 `main` 函数作为程序入口点。这在操作系统开发中很常见，因为我们需要直接控制底层硬件，而不依赖于标准库提供的功能。
2. `#[macro_use] extern crate log;` 和 `extern crate alloc;` 是引入外部 crate 的语句，分别引入了日志库和分配器库。这些库提供了日志记录和内存分配的功能，在操作系统开发中非常有用。
3. `core::arch::asm;` 引入了 Rust 的内联汇编功能，允许我们在 Rust 代码中直接编写汇编指令。
4. `uefi::{Status, entry};` 引入了 UEFI crate 中的 `Status` 类型和 `entry` 宏，`Status` 用于表示函数的返回状态，而 `entry` 宏用于定义程序的入口点。
5. `#[entry] fn efi_main() -> Status { ... }` 定义了程序的入口函数 `efi_main`，这个函数会在 UEFI 环境下被调用。函数返回一个 `Status` 类型的值，表示函数执行的结果。
6. 在 `efi_main` 函数中，首先调用了 `::helpers::init()` 来初始化 UEFI 的实用工具，如果初始化失败会 panic 并输出错误信息。接着设置了日志级别为 `Info`。
7. 定义了一个变量 `std_num`，赋值为我的学号 24312063。

答:UEFI (统一可扩展固件接口) 和 Legacy BIOS (基本输入输出系统) 是两种不同的计算机启动固件接口。它们之间的主要区别如下:

== 启动方式: Legacy BIOS 使用 MBR (主引导记录) 分区方案, 而 UEFI 使用 GPT (GUID 分区表) 分区方案。UEFI 支持更大的硬盘和更多的分区。

- 启动速度: UEFI 启动速度通常比 Legacy BIOS 快, 因为它使用更现代的启动机制和更高效的代码。
- 安全性: UEFI 支持 Secure Boot (安全启动), 可以防止未经授权的操作系统和驱动程序加载, 提高系统安全性。Legacy BIOS 没有这种功能。
- 用户界面: UEFI 通常提供更友好的图形用户界面, 而 Legacy BIOS 主要是基于文本的界面。

2. 尝试解释 Makefile 中的命令做了哪些事情? 或许你可以参考下列命令来得到更易读的解释:

```
python ysos.py run --dry-run
```

答:这个命令是使用 Python 运行一个名为 ysos.py 的脚本, 并传递了两个参数: run 和 -dry-run。

- run 参数可能是告诉脚本执行某个特定的操作, 可能是运行一个程序或者执行某个任务。
- -dry-run 参数通常用于模拟执行命令而不实际执行它。这意味着脚本会显示它将要执行的操作, 但不会真正执行任何更改。这对于测试和验证命令的正确性非常有用, 可以帮助用户了解命令的效果而不冒风险。

3. 利用 cargo 的包管理和 docs.rs 的文档, 我们可以很方便的使用第三方库。这些库的源代码在哪里? 它们是什么时候被编译的?

答:第三方库的源代码通常托管在 GitHub 或其他代码托管平台上。当我们使用 cargo 来添加依赖时, cargo 会从 crates.io (Rust 的包注册中心) 下载库的源代码, 并将其存储在本地的 cargo 缓存目录中 (通常位于 `./cargo/registry/src/`)。这些库是在我们第一次构建项目时被编译的, 或者当我们更新依赖时重新编译。编译后的库文件会被存储在 target 目录下的 debug 或 release 文件夹中, 具体取决于我们使用的是 debug 模式还是 release 模式。

4. 为什么我们需要使用 entry 而不是直接使用 main 函数作为程序的入口?

答:在 Rust 中，`entry` 是一个属性宏，用于指定程序的入口点。我们需要使用 `entry` 而不是直接使用 `main` 函数作为程序的入口，是因为在某些环境（如嵌入式系统或操作系统开发）中，`main` 函数可能不适合作为入口点。`entry` 允许我们定义一个自定义的入口函数，这个函数可以根据特定的环境需求进行配置和初始化，而不受 `main` 函数的限制。这对于操作系统开发来说尤其重要，因为我们可能需要在程序启动时执行一些特定的初始化步骤，而这些步骤可能不适合放在 `main` 函数中。



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验一：操作系统的启动

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 了解页表的作用、ELF 文件格式、操作系统在 x86 架构的基本启动过程。
2. 尝试使用 UEFI 加载并跳转到内核执行内核代码。
3. 实现基于 uart16550 的串口驱动，使用宏启用输出能力、并启用日志系统。
4. 学习并尝试使用调试器对内核进行调试。

二. 实验内容

1. 编译内核 ELF
2. 在 UEFI 中加载内核：
 - 加载相关文件
 - 更新控制寄存器
 - 映射内核文件
 - 跳转执行
 - 调试内核
3. UART 与日志输出
 - 串口驱动
 - 日志输出
 - Panic 处理
4. 思考题

三. 实验过程

1. 编译内核 ELF

1.1. 步骤一：知识学习

1.1.1. 了解分页内存

页表是操作系统中常见的内存模型，它是一个虚拟地址到物理地址的映射表，通过它可以实现虚拟内存。

在 x64 架构中，页表不再是一个简单连续的映射表，而是一个树状结构，这样的结构被称为多级页表。多级页表的设计使得内存管理更加灵活和高效，可以支持更大的地址空间。在本实验的实现中，会使用到四级页表。

页表分级的设计目的和映射方式：

- 一级页表 (PML4)：包含 512 个条目，每个条目指向一个二级页表。
- 二级页表 (PDPT)：包含 512 个条目，每个条目指向一个三级页表。
- 三级页表 (PD)：包含 512 个条目，每个条目指向一个四级页表。
- 四级页表 (PT)：包含 512 个条目，每个条目指向一个物理页框。

每个页表条目包含了物理地址和一些标志位，如存在位、读写位、用户/内核位等。通过设置这些标志位，操作系统可以控制内存访问权限和行为。

本实验直接使用 x86_64crate 提供的 PageTable 结构体和 Cr3 寄存器的封装,来实现页表的创建和管理。

这个封装提供了：

1. 类型安全的页表：提供 PageTable 结构体，你可以像操作数组一样访问那 512 个项，而不是直接操作内存地址。
2. 映射器 (Mapper)：提供 OffsetPageTable 或 MappedPageTable 等抽象。你只需要调用 .map_to() 函数，它会自动帮你完成 L4 -> L3 -> L2 -> L1 的逐级查找和填充。
3. 寄存器包装：提供 Cr3::write() 这种函数。你传给它一个物理地址，它帮你执行那条把地址塞进 CPU 核心的指令。
4. 位标志定义：提供 PageTableFlags，让你用 WRITABLE | PRESENT 这种人类能读懂的单词，而不是 0x3 这种神秘数字。

1.1.2. 了解 ELF 文件格式

ELF (Executable and Linkable Format) 是一种常见的可执行文件格式，广泛用于类 Unix 系统中。它定义了文件的结构和内容，包括代码段、数据段、符号表等信息。在操作系统开发中，内核通常以 ELF 格式编译生成，这样可以方便地加载和执行。

ELF 文件大体上由文件头和数据组成，它还可以加上额外的调试信息。

一般来说，ELF 文件包含以下几个重要部分：

- ELF Header：包含文件的基本信息，如类型、架构、入口点地址等。
- Program Header Table：描述了程序的内存映射，包括代码段、数据段等。
- Section Header Table：描述了文件的各个节 (section)，如符号表、字符串表等。
- Code and Data Sections：包含实际的代码和数据。

在本实验中，我们需要编译内核代码生成一个 ELF 文件，然后在 UEFI 环境中加载这个 ELF 文件并执行其中的代码。

控制 ELF 的结构：

- 源码层：使用编译器属性

```
int __attribute__((section(".myvariable"))) a = 0;
int __attribute__((section(".myfunction"))) main() { return 0; }
```

效果:强制将特定的数据或代码放入自定义的 section 中，而不是默认的 .text 或 .data 等 section。这对于操作系统开发非常有用，可以让我们更精确地控制内核的内存布局。

- 链接器层：使用链接器脚本

链接器脚本是一种文本文件，定义了如何将不同的输入文件和 section 映射到最终的内存地址空间中。通过链接器脚本，我们可以指定特定的 section 应该被放置在内核映像的哪个位置，以及它们的对齐方式等属性。

```
SECTIONS
{
    . = 0xc0ffee00;
    .text : { *(.text) }
    . = ALIGN(0x1000);
    .data : { *(.data) }
    .bss : { *(.bss) }
}
```

效果:将 .text 段放在 0xc0ffee00 地址开始，.data 段紧跟其后，并且 .bss 段对齐到 0x1000 边界。这种方式可以让我们精确控制内核的内存布局，确保代码和数据被放置在预期的位置。

- 编译指令：控制调试与符号信息

```
gcc main.c -c -o main.o && ld main.o -T ./script.ld -o main
```

效果:编译生成一个 ELF 文件，并且通过链接器脚本控制其内存布局。这个命令会将 main.c 编译成 main.o，然后使用链接器脚本将 main.o 链接成一个可执行的 ELF 文件 main。

参数解释：`-c`：告诉编译器只编译源文件，不进行链接，生成一个目标文件 (.o)。`-o main.o`：指定输出文件名为 main.o。`-T ./script.ld`：指定链接器脚本文件为 script.ld，这个脚本定义了 ELF 文件的内存布局和 section 的映射。`-o main`：指定最终生成的可执行文件名为 main。

为什么需要控制 ELF 的结构？

在操作系统开发中，内核需要被加载到特定的内存地址，并且需要有特定的布局来满足硬件和软件的要求。通过控制 ELF 文件的结构，我们可以确保内核代码和数据被放置在正确的位置，从而保证内核能够正确地运行。

1. 适配硬件与内存映射

只有将权限相同的 section 打包进同一个 segment，才能正确设置段权限（如可执行、可写等），否则可能会导致内核无法正常运行。

2. 定义程序“生存空间”

- 内核开发：内核通常需要运行在高位虚拟地址，必须通过链接脚本强制指定。
- 裸机/嵌入式：硬件寄存器或存储器的物理地址是固定的，必须通过 ELF 结构确保代码被加载到正确位置。

3. 优化加载效率

- 合并加载：OS 加载器以 segment 为单位进行搬运。通过控制 ELF 结构，减少 segment 的数量可以显著加快程序启动速度。
- 按需加载：只有标记为 LOAD 类型的段才会被分配物理内存，其余部分留在磁盘上，节省内存空间。

4. 安全加固

通过控制 ELF 结构，可以将敏感代码和数据放在特定的 section 中，并设置适当的权限（如只读、不可执行等），从而提高系统的安全性。

1.2. 步骤二：编译内核 ELF

按照实验文档，我在 crates/kernel 目录下运行了 `cargo build --release`，但是出现了几个问题导致编译失败，我排查了这些报错并逐一解决。最后完成了这部分任务。

1.2.1. 问题一：找不到全局内存分配器

在普通应用程序里，如果想用动态内存，底层会自动调用操作系统的 `malloc` 和 `free`。但是，由于我正在写操作系统内核本身，（`#![no_std]` 环境），我的代码底下并没有另一个操作系统来提供 `malloc`。当我引入了相关核心库时，Rust 编译器就会要求指定分配内存的负责人。如果没有，编译器就会拒绝编译。

于是，就有如下报错信息：

```
error: no global memory allocator found but one is required; link to std or add
#[global_allocator] to a static item that implements the GlobalAlloc trait
```

为了解决这个问题，我在 kernel/src/main.rs 文件最下方添加了代码：

```

struct DummyAllocator;

unsafe impl core::alloc::GlobalAlloc for DummyAllocator {
    unsafe fn alloc(&self, _layout: core::alloc::Layout) -> *mut u8 {
        core::ptr::null_mut()
    }

    unsafe fn dealloc(&self, _ptr: *mut u8, _layout: core::alloc::Layout) {}
}

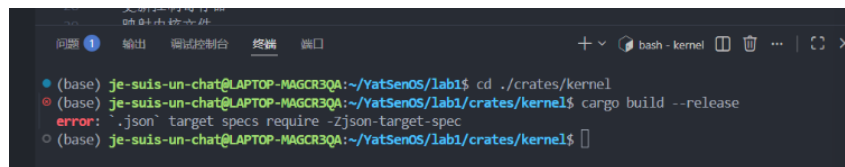
#[global_allocator]
static ALLOCATOR: DummyAllocator = DummyAllocator;

```

这段代码给程序添加了一个名为 `DummyAllocator` 的伪分配器，满足了编译条件。

1.2.2. 问题二：命令参数缺失

在运行这个命令时，报了如下错误：



```

(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1$ cd ./crates/kernel
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1/crates/kernel$ cargo build --release
error: '.json' target specs require -Zjson-target-spec
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1/crates/kernel$

```

按照提示，我直接在命令后面添加相应参数即可解决：

```
cargo +nightly build --release -Z build-std -Z json-target-spec
```

1.2.3. 问题三：Cargo 依赖下载网络失败

在拉取 crates.io 索引时，频繁报出如下错误信息：

```

(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1/crates/kernel$ cargo build --release -Z json-target-spec
Updating `rsproxy-sparse` index
Locking 30 packages to latest Rust 1.96.0-nightly compatible versions
warning: spurious network error (2 tries remaining): [35] SSL connect error (TLS connect error: error:0A000126:SSL
routines::unexpected eof while reading)
network error (2 tries remaining): [35] SSL connect error (TLS connect error: error:0A000126:SSL routines::unexpected
eof while reading)
error: failed to download from `https://rsproxy.cn/api/v1/crates/x86/0.52.0/download`
Caused by: warning: spurious network error (2 tries remaining): [35] SSL connect error (TLS connect error:
error:0A000126:SSL routines::unexpected eof while reading)
warning: spurious network error (2 tries remaining): [35] SSL connect error (TLS connect error: error:0A000126:SSL
routines::unexpected eof while reading)
warning: spurious network error (2 tries remaining): [35] SSL connect error (TLS connect error: error:0A000126:SSL
routines::unexpected eof while reading)
warning: spurious n
[35] SSL connect error (TLS connect error: error:0A000126:SSL routines::unexpected eof while reading)

```

为了解决这个问题，我在 config.toml 文件下添加了清华大学的镜像源：

```
[source.crates-io]
replace-with = 'tuna'

[source.tuna]
registry = "https://mirrors.tuna.tsinghua.edu.cn/git/crates.io-index.git"

[net]
git-fetch-with-cli = true
```

但是还是出现了如下报错：

```
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1/crates/kernel$ cargo update
Updating `tuna` index
warning: spurious network error (3 tries remaining): [35] SSL connect error (TLS connect error: error:0A000126:SSL
routines::unexpected eof while reading); class=Net (12)
warning: spurious network error (2 tries remaining): [35] SSL connect error (TLS connect error: error:0A000126:SSL
routines::unexpected eof while reading); class=Net (12)
warning: spurious network error (1 try remaining): [35] SSL connect error (TLS connect error: error:0A000126:SSL
routines::unexpected eof while reading); class=Net (12)
error: failed to get `arrayvec` as a dependency of package `yyos_boot v0.1.0 (/home/je-suis-un-chat/YatSenOS/lab1/
crates/boot)`

Caused by:
failed to load source for dependency `arrayvec`

Caused by:
unable to update registry `crates-io`

Caused by:
failed to update replaced source registry `crates-io`

Caused by:
failed to fetch `https://mirrors.tuna.tsinghua.edu.cn/git/crates.io-index.git`

Caused by:
network failure seems to have happened
if a proxy or similar is necessary `net.git-fetch-with-cli` may help here
https://doc.rust-lang.org/cargo/reference/config.html#netgit-fetch-with-cli

Caused by:
[35] SSL connect error (Send failure: Broken pipe); class=Net (12)
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1/crates/kernel$
```

这是因为我使用的清华源还在用旧的 Git 协议，由于网络环境或 SSL 证书问题，握手时发生了 pipe 。

为了解决这个问题，我在 config.toml 文件中添加了一行配置：

```
[net]
git-fetch-with-cli = true
```

改用 Ubuntu 系统自带的 git 命令行工具来解决 SSL 错误。

我还调整了全局 Git 配置，让它能适配我的清华源：

```
# 增加 Git 的网络缓冲区大小（防止大文件传输中断）
git config --global http.postBuffer 524288000

# 如果是因为 SSL 证书链验证问题（常见于校园网或公司内网代理），可以临时关闭验证
git config --global http.sslVerify false

# 设置低速连接不超时（防止在索引更新慢时断开）
git config --global http.lowSpeedLimit 0
git config --global http.lowSpeedTime 999999
```

在再次执行命令前，彻底清空代理：

```
unset http_proxy
unset https_proxy
unset ALL_PROXY
cargo update
```

最终解决了这个网络问题。

解决了上述三个问题后，我的 ELF 内核终于编译成功了。

1.3. 步骤三：找到编译产物并用命令查看其基本信息

使用 `ls -lh` 命令查看编译产物基本信息：

```
readelf -l target/x86_64-unknown-none/release/ysos_kernel
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1$ readelf -l target/x86_64-unknown-none/release/ysos_kernel
```

```
Elf file type is EXEC (Executable file)
Entry point 0xffffffff0000010d0
There are 7 program headers, starting at offset 64
```

Program Headers:

Type	Offset	VirtAddr	PhysAddr
	FileSiz	MemSiz	Flags Align
LOAD	0x00000000000001000	0xffffffff000000000	0xffffffff000000000
	0x00000000000005b4	0x00000000000005b4	R 0x1000
LOAD	0x00000000000002000	0xffffffff0000001000	0xffffffff0000001000
	0x00000000000001f5a	0x00000000000001f5a	R E 0x1000
LOAD	0x00000000000004000	0xffffffff0000003000	0xffffffff0000003000
	0x00000000000001020	0x00000000000001020	RW 0x1000
LOAD	0x00000000000006000	0xffffffff0000005000	0xffffffff0000005000
	0x00000000000000000	0x00000000000000020	RW 0x1000
GNU_RELRO	0x00000000000005000	0xffffffff0000004000	0xffffffff0000004000
	0x00000000000000020	0x00000000000000020	R 0x1
GNU_EH_FRAME	0x0000000000000158c	0xffffffff000000058c	0xffffffff000000058c
	0x0000000000000000c	0x0000000000000000c	R 0x4
GNU_STACK	0x00000000000000000	0x00000000000000000	0x00000000000000000
	0x00000000000000000	0x00000000000000000	RW 0x0

Section to Segment mapping:

Segment Sections...

```
00 .rodata .eh_frame_hdr .eh_frame
01 .text
02 .data .got
03 .bss
04 .got
05 .eh_frame_hdr
06
```

```
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1$
```

问题 24 输出 调试控制台 终端 端口

```
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1$ readelf -h target/x86_64-unknown-none/release/ysos_kernel
```

ELF Header:

```
Magic: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
Class: ELF64
Data: 2's complement, little endian
Version: 1 (current)
OS/ABI: UNIX - System V
ABI Version: 0
Type: EXEC (Executable file)
Machine: Advanced Micro Devices X86-64
Version: 0x1
Entry point address: 0xffffffff0000010d0
Start of program headers: 64 (bytes into file)
Start of section headers: 34872 (bytes into file)
Flags: 0x0
Size of this header: 64 (bytes)
Size of program headers: 56 (bytes)
Number of program headers: 7
Size of section headers: 64 (bytes)
Number of section headers: 12
Section header string table index: 10
```

```
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1$
```

6 18 0 Update is ready, click to restart.

```

• (base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1/target/x86_64-unknown-none/release$ readelf -S ysos_kernel
There are 12 section headers, starting at offset 0x8838:

```

Section Headers:

[Nr]	Name	Type	Address	Offset
	Size	EntSize	Flags Link Info Align	
[0]		NULL	0000000000000000	00000000
	0000000000000000	0000000000000000	0 0 0	
[1]	.rodata	PROGBITS	ffffff0000000000	00001000
	000000000000058b	0000000000000000	AMS 0 0 4	
[2]	.eh_frame_hdr	PROGBITS	ffffff000000058c	0000158c
	000000000000000c	0000000000000000	A 0 0 4	
[3]	.eh_frame	PROGBITS	ffffff0000000598	00001598
	000000000000001c	0000000000000000	A 0 0 8	
[4]	.text	PROGBITS	ffffff0000001000	00002000
	0000000000001f5a	0000000000000000	AX 0 0 16	
[5]	.data	PROGBITS	ffffff0000003000	00004000
	00000000000001d8	0000000000000000	WA 0 0 8	
[6]	.got	PROGBITS	ffffff0000004000	00005000
	0000000000000020	0000000000000000	WA 0 0 8	
[7]	.bss	NOBITS	ffffff0000005000	00006000
	0000000000000020	0000000000000000	WA 0 0 8	
[8]	.comment	PROGBITS	0000000000000000	00006000
	0000000000000093	0000000000000001	MS 0 0 1	
[9]	.symtab	SYMTAB	0000000000000000	00006098
	0000000000000cc0	0000000000000018	11 105 8	
[10]	.shstrtab	STRTAB	0000000000000000	00006d58
	000000000000005a	0000000000000000	0 0 1	
[11]	.strtab	STRTAB	0000000000000000	00006db2
	0000000000001a82	0000000000000000	0 0 1	

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
D (mbind), l (large), p (processor specific)

1.4. 步骤四：回答实验任务问题

1.4.1. Q1：编译产物的架构相关信息，与配置文件中的描述是否一致？

A1：编译产物的各项底层架构参数与 x86_64-unknown-none.json 配置文件中的描述完美吻合。这说明 Rust 编译器（rustc）成功读取了我们提供的自定义目标配置（Target JSON），并严格按照该配置的要求，完成了跨平台交叉编译（Cross-compilation）工作，生成了符合 YatSenOS 要求的裸机（none 操作系统）二进制执行文件。

1.4.2. Q2：找出内核的入口点，它是被如何控制的？结合源码、链接、加载的过程，谈谈你的理解。

A2：在计算机体系结构和操作系统工程中，内核入口点的控制机制是一个跨越编译期、链接期和运行期的严格协同过程。内核的入口点并不是硬件自动识别的，而是通过源码定义、链接器硬编码、加载器解析跳转这三个阶段来实现控制权的精确转移。

在源码层面，内核通过调用底层的宏 (`boot::entry_point!`),在语言级别静态定义了一个不受编译器名称修饰机制干扰的全局符号 (`_start`),从而向外部提供了一个具有标准调用约定的确定性标识。

进入链接阶段后，链接器严格遵循自定义链接脚本 (`kernel.ld`) 的内存布局规范，通过 `ENTRY` 指令声明将该符号解析为程序的唯一入口，并根据位置计数器为其所在的代码段分配绝对的内存基址，最终将该地址硬编码写入最终生成的 ELF 可执行文件头部的入口字段中。

在系统启动的加载阶段，先期运行的引导程序作为加载器，会动态解析该 ELF 文件的文件头和程序头表，按规范将内核的代码段和数据段等搬运至物理内存的对应地址空间，随后直接提取出头部预存的入口地址数值，将其强制转换为底层的无返回值函数指针，最后通过执行底层的机器级跳转指令直接修改中央处理器的指令指针寄存器，完成从引导环境到内核初始指令执行流的精确控制权转移。

1.4.3. Q3：请找出编译产物的 segments 的数量，并且用表格的形式说明每一个 segments 的权限、是否对齐等信息。

序号	类型 (Type)	权限标志 (Flg)	实际权限	对齐 (Align)	映射的节 (Section)
00	LOAD	RE	可读、可执行	0x1000	.text
01	LOAD	R	只读	0x1000	.rodata
02	LOAD	RW	可读、可写	0x1000	.bss
03	GNU_STACK	RW	可读、可写	0x0	(无)

表 1 编译产物 Segments 权限与对齐信息表

2. 在 UEFI 中加载内核

2.1. 步骤一：阅读并理解 crates/boot 文件夹中的代码文件

2.1.1. boot 文件夹的总体作用

在 CPU 上电后，它并不知道如何运行 Rust 编写的内核。boot 文件夹的作用就是：

1. 接管硬件：通过 UEFI 固件获得访问磁盘、显示器和内存的权限。
2. 环境初始化：开启分页机制，建立虚拟地址映射。
3. 搬运内核：从磁盘找到内核 ELF 文件，把它加载到物理内存中。
4. 权力交接：把收集到的硬件信息打包成 `BootInfo`，然后跳进内核入口点。

2.1.2. config.rs 文件

读取并解析文本形式的配置文件。

2.1.3. fs.rs 文件

通过 UEFI 协议与磁盘打交道，负责把内核文件从磁盘搬到内存里。

2.1.4. allocator.rs 文件

提供引导阶段的动态内存管理。

2.1.5. lib.rs 文件

确保内核知道从哪里开始运行，以及如何拿到引导程序准备好的数据或程序。

2.1.6. main.rs 文件

控制整个引导流程的生命周期。

2.2. 步骤二：补全 boot/src/main.rs 代码

2.2.1. 加载相关文件

在 crates/boot/src/fs.rs 中，提供了一些文件操作相关的函数可调用，使用这些函数分别加载配置文件和内核 ELF，代码如下：

```
// 1. 加载并解析配置文件
let mut config_file = fs::open_file("\\EFI\\BOOT\\boot.conf");
let config_content = fs::load_file(&mut config_file);
let config = config::Config::parse(config_content);

info!("Config解析成功: {:#x?}", config);

// 2. 加载内核 ELF 文件
let mut kernel_file = fs::open_file(config.kernel_path);
let kernel_content = fs::load_file(&mut kernel_file);
let elf = xmas_elf::ElfFile::new(kernel_content).expect("Failed to parse kernel ELF");

unsafe {
    set_entry(elf.header.pt2.entry_point() as usize);
}
```

2.2.2. 更新控制寄存器

CR0 寄存器是 x86 架构下的一个控制寄存器，它决定了 CPU 的工作模式和内存管理行为。其中第 16 位被称为 WP 位（写保护位）。在正常的保护模式或长模式下，如果一个内存页在页表被标记为“只读”，那么即便是内核代码，一旦尝试写入该区域，CPU 硬件就会立刻触发页错误，直接导致系统崩溃或重启。

为了进行后续读取并映射内核 ELF 文件的操作,现在需要使用 Cr0 寄存器禁用根页表的写保护,以便进行后续的内存映射操作。

```
unsafe {  
    // 暂时关闭写保护以修改页表  
    Cr0::update(|f| f.remove(Cr0Flags::WRITE_PROTECT));  
}
```

2.2.3. 映射内核文件并跳转执行

2.2.3.1. 前置知识：分页内存管理

虽然前文已经写过,为了保持每个子任务的相对独立性和完整性,在这里再简要概述:

1. 页表

页表是操作系统中常用的内存模型,用它可实现虚拟内存、进行内存隔离、避免内存碎片等。它是一个虚拟地址到物理地址的映射表,通过它可实现虚拟内存。

2. 页表表项

页表中的每一个表项都是一个 64 位的数据,能够描述一个页表项的属性,及一些权限管理:

R (是否可读);

W (是否可写);

X (是否可执行);

3. 虚拟内存

通过利用页表,操作系统可以实现虚拟内存,将一部分分配出的物理内存映射到虚拟地址空间中。

4. x64 相关寄存器

在 x64 架构中,有一些特殊的寄存器与页表相关,且在本实现中会用到,它们分别是:

- CR0: 控制寄存器,存储了一些控制系统运行状态的标志位,包括根页表的写保护位。
- CR2: 页错误地址寄存器,存储了最近一次页错误的虚拟地址。
- CR3: 页表根地址寄存器,存储了页表的物理地址,也就是 PML4 的地址。

2.2.3.2. 前置知识：内核 ELF 文件格式

ELF 是一种文件格式,既可用作可执行文件,也可用作共享文件库文件。ELF 文件大体上由文件头和数据组成,还可以加上额外的调试信息。

一般来说,ELF 有以下几个部分:

- ELF 文件头
- section header
- program header
- 文件内容: section 和 segment

2.2.3.3. 前置知识：操作系统在 x86 架构的基本启动过程

1. 上电自检：按下电源键，电源管理器向 CPU 发送 RESET 信号。CPU 初始化寄存器。
2. 固件接管：CPU 开始执行主板 Flash 芯片上的固件代码(BIOS 或 UEFI)
3. 硬件扫描：初始化内存、检查显卡、扫描磁盘总线。
- 4.UEFI 加载：
 - 初始化协议栈：建立起完整的文件系统支持(FAT32)、网络协议栈和图形输出(GOP)
 - 加载 EFI 应用程序：固件直接读取文件系统里的 .efi 文件，如果没有注册条目，它会寻找默认的 EFIBOOTBOOTX64.EFI
 - ExitBootServices:启动程序准备好内存映射后，调用此服务。UEFI 固件释放对硬件的控制权，系统正式进入 OS 掌控时间
- 5.引导 CPU 进入 64 位长模式：开启 PAE、加载页表、设置 EFER.LME、开启分页

2.2.3.4. 架构了解：阅读配置文件定义中有关内核栈的内容

这份代码为 YatSenOS 的引导程序定义了一个配置解析器，它通过读取并解析一个名为 boot.conf 的文本文件，动态地决定内核在启动时的内存布局参数（如内存栈的虚拟基地址、2MB 的栈空间大小以及物理内存映射偏移量），从而让开发者无需重新编译代码，就能灵活地控制内核运行时的硬件环境和路径信息。

2.2.3.5. 了解了前置知识和项目架构之后，对代码进行以下补充：

```
// 4. 建立虚拟内存映射
let mut page_table = current_page_table();
let mut frame_allocator = UEFIFrameAllocator;

unsafe {
    // 暂时关闭写保护以修改页表
    Cr0::update(|f| f.remove(Cr0Flags::WRITE_PROTECT));
}

// --- 步骤 4.1: 线性映射整个物理内存 ---
// 这是修复 Page Fault 的关键：为 load_elf 提供可写的线性地址空间
elf::map_physical_memory(
    config.physical_memory_offset,
    max_phys_addr,
    &mut page_table,
    &mut frame_allocator,
);
```

```

// --- 步骤 4.2: 加载内核段 ---
elf::load_elf(
    &elf,
    config.physical_memory_offset,
    &mut page_table,
    &mut frame_allocator,
).expect("Failed to load and map ELF segments");

// --- 步骤 4.3: 映射内核栈 ---
elf::map_range(
    config.kernel_stack_address,
    config.kernel_stack_size,
    &mut page_table,
    &mut frame_allocator,
).expect("Failed to map kernel stack");

unsafe {
    // 重新开启写保护
    Cr0::update(|f| f.insert(Cr0Flags::WRITE_PROTECT));
}

free_elf(elf);

```

```
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenos/lab1$ python ysos.py launch -d
BdsDxe: loading Boot0001 "UEFI QEMU HARDDISK QM00001 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Primary,Master,0x0)
BdsDxe: starting Boot0001 "UEFI QEMU HARDDISK QM00001 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Primary,Master,0x0)
[ INFO]: crates/boot/src/main.rs@022: Running UEFI bootloader...
[ INFO]: crates/boot/src/fs.rs@050: Load file "boot.conf" to memory, size = 684
[ INFO]: crates/boot/src/config.rs@052: parse kernel_stack_address = 0xffffffff010000000
[ INFO]: crates/boot/src/config.rs@052: parse kernel_stack_size = 512
[ INFO]: crates/boot/src/config.rs@052: parse physical_memory_offset = 0xfffff80000000000
[ INFO]: crates/boot/src/config.rs@052: parse kernel_path = \KERNEL.ELF
[ INFO]: crates/boot/src/config.rs@052: parse kernel_stack_auto_grow = 0
[ INFO]: crates/boot/src/main.rs@029: Config解析成功: Config {
[ INFO]: crates/boot/src/main.rs@029:     kernel_stack_address: 0xffffffff010000000,
[ INFO]: crates/boot/src/main.rs@029:     kernel_stack_auto_grow: 0x0,
[ INFO]: crates/boot/src/main.rs@029:     kernel_stack_size: 0x200,
[ INFO]: crates/boot/src/main.rs@029:     physical_memory_offset: 0xfffff80000000000,
[ INFO]: crates/boot/src/main.rs@029:     kernel_path: "\\KERNEL.ELF",
[ INFO]: crates/boot/src/main.rs@029:     cmdline: "",
[ INFO]: crates/boot/src/main.rs@029:     load_apps: false,
[ INFO]: crates/boot/src/main.rs@029: }
[ INFO]: crates/boot/src/fs.rs@050: Load file "KERNEL.ELF" to memory, size = 140144
[ INFO]: crates/boot/src/main.rs@067: Ready to load ELF...
[ INFO]: crates/boot/src/main.rs@078: ELF loaded successfully!
[ INFO]: crates/boot/src/fs.rs@065: Free ELF file, pages = 35, addr = 0x000000004078000
[ INFO]: crates/boot/src/main.rs@100: Exiting boot services...

__V__      __V__      __V__      __V__      __V__
 \ /  _  _  / /  _  _  \ /  _  _  \ /  _  _  \ /  _  _
// // // // // // // // // // // // // // // // // //
/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_

v0.1.0

[+] Serial Initialized.
qemu-system-x86_64: QEMU: Terminated via GDBstub
```

2.2.4. 调试内核

- 在第一个终端启动 QEMU：

```
make debug
```

- 在另一个终端启动 GDB:

[gdb target/x86_64-unknown-none/release-with-debug/yyos_kernel](#)

-进入 GDB 的黑框界面后，依次输入以下命令：

target remote:1234 #连接虚拟机
layout split #加载源代码界面
b init #在init函数处打断点
c #让程序跑起来

调试页面：

```

crates/kernel/src/lib.rs
 9 #[macro_use]
10 pub mod utils;
11 pub use utils::*;
12
13 mod drivers;
14
15 use boot::BootInfo;
16 use uefi::{Status, runtime::ResetType};
17
18 pub fn init(boot_info: &'static BootInfo) {
19     unsafe {
20         // set uefi system table
21         uefi::table::set_system_table(boot_info.system_table.cast().as_ptr());
22     }
23
24     drivers::serial::init(); // init serial output
25     logger::init(); // init logger system
26 }

```

```

B+> 0xffffffff0000017f9 <_RNvCslM0qG26HKL7_11ysos_kernel4init+9> mov (%rdi),%rdi
0xffffffff0000017fc <_RNvCslM0qG26HKL7_11ysos_kernel4init+12> call 0xffffffff00000000
0xffffffff000001801 <_RNvCslM0qG26HKL7_11ysos_kernel4init+17> movzbl 0x47fc(%rip),%
0xffffffff000001808 <_RNvCslM0qG26HKL7_11ysos_kernel4init+24> cmp $0x2,%al
0xffffffff00000180a <_RNvCslM0qG26HKL7_11ysos_kernel4init+26> jne 0xffffffff00000000
0xffffffff000001810 <_RNvCslM0qG26HKL7_11ysos_kernel4init+32> call 0xffffffff00000000
0xffffffff000001815 <_RNvCslM0qG26HKL7_11ysos_kernel4init+37> movzwl 0x47e6(%rip),%
0xffffffff00000181c <_RNvCslM0qG26HKL7_11ysos_kernel4init+44> lea 0x3(%rcx),%esi
0xffffffff00000181f <_RNvCslM0qG26HKL7_11ysos_kernel4init+47> lea 0x2(%rcx),%edi
0xffffffff000001822 <_RNvCslM0qG26HKL7_11ysos_kernel4init+50> lea 0x1(%rcx),%r8d
0xffffffff000001826 <_RNvCslM0qG26HKL7_11ysos_kernel4init+54> mov %ecx,%r9d
0xffffffff000001829 <_RNvCslM0qG26HKL7_11ysos_kernel4init+57> add $0x4,%r9d
0xffffffff00000182d <_RNvCslM0qG26HKL7_11ysos_kernel4init+61> mov %r8d,%edx
0xffffffff000001830 <_RNvCslM0qG26HKL7_11ysos_kernel4init+64> xor %eax,%eax
0xffffffff000001832 <_RNvCslM0qG26HKL7_11ysos_kernel4init+66> out %al,(%dx)
0xffffffff000001833 <_RNvCslM0qG26HKL7_11ysos_kernel4init+67> mov %esi,%edx
0xffffffff000001835 <_RNvCslM0qG26HKL7_11ysos_kernel4init+69> mov $0x80,%al

```

```

remote Thread 1.1 (src) In: ysos kernel::init L21 PC: 0xffffffff0000017f9
Dump of assembler code for function _RNvCslM0qG26HKL7_11ysos_kernel4init:
0xffffffff0000017f0 <+0>: push %rbp
0xffffffff0000017f1 <+1>: mov %rsp,%rbp
0xffffffff0000017f4 <+4>: push %rbx
0xffffffff0000017f5 <+5>: sub $0x78,%rsp
=> 0xffffffff0000017f9 <+9>: mov (%rdi),%rdi
0xffffffff0000017fc <+12>: call 0xffffffff000002020 <_RNvMntCschOVYFhrwIQ_4uefi5table
16set_system_table>
0xffffffff000001801 <+17>: movzbl 0x47fc(%rip),%eax # 0xffffffff000000604 <_RNv
ntntCslM0qG26HKL7_11ysos_kernel7drivers6serial6SERIAL+4>
0xffffffff000001808 <+24>: cmp $0x2,%al
0xffffffff00000180a <+26>: jne 0xffffffff000001b6e <_RNvCslM0qG26HKL7_11ysos_kernel
4init+894>
0xffffffff000001810 <+32>: call 0xffffffff000001be0 <_RNvMntntCslM0qG26HKL7_11ysos_ke
--Type <RET> for more, q to quit, c to continue without paging--Quit
(gdb) layout split
(gdb)

```

用 QEMU 的 monitor 模式查看内存布局：

(qemu) info mem

0000000000000000-0000000004a00000 0000000004a00000 -rw
0000000004a00000-0000000004c00000 0000000000200000 -r-
0000000004c00000-0000000005800000 0000000000c00000 -rw
0000000005800000-0000000005e00000 0000000000600000 -r-
0000000005e00000-0000010000000000 000000ffa200000 -rw
ffff800000000000-ffff810000200000 0000010000200000 -rw
fffff00000000000-fffff0000004000 0000000000004000 -r-
fffff0000004000-fffff0000007000 0000000000003000 -rw
fffff01000000000-fffff0100200000 0000000000200000 -rw

(qemu) info mtree

address-space: i440FX

0000000000000000-ffffffffffffff (prio 0, i/o): bus master container
0000000000000000-ffffffffffffff (prio 0, i/o): alias bus master @system 0000000000000000-ffffffffffffff

address-space: I/O

0000000000000000-000000000000ffff (prio 0, i/o): io
0000000000000000-0000000000000007 (prio 0, i/o): dma-chan
0000000000000008-000000000000000f (prio 0, i/o): dma-cont
0000000000000020-0000000000000021 (prio 0, i/o): pic
0000000000000040-0000000000000043 (prio 0, i/o): pit
0000000000000060-0000000000000060 (prio 0, i/o): i8042-data
0000000000000061-0000000000000061 (prio 0, i/o): pcspk
0000000000000064-0000000000000064 (prio 0, i/o): i8042-cmd
0000000000000070-0000000000000071 (prio 0, i/o): rtc
0000000000000070-0000000000000070 (prio 0, i/o): rtc-index
000000000000007e-000000000000007f (prio 0, i/o): kvmvapic
0000000000000080-0000000000000080 (prio 0, i/o): ioport80
0000000000000081-0000000000000083 (prio 0, i/o): dma-page
0000000000000087-0000000000000087 (prio 0, i/o): dma-page
0000000000000089-000000000000008b (prio 0, i/o): dma-page
000000000000008f-000000000000008f (prio 0, i/o): dma-page
0000000000000092-0000000000000092 (prio 0, i/o): port92
00000000000000a0-00000000000000a1 (prio 0, i/o): pic
00000000000000b2-00000000000000b3 (prio 0, i/o): apm-io

00000000000000c0-00000000000000cf (prio 0, i/o): dma-chan
00000000000000d0-00000000000000df (prio 0, i/o): dma-cont
00000000000000f0-00000000000000f0 (prio 0, i/o): ioportF0
0000000000000170-0000000000000177 (prio 0, i/o): ide
00000000000001ce-00000000000001d1 (prio 0, i/o): vbe
00000000000001f0-00000000000001f7 (prio 0, i/o): ide
0000000000000376-0000000000000376 (prio 0, i/o): ide
0000000000000378-000000000000037f (prio 0, i/o): parallel
00000000000003b4-00000000000003b5 (prio 0, i/o): vga
00000000000003ba-00000000000003ba (prio 0, i/o): vga
00000000000003c0-00000000000003cf (prio 0, i/o): vga

```

00000000000003d4-00000000000003d5 (prio 0, i/o): vga
00000000000003da-00000000000003da (prio 0, i/o): vga
00000000000003f1-00000000000003f5 (prio 0, i/o): fdc
00000000000003f6-00000000000003f6 (prio 0, i/o): ide
00000000000003f7-00000000000003f7 (prio 0, i/o): fdc
00000000000003f8-00000000000003ff (prio 0, i/o): serial
00000000000004d0-00000000000004d0 (prio 0, i/o): elcr
00000000000004d1-00000000000004d1 (prio 0, i/o): elcr
0000000000000510-0000000000000511 (prio 0, i/o): fwcfg
0000000000000514-000000000000051b (prio 0, i/o): fwcfg.dma
0000000000000cf8-0000000000000cfb (prio 0, i/o): pci-conf-idx
0000000000000cf9-0000000000000cf9 (prio 1, i/o): piix-reset-control
0000000000000cf-0000000000000cf (prio 0, i/o): pci-conf-data
0000000000005658-0000000000005658 (prio 0, i/o): vmport
000000000000ae00-000000000000ae17 (prio 0, i/o): acpi-pci-hotplug
000000000000af00-000000000000af0b (prio 0, i/o): acpi-cpu-hotplug
000000000000afe0-000000000000afe3 (prio 0, i/o): acpi-gpe0
000000000000b000-000000000000b03f (prio 0, i/o): piix4-pm
000000000000b000-000000000000b003 (prio 0, i/o): acpi-evt
000000000000b004-000000000000b005 (prio 0, i/o): acpi-cnt
000000000000b008-000000000000b00b (prio 0, i/o): acpi-tmr
000000000000b100-000000000000b13f (prio 0, i/o): pm-smbus
000000000000c000-000000000000c00f (prio 1, i/o): piix-bmdma-container
000000000000c000-000000000000c003 (prio 0, i/o): piix-bmdma
000000000000c004-000000000000c007 (prio 0, i/o): bmdma
000000000000c008-000000000000c00b (prio 0, i/o): piix-bmdma
000000000000c00c-000000000000c00f (prio 0, i/o): bmdma

```

address-space: cpu-smm-0

```

0000000000000000-ffffffffffffff (prio 0, i/o): memory
0000000000000000-00000000ffffff (prio 1, i/o): alias smram @smram 0000000000000000-00000000ffffff
0000000000000000-ffffffffffffff (prio 0, i/o): alias memory @system 0000000000000000-ffffffffffffff

```

address-space: piix3-ide

```

0000000000000000-ffffffffffffff (prio 0, i/o): bus master container
0000000000000000-ffffffffffffff (prio 0, i/o): alias bus master @system 0000000000000000-ffffffffffffff

```

address-space: VGA

```

0000000000000000-ffffffffffffff (prio 0, i/o): bus master container
0000000000000000-ffffffffffffff (prio 0, i/o): alias bus master @system 0000000000000000-ffffffffffffff

```

address-space: PIIX3

0000000000000000-ffffffffffff (prio 0, i/o): bus master container

0000000000000000-ffffffffffff (prio 0, i/o): alias bus master @system 0000000000000000-ffffffffffff

address-space: PIIX4_PM

0000000000000000-ffffffffffff (prio 0, i/o): bus master container

0000000000000000-ffffffffffff (prio 0, i/o): alias bus master @system 0000000000000000-ffffffffffff

address-space: cpu-memory-0

address-space: memory

0000000000000000-ffffffffffff (prio 0, i/o): system

0000000000000000-0000000005ffff (prio 0, ram): alias ram-below-4g @pc.ram 0000000000000000-0000000005ffff

0000000000000000-ffffffffffff (prio -1, i/o): pci

0000000000a0000-0000000000a000 (prio 2, ram): alias vga.chain4 @vga.vram 0000000000000000-000000000000ffff

0000000000a0000-0000000000b000 (prio 1, i/o): vga-lowmem

0000000000c0000-0000000000d000 (prio 1, rom): pc.rom

0000000000e0000-0000000000ffff (prio 1, rom): alias isa-bios @pc.bios 00000000003e0000-00000000003ffff

0000000080000000-0000000080ffff (prio 1, ram): vga.vram

0000000081010000-00000000810100 (prio 1, i/o): vga.mmio

0000000081010000-000000008101017f (prio 0, i/o): edid

0000000081010400-000000008101041f (prio 0, i/o): vga ioports remapped

0000000081010500-0000000081010515 (prio 0, i/o): bochs dispi interface

0000000081010600-0000000081010607 (prio 0, i/o): qemu extended regs

00000000ff000000-00000000ffff (prio 0, rom): pc.bios

0000000000a0000-0000000000b000 (prio 1, i/o): alias smram-region @pci 0000000000a0000-0000000000b000

0000000000c0000-0000000000c3fff (prio 1, ram): alias pam-rom @pc.ram 0000000000c0000-0000000000c3fff

0000000000c4000-0000000000c7fff (prio 1, i/o): alias pam-pci @pci 0000000000c4000-0000000000c7fff

0000000000c8000-0000000000cbfff (prio 1, i/o): alias pam-pci @pci 0000000000c8000-0000000000cbfff

0000000000cc000-0000000000cffff (prio 1, i/o): alias pam-pci @pci

: alias pam-pci @pci

0000000000cc000-0000000000cffff

0000000000d0000-0000000000d3fff (prio 1, i/o): alias pam-pci @pci 0000000000d0000-0000000000d3fff

0000000000d4000-0000000000d7fff (prio 1, i/o): alias pam-pci @pci 0000000000d4000-0000000000d7fff

0000000000d8000-0000000000dbfff (prio 1, i/o)

0000000000d8000-0000000000dbfff

0000000000dc000-0000000000dffff (prio 1, i/o): alias pam-pci @pci 0000000000dc000-0000000000dffff

00000000000e0000-00000000000e3fff (prio 1, i/o): alias pam-pci @pci
 00000000000e0000-00000000000e3fff
 00000000000e4000-00000000000e7fff (prio 1, i/o): alias pam-pci @pci 00000000000e4000-00000000000e7fff
 00000000000e8000-00000000000ebfff (prio 1, i/o): alias pam-pci @pci 00000000000e8000-00000000000ebfff

 00000000000ec000-00000000000effff (prio 1, i/o): alias pam-pci @pci 00000000000ec000-00000000000effff
 00000000000f0000-00000000000fffff (prio 1, i/o): alias pam-pci @pci 00000000000f0000-00000000000fffff
 0000000fec00000-0000000fec00fff (prio 0, i/o): ioapic
 0000000fed00000-0000000fed003ff (prio 0, i/o): hpet
 0000000fee00000-0000000feefffff (prio 4096, i/o): apic-msi

memory-region: system

0000000000000000-ffffffffffffff (prio 0, i/o): system
 0000000000000000-000000005fffff (prio 0, ram): alias ram-below-4g @pc.ram 0000000000000000-000000005fffff
 0000000000000000-ffffffffffffff (prio -1, i/o): pci
 00000000000a0000-00000000000affff (prio 2, ram): alias vga.chain4 @vga.vram 0000000000000000-000000000000fffff
 00000000000a0000-00000000000bffff (prio 1, i/o): vga-lowmem
 00000000000c0000-00000000000dffff (prio 1, rom): pc.rom
 00000000000e0000-00000000000fffff (prio 1, rom): alias isa-bios @pc.bios 00000000003e0000-00000000003fffff
 0000000080000000-0000000080fffff (prio 1, ram): vga.vram
 0000000081010000-0000000081010fff (prio 1, i/o): vga.mmio
 0000000081010000-000000008101017f (prio 0, i/o): edid
 0000000081010400-000000008101041f (prio 0, i/o): vga ioports remapped
 0000000081010500-0000000081010515 (prio 0, i/o): bochs dispi interface
 0000000081010600-0000000081010607 (prio 0, i/o): qemu extended regs
 00000000ffc00000-00000000ffffffffff (prio 0, rom): pc.bios

```
0000000000a0000-0000000000bffff (prio 1, i/o): alias smram-region @pci 0000000000a0000-0000000000bffff
0000000000c0000-0000000000c3fff (prio 1, ram): alias pam-rom @pc.ram
0000000000c0000-0000000000c3fff
0000000000c4000-0000000000c7fff (prio 1, i/o): alias pam-pci @pci 0000000000c4000-0000000000c7fff
0000000000c8000-0000000000cbfff (prio 1, i/o): alias pam-pci @pci 0000000000c8000-0000000000cbfff
0000000000cc000-0000000000cffff (prio 1, i/o): alias pam-pci @pci 0000000000cc000-0000000000cffff
0000000000d0000-0000000000d3fff (prio 1, i/o): alias pam-pci @pci 0000000000d0000-0000000000d3fff
0000000000d4000-0000000000d7fff (prio 1, i/o): alias pam-pci @pci 0000000000d4000-0000000000d7fff
0000000000d8000-0000000000dbfff (prio 1, i/o): alias pam-pci @pci 0000000000d8000-0000000000dbfff
0000000000dc000-0000000000dffff (prio 1, i/o): alias pam-pci @pci 0000000000dc000-0000000000dffff
0000000000e0000-0000000000e3fff (prio 1, i/o): alias pam-pci @pci 0000000000e0000-0000000000e3fff
0000000000e4000-0000000000e7fff (prio 1, i/o): alias pam-pci @pci 0000000000e4000-0000000000e7fff
0000000000e8000-0000000000ebfff (prio 1, i/o): alias pam-pci @pci 0000000000e8000-0000000000ebfff
0000000000ec000-0000000000effff (prio 1, i/o): alias pam-pci @pci
0000000000ec000-0000000000effff
0000000000f0000-0000000000fffff (prio 1, i/o): alias pam-pci @pci 0000000000f0000-0000000000fffff
00000000fec0000-00000000fec00ff (prio 0, i/o): ioapic
00000000fed0000-00000000fed003ff (prio 0, i/o): hpet
00000000fee0000-00000000feefffff (prio 4096, i/o): apic-msi
```

memory-region: smram

```
0000000000a0000-0000000000bffff (prio 0, ram): alias smram-low @pc.ram 0000000000a0000-0000000000bffff
```

memory-region: pc.ram

```
000000000000000-0000000005fffff (prio 0, ram): pc.ram
```

memory-region: vga.vram

```
0000000080000000-0000000080fffff (prio 1, ram): vga.vram
```

memory-region: pc.bios

```
00000000ffc0000-00000000fffffff (prio 0, rom): pc.bios
```

memory-region: pci

```
000000000000000-fffffffffffff (prio -1, i/o): pci
0000000000a0000-0000000000affff (prio 2, ram): alias vga.chain4 @vga.vram 000000000000000-000000000000fffff
0000000000a0000-0000000000bffff (prio 1, i/o): vga-lowmem
0000000000c0000-0000000000dffff (prio 1, rom): pc.rom
0000000000e0000-0000000000fffff (prio 1, rom): alias isa-bios @pc.bios 00000000003e0000-00000000003fffff
0000000080000000-0000000080fffff (prio 1, ram): vga.vram
0000000081010000-0000000081010fff (prio 1, i/o): vga.mmio
0000000081010000-000000008101017f (prio 0, i/o): edid
0000000081010400-000000008101041f (prio 0, i/o): vga ioports remapped
0000000081010500-0000000081010515 (prio 0, i/o): bochs dispi interface
0000000081010600-0000000081010607 (prio 0, i/o): qemu extended regs
00000000ffc0000-00000000fffffff (prio 0, rom): pc.bios
```

查看内核的加载情况：

```

● (base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1$ readelf -h esp/KERNEL.ELF
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                           ELF64
  Data:                               2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                               EXEC (Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:                0xffffffff0000010d0
  Start of program headers:          64 (bytes into file)
  Start of section headers:         138864 (bytes into file)
  Flags:                               0x0
  Size of this header:                64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:          7
  Size of section headers:           64 (bytes)
  Number of section headers:         20
  Section header string table index: 18
● (base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1$ readelf -l esp/KERNEL.ELF

Elf file type is EXEC (Executable file)
Entry point 0xffffffff0000010d0
There are 7 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz            MemSiz            Flags   Align
LOAD            0x0000000000001000 0xffffffff00000000 0xffffffff00000000
                 0x000000000000062c 0x000000000000062c R       0x1000
LOAD            0x0000000000002000 0xffffffff000001000 0xffffffff000001000
                 0x000000000000261a 0x000000000000261a R E     0x1000
LOAD            0x0000000000005000 0xffffffff000004000 0xffffffff000004000
                 0x0000000000001020 0x0000000000001020 RW      0x1000
LOAD            0x0000000000007000 0xffffffff000006000 0xffffffff000006000
                 0x0000000000000000 0x0000000000000020 RW      0x1000
GNU_RELRO       0x0000000000006000 0xffffffff000005000 0xffffffff000005000
                 0x0000000000000020 0x0000000000000020 R       0x1
GNU_EH_FRAME    0x0000000000001600 0xffffffff000000600 0xffffffff000000600
                 0x000000000000000c 0x000000000000000c R       0x4
GNU_STACK       0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000000 0x0000000000000000 RW      0x0

Section to Segment mapping:
Segment Sections...
00  .rodata .eh_frame_hdr .eh_frame
01  .text
02  .data .got
03  .bss
04  .got
05  .eh_frame_hdr
06
○ (base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1$

```

```
(qemu) info registers -a
```

```
CPU#0
```

```
RAX=ffffffff00000010d0 RBX=ffffffff0100000000 RCX=0000000005e93ea0 RDX=ffffffff01001ffff8
RSI=ffff800000000000 RDI=0000000005e93ea0 RBP=ffffffff01001fff38 RSP=ffffffff01001ffeb8
R8 =0000000000000000 R9 =00000000000000501 R10=00000000056e6880 R11=0000000000000868
R12=00000000040ad040 R13=ffffffff1ffaaffc0 R14=0000000002000000 R15=00000000055ec018
RIP=ffffffff00000017f9 RFL=00000096 [--S-AP-] CPL=0 II=0 A20=1 SMM=0 HLT=0
ES =0030 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
CS =0038 0000000000000000 ffffffff 00af9a00 DPL=0 CS64 [-R-]
SS =0030 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
DS =0030 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
FS =0030 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
GS =0030 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
LDT=0000 0000000000000000 0000ffff 00008200 DPL=0 LDT
TR =0000 0000000000000000 0000ffff 00008b00 DPL=0 TSS64-busy
GDT= 00000000055dc000 00000047
IDT= 0000000005138018 00000fff
CR0=80010033 CR2=0000000000000000 CR3=0000000005801000 CR4=00000668
DR0=0000000000000000 DR1=0000000000000000 DR2=0000000000000000 DR3=0000000000000000
DR6=00000000ffff0fff DR7=0000000000000400
EFER=00000000000000d00
FCW=037f FSW=0000 [ST=0] FTW=00 MXCSR=00001f80
FPR0=0000000000000000 0000 FPR1=0000000000000000 0000
FPR2=0000000000000000 0000 FPR3=0000000000000000 0000
FPR4=0000000000000000 0000 FPR5=0000000000000000 0000
FPR6=0000000000000000 0000 FPR7=8000000000000000 4006
XMM0=0000000000000000 0000000000000000 XMM01=0000000000000000 0000000000000000
XMM02=0000000000000000 0000000000000000 XMM03=0000000000000000 0000000000000000
XMM04=0000000000000000 0000000000000000 XMM05=0000000000000000 0000000000000000
XMM06=0000000000000000 0000000000000000 XMM07=0000000000000000 0000000000000000
XMM08=0000000000000000 0000000000000000 XMM09=0000000000000000 0000000000000000
XMM10=0000000000000000 0000000000000000 XMM11=0000000000000000 0000000000000000
XMM12=0000000000000000 0000000000000000 XMM13=0000000000000000 0000000000000000
XMM14=0000000000000000 0000000000000000 XMM15=0000000000000000 0000000000000000
(qemu) █
```

2.3. 步骤三：回答问题

2.3.1. set_entry 函数做了什么？为什么它是 unsafe 的？

答:它的任务是“写内存”,但写的不是普通数据,而是 CPU 硬件识别的页表项。

其具体操作包括：

- 计算索引：根据传入的虚拟地址，提取出对应层级的索引。
- 构造数据：结合物理页框地址和位标志
- 写入内存:将构造好的 64 位数值写入到页表在内存中对应的偏移地址处。

将其标记为 unsafe 主要有以下三个原因:

- 直接操作物理内存地址：该函数需要将一个整数值（物理地址）写入到特定的内存地址，如果地址计算错误，可能会覆盖内核的代码区、堆栈，甚至修改了不该动的硬件寄存器。这种任意内存写入是极其危险的。

- 破坏内存安全语义:该函数可以通过修改页表,让两个不同的虚拟地址指向同一个物理地址或者撤销某个内存页的映射。如果该函数在程序还在引用这块内存时撤销了映射,后续的访问会导致 Page Fault 或释放后使用,这违反了 Rust 的安全契约。
- 触发未定义行为:修改页表直接影响 CPU 的硬件行为。如果该函数错误地关闭了某个页面的权限位,CPU 会立即崩溃。

2.3.2. `jump_to_entry` 函数做了什么?要传递给内核的参数位于哪里?查询 `call` 指令的行为和 `x86_64` 架构的调用约定,借助调试器进行说明。

答:

1. `jump_to_entry` 到底做了什么?

从汇编层面看,它本质上是一个绝对跳转或函数调用。

地址转换:它将内核 ELF 文件的入口点地址(Entry Point)加载到寄存器中。

上下文切换:它丢弃了 Bootloader 的栈帧,通过修改指令指针寄存器(RIP),让 CPU 开始执行内核的第一条指令。

参数准备:在跳转之前,它必须按照 `x86_64` 调用约定将预先准备好的 `BootConfig` 结构体指针放置在特定的寄存器中。

2. 传递给内核的参数位于哪里?

根据 System V AMD64 ABI(Linux 和大多数开源内核遵循的调用约定),函数调用的前 6 个整数或指针参数依次通过以下寄存器传递:

RDI (第一个参数) — `YatSenOS` 的 `BootConfig` 指针通常就在这里。

RSI (第二个参数)

RDX (第三个参数)

RCX

R8

R9

因此,内核的入口函数会去 RDI 寄存器中寻找那个指向内存映射表、帧缓冲区信息的结构体地址。

3. 深入理解 `call` 指令的行为

在调试器(如 GDB 或 QEMU Monitor)中观察 `call` 指令,你会发现它完成了两件事:

压栈 (Push RIP): 将 `call` 指令下一条指令的地址压入栈中(保存返回地址)。

跳转 (Jump): 将 RIP 设置为目标函数的起始地址。

注意:在 `jump_to_entry` 中,有时我们会使用 `jmp` 而不是 `call`。因为内核启动后永远不会“返回”给 Bootloader,所以不需要在栈上保存返回地址。

2.3.3. `entry_point!` 宏做了什么?内核为什么需要使用它声明自己的入口点?

答:它是一个代码生成器(Wrapper)。当写下 `entry_point!(kernel_main)` 时,这个宏会在底层自动生成一段符合底层硬件和引导程序规范的样板代码。

如果不使用这个宏,直接手写一个函数,内核将面临以下三个致命问题。这也是必须使用该宏的原因:

A. 解决函数名修饰(Name Mangling)问题

问题: Rust 编译器默认会对函数名进行“混淆/修饰”(Mangling),但链接器(Linker)和引导程序在寻找入口点时,通常只认一个固定的名字。

宏的解决方式: 宏会在生成的底层函数上自动加上 `no_mangle` 属性,确保入口符号名在编译后原封不动,保证 Bootloader 能够精准跳转。

B. 统一调用约定(Calling Convention / ABI)

问题: Rust 语言自身的函数调用约定(Rust ABI)是不稳定的,并且与 C 语言标准不同(寄存器使用规则可能不一样)。而 Bootloader 执行跳转时(如上文提到的利用 `rdi` 传递参数),遵循的是标准的 System V AMD64 ABI (C ABI)。如果不用 C ABI 接收参数,内核一启动就会读错寄存器,导致崩溃。

宏的解决方式: 宏强制生成带有 `extern "C"` 签名的入口函数,确保内核接收参数的方式与 Bootloader 传参的方式严丝合缝。

C. 保证内存与类型安全(最核心的 Rust 特色)

问题: 在 C 语言或汇编中,入口点就是一个内存地址,你可以强行把任何东西传给它,编译器不会报错(但这会导致运行时的未定义行为)。Rust 作为一门强调安全的语言,绝不允许这种事情发生。

宏的解决方式: `entry_point!` 宏会在编译期进行严格的类型检查。它要求你传入的 `kernel_main` 函数必须满足特定的签名(通常是接收一个 `&'static BootInfo` 并返回!,即永不返回)。如果你手误写成了 `fn kernel_main(x: i32)`,宏在编译阶段就会直接报错,将致命的内存错位问题扼杀在摇篮里。

总结

`entry_point!` 宏封装了所有脏活累活。它把底层那些危险的、与架构强相关的 `unsafe` 胶水代码隐藏了起来,让内核开发者可以安心地在一个类型安全、签名正确的纯 Rust 函数(`kernel_main`)中开始编写操作系统的核心逻辑。

2.3.4. 如何为内核提供直接访问物理内存的能力?你知道几种方式?代码中所采用的是哪一种?

答:

1. 内核访问物理内存的四种常见方式

A. 恒等映射 (Identity Mapping)

原理：将物理地址 X 直接映射到虚拟地址 X 。

优点：物理地址和虚拟地址完全一样，无需计算，非常直观。

缺点：极其浪费虚拟地址空间，且会占据低端地址（通常用于用户态程序），容易引发地址冲突。现代 OS 仅在 Bootloader 刚启动分页的那一小段时间使用。

B. 固定偏移映射 / 线性映射 (Fixed Offset Mapping / Linear Mapping)

原理：选择一个极高的虚拟地址作为起点（即 Offset, 通常远高于用户态程序的可用地址），将物理地址 X 映射到虚拟地址 $X + \text{Offset}$ 。

优点：转换极其快速(只需加减偏移量), 且能通过一个连续的虚拟地址空间直接访问所有的物理内存。64 位系统的虚拟地址空间巨大，完全放得下整个物理内存。现代 Linux 内核(通过 `page_offset_base`)和绝大多数 64 位教学 OS 都采用这种方法。

缺点：需要在启动阶段就把所有物理内存映射好，占用一部分页表空间（通常使用 2MiB 或 1GiB 大页来优化）。

C. 临时映射 (Temporary Mapping)

原理：内核保留一小块专门的虚拟地址空间。当需要访问某个物理页时，临时修改这块虚拟空间的页表项，将其指向目标物理页；访问完成后解除映射。

优点：节省虚拟地址空间（在 32 位系统中物理内存大于虚拟地址空间时非常有用，比如早期的 Linux 高端内存 HighMem 机制）。

缺点：极其低效。每次访问物理内存都需要修改页表并刷新 TLB。

D. 递归页表映射 (Recursive Page Table Mapping) 原理：一种非常巧妙的技巧，将最高级页表(如 P4)的最后一个表项指向 P4 页表自身的物理地址。

适用场景：专门用于访问和修改页表本身。

这份代码采用的是固定偏移映射：

在 `map_physical_memory` 函数中，遍历了从 0 到 `max_addr` 的所有物理帧，并为它们逐一建立映射：

```
// 计算出的虚拟地址 = 物理帧起始地址 + 预设的 offset
let page = Page::containing_address(VirtAddr::new(frame.start_address().as_u64() + offset));

// 将该虚拟地址页映射到该物理帧
page_table.map_to(page, frame, flags, frame_allocator)...
```

这里利用了 Size2MiB 大页，一口气把所有物理内存映射到了以 `offset` 为起点的高半区虚拟地址中。

在 `load_segment` 函数中，为了把 ELF 文件的数据拷贝到刚刚分配的物理页中，必须在开启分页的情况下往该物理页里写数据。由于直接写物理地址会崩溃，代码利用了刚才建立的偏移映射机制：

```
// dest_ptr 是一个可以直接解引用的虚拟地址
// 它等于：新申请的物理页地址 + 线性映射的 offset
let dest_ptr = (frame.start_address().as_u64() + physical_offset) as *mut u8;

// 然后安全地通过虚拟地址写入数据
copy_nonoverlapping(..., dest_ptr, ...);
```

2.3.5. 为什么 ELF 文件中不描述栈的相关内容？栈是如何被初始化的？它可以被任意放置吗？

答：

1. 为什么 ELF 文件中不描述栈的相关内容？

简而言之:ELF 文件是程序的“静态蓝图”，而栈是程序运行时的“动态草稿纸”。

ELF 存的是“已知”：ELF 文件中包含的代码段(.text)、数据段(.data)和只读数据段(.rodata),在编译时内容和大小就已经完全确定了。

栈处理的是“未知”：栈是用来保存函数调用时的局部变量、返回地址和寄存器状态的。程序在运行前，根本无法预知函数会嵌套调用多少层、会产生多少局部变量。因此，把栈的数据打包进 ELF 文件既不现实，也毫无意义。

2. 栈是如何被初始化的？

在操作系统和底层开发中，栈的初始化分为两个动作：分配内存 和 拨动指针。

• 第一步：分配内存并映射

```
// --- 步骤 4.3: 映射内核栈 ---
elf::map_range(
    config.kernel_stack_address,
    config.kernel_stack_size,
    ...
)
```

Bootloader 通过向页表申请物理页，在虚拟内存中强行“圈”出了一块干净的空间，留作内核栈使用。

• 第二步：设置栈顶指针寄存器

内存有了,CPU 怎么知道去哪里用？在 x86_64 架构下,CPU 只认 rsp(Stack Pointer)寄存器。

```
let stacktop = config.kernel_stack_address + config.kernel_stack_size * 0x1000 - 8;
jump_to_entry(&bootinfo, stacktop);
```

Bootloader 计算出了这段内存的最高地址（因为 x86 架构的栈是向下生长的，从高地址向低地址蔓延），并通过汇编指令将这个 stacktop 的值塞进了 rsp 寄存器。一旦 rsp 被赋值，栈就正

式初始化完毕了！接下来内核执行的第一条 push 或 call 指令，就会自动把数据写进你刚刚映射的那块内存里。

3. 栈可以被任意放置吗？

理论上：可以。只要是一块映射了物理内存且具有读写权限的虚拟地址空间，你把 rsp 指向哪里，哪里就是栈。

工程实践上：不可以随意放，必须遵守以下铁律：

- 绝不能与已有数据重叠：如果你把栈放在了代码段或者数据段附近，随着函数的深入调用，栈向下生长，就会悄无声息地覆盖掉你的内核代码或全局变量，导致系统极其诡异地崩溃。
- 必须满足对齐要求：根据 System V AMD64 ABI,在执行 call 指令进入普通 C/Rust 函数之前，栈指针 rsp 必须是 16 字节对齐的。不对齐会导致使用 SSE/AVX 向量指令（如浮点运算）时直接触发硬件异常。
- 留出安全距离(Guard Page): 为了防止“栈溢出(Stack Overflow)”破坏其他内存，成熟的 OS 通常会在栈的最低地址处放置一个不可读写的空页(Guard Page)。这样一旦栈生长过头碰到了这个页,CPU 会立刻触发 Page Fault 被内核捕获，而不是任由其破坏周围的数据。

2.3.6. 请解释指令 layout asm 的功能。倘若想找到当前运行内核所对应的 Rust 源码，应该使用什么 GDB 指令？

layout asm 是 GDB TUI 模式下的一个命令，其全称是 Layout Assembly(汇编视图)。

核心功能：当你在 GDB 中输入这个指令后，终端终端的上方会分出一个独立的窗口，实时显示当前正在执行的汇编代码。

动态追踪：窗口中会高亮显示程序计数器(rip 寄存器)当前指向的那条汇编指令。当你使用 si(单步执行指令)或 ni 往下走时，高亮条会跟着实时移动。

适用场景：在 OS 开发中，当你刚跳入内核入口点,或者在调试上下文切换、中断处理等纯汇编逻辑时,layout asm 能让你像拥有“X 光眼”一样，清晰地看到 CPU 真正在执行什么机器指令。

为了找到当前运行内核所对应的 Rust 源码，我使用了 layout split 指令,这时 GDB 会将屏幕分为三个区域:上方显示 Rust 源码,中间显示对应的汇编代码,下方是命令行输入区。

```
crates/kernel/src/lib.rs
9 #[macro_use]
10 pub mod utils;
11 pub use utils::*;
12
13 mod drivers;
14
15 use boot::BootInfo;
16 use uefi::{Status, runtime::ResetType};
17
18 pub fn init(boot_info: &'static BootInfo) {
19     unsafe {
20         // set uefi system table
21         uefi::table::set_system_table(boot_info.system_table.cast().as_ptr());
22     }
23
24     drivers::serial::init(); // init serial output
25     logger::init(); // init logger system
26 }

B> 0xffffffff0000017f9 < RNVCSLM0qG26HKL7_11sysos_kernel4init+9> mov    (%rdi),%rdi
0xffffffff0000017fc < RNVCSLM0qG26HKL7_11sysos_kernel4init+12> call   0xffffffff00000000
0xffffffff000001801 < RNVCSLM0qG26HKL7_11sysos_kernel4init+17> movzbl 0x47fc(%rip),%eax
0xffffffff000001808 < RNVCSLM0qG26HKL7_11sysos_kernel4init+24> cmp    %eax,%al
0xffffffff00000180a < RNVCSLM0qG26HKL7_11sysos_kernel4init+26> jne     0xffffffff00000000
0xffffffff000001810 < RNVCSLM0qG26HKL7_11sysos_kernel4init+32> call   0xffffffff00000000
0xffffffff000001815 < RNVCSLM0qG26HKL7_11sysos_kernel4init+37> movzwl 0x47e6(%rip),%eax
0xffffffff00000181c < RNVCSLM0qG26HKL7_11sysos_kernel4init+44> lea     0x3(%rcx),%esi
0xffffffff00000181f < RNVCSLM0qG26HKL7_11sysos_kernel4init+47> lea     0x2(%rcx),%edi
0xffffffff000001822 < RNVCSLM0qG26HKL7_11sysos_kernel4init+50> lea     0x1(%rcx),%rdi
0xffffffff000001826 < RNVCSLM0qG26HKL7_11sysos_kernel4init+54> mov     %ecx,%r9d
0xffffffff000001829 < RNVCSLM0qG26HKL7_11sysos_kernel4init+57> add     %eax,%r9d
0xffffffff00000182d < RNVCSLM0qG26HKL7_11sysos_kernel4init+61> mov     %r8d,%edx
0xffffffff000001830 < RNVCSLM0qG26HKL7_11sysos_kernel4init+64> xor     %eax,%eax
0xffffffff000001832 < RNVCSLM0qG26HKL7_11sysos_kernel4init+66> out     %al,(%dx)
0xffffffff000001833 < RNVCSLM0qG26HKL7_11sysos_kernel4init+67> mov     %esi,%edx
0xffffffff000001835 < RNVCSLM0qG26HKL7_11sysos_kernel4init+69> mov     %eax,%al

remote Thread 1.1 (src) In: ysos kernel::init L21 PC: 0xffffffff0000017f9
Dump of assembler code for function RNVCSLM0qG26HKL7_11sysos_kernel4init:
0xffffffff0000017f0 <+0>: push    %rbp
0xffffffff0000017f1 <+1>: mov     %rsp,%rbp
0xffffffff0000017f4 <+4>: push    %rbx
0xffffffff0000017f5 <+5>: sub     $0x78,%rsp
=> 0xffffffff0000017f9 <+9>: mov     (%rdi),%rdi
0xffffffff0000017fc <+12>: call    0xffffffff000002020 <_RNWttCschoVYHrwIQ_AuefiStable
16set_system_table>
0xffffffff000001801 <+17>: movzbl 0x47fc(%rip),%eax # 0xffffffff000000004 <_RNv
HttCslM0qG26HKL7_11sysos_kernel7drivers6serial6SERIAL+4>
0xffffffff000001808 <+24>: cmp     %eax,%al
0xffffffff00000180a <+26>: jne     0xffffffff000001b6e <_RNvCslM0qG26HKL7_11sysos_kerne
l4init+894>
0xffffffff000001810 <+32>: call    0xffffffff000001be0 <_RNvttCslM0qG26HKL7_11sysos_ke
--Type <RET> for more, q to quit, c to continue without paging--Quit
(gdb) layout split
(gdb) |
```

2.3.7. 假如在编译时没有启用 `DBG_INFO=true`, 调试过程会有什么不同?

答：

编译过程的本质，是将具备高度抽象语义（如变量类型、作用域、函数边界）的源代码，降维映射为抹除了所有结构化特征的线性机器指令与绝对内存地址。在此过程中启用 `DBG_INFO=true` (即向生成的 ELF 文件中注入 DWARF 等格式的调试符号), 其核心技术意义在于构建并保留了一个元数据映射层(Metadata Mapping Layer)。

当调试器接管包含调试符号的程序时，其执行的是基于语义的逆向解析：调试器通过解析 ELF 文件中的 `.debug_info`、`.debug_line` 等特殊节区，将处理器当前指令指针所指向的十六进制物理/虚拟地址, 精准映射回源文件中的对应行号。同时, 利用符号表中静态保留的类型签名(Type Signature), 调试器能够自动计算内存偏移量并执行数据反序列化, 使开发者得以直接观测结构化的高级语言抽象数据模型。此外, 依靠调用帧信息(CFI, Call Frame Information), 调试器能够可靠地进行栈回溯(Stack Unwinding), 还原精确的函数调用关系链。

反之, 若在编译时剥离调试信息, 调试过程将发生严重的语义降级(Semantic Degradation): 由于元数据映射层完全丢失, 调试器被迫退化为纯硬件视角的指令集观测工具。原本具名的函数入口退化为匿名的跳转地址, 类型安全的数据结构退化为无语义标识的连续字节流。在此模式下,

调试工具丧失了语义重构的自动化支持能力,迫使开发者必须手动介入,严格依据底层架构规范与系统级应用二进制接口(如 System V AMD64 ABI),通过人脑推演寄存器寻址规律以及栈基址指针与栈顶指针的相对偏移,以此在机器码的荒原中人工逆向复原程序的控制流与数据流。

2.3.8. 你如何选择了你的调试环境?截图说明你在调试界面(TUI 或 GUI)上可以获取到哪些信息?

我构建的调试环境采用了系统级开发中最经典的“双机调试”(Remote Debugging)架构:

执行端(QEMU 硬件模拟器):作为被调试的“靶机”。通过传入 -s -S 参数,QEMU 在启动时不仅模拟了完整的 x86_64 硬件环境,还在底层开启了一个 GDB Server(监听 1234 端口),并在 CPU 执行第一条指令前将其冻结。

控制端(GDB 调试器):作为调试的“宿主机”。它加载了带有完整调试符号表(Debug Info)的 ELF 文件,并通过 TCP 协议连接到 QEMU。此时,GDB 拥有了对虚拟 CPU 的绝对控制权。

在这个调试界面上,可以看到以下五个维度的核心信息:

- 控制流与代码映射层: Rust 源码视图、汇编指令和调用栈回溯。
- 寄存器状态
- 内存观测: 栈帧数据、堆与全局变量、直接内存转储
- 硬件寻址与页表结构
- 调试控制状态

3. UART 与日志输出

3.1. 步骤一: 知识学习

- 串口: 一种常见的计算机接口,用于在计算机和外部设备之间进行串行数据传输。串口是一种通用的调试接口,几乎所有的计算机和嵌入式设备都提供了串口接口。它提供了低级别的硬件访问能力,可以直接与设备进行通信。因此,串口通常用于低级别的系统调试和硬件调试,例如在操作系统启动之前或操作系统不可用的情况下进行调试。

总而言之,串口本质上是一个最原始的数据通道,对于正在开发的操作系统,它之所以如此重要,主要有以下三大原因:

1. 不需要复杂的驱动

如果要往屏幕打印一个字符:需要初始化显卡、配置显存帧缓冲、加载字体库、计算像素位置,甚至还要写一个完整的驱动程序。这在内存刚启动、连内存分配器都还没写好的早期阶段,是不可能的。

如果要往串口打印一个字符 :在 X86 架构下 ,只需要一条极其简单的汇编指令 标准的 COM1 串口被硬编码在 CPU 的 I/O 端口 0x3F8 上。只需要把字符塞进这个端口 ,硬件芯片(UART)就会自动帮你把它发送出去。

2. 内核的调试窗口

因为向串口发送数据极度依赖底层硬件 ,且完全不依赖操作系统的其他高级子系统 ,它成为了系统崩溃时的最佳汇报工具。当内核发生严重的 Page Fault 或 Panic ,导致整个系统锁死、屏幕直接卡或黑屏时 ,串口通常依然能工作。内核开发者会在崩溃前一瞬间 ,把寄存器状态和错误日志往串口里塞。这些信息就是排查机器死机的唯一线索。

3. QEMU 中的“幽灵串口”

在 QEMU 虚拟机里 ,一切硬件都是用软件模拟出来的。

- UART :在串行交互界面上负责对数据完成编解码的硬件芯片。它采用异步通信和全双工通信。芯片的核心工作是做“并转串”和“串转并”的翻译官。
- COM :主板上通常会预留几个 UART 硬件接口。为了方便软件去找到它们 ,系统给它们起了固定名字 :COM1、CON2 等。更重要的是 ,在 x86 架构下 ,COM1 对应着一个固定的 I/O 端口基地址 :0x3F8。
- x86 I/O 端口

CPU 与计算机外部 I/O 设备的常见交互模式分为 内存映射 I/O(MMIO) 和端口映射 I/O(PIO) 两种。

Memory-mapped I/O (MMIO) 即通过将需要进行交互的 I/O 设备的相关寄存器映射到某一段内存地址空间 ,从而实现对 I/O 设备的访问。在启用虚拟内存机制的系统中 ,这些内存空间同样需要通过虚拟地址进行访问。

Port-mapped I/O 即将 I/O 设备的相关寄存器编址在相对与内存地址独立的地址空间 ,并使用专门的指令与 I/O 设备进行交互。在 x86 系统中 ,I/O 端口的地址空间为 0x0000 - 0xFFFF ,可以通过 in 和 out 指令进行访问。

3.2. 步骤二：串口驱动

3.2.1. 在考虑 IO 设备驱动 (SerialPort) 的设计时 ,需要考虑如下问题 :

3.2.1.1. 为了描述驱动的状态 ,需要存储哪些数据 ?

答 :轮询模式下只需要存储一个基址端口号(如 0x3F8) ;中断与缓冲模式下还需要增加软件层面的状态数据 ,例如发送/接收环形缓冲区、初始化标志位。

3.2.1.2. 需要如何与硬件进行交互 ?

答 :操作系统必须跨越软件的边界吗 ,直接对硬件发出电信号。在 x86 架构中 ,主要通过端口映射 I/O 来完成 :

- 配置与控制：通过向特定偏移的端口(如基址+1、基址+3)写入控制字，来指挥硬件的行为。比如开启 DLAB 标志位、设置波特率除数、配置 8N1 数据格式。
- 状态探测：硬件处理速度远比 CPU 慢。在发送数据前，必须读取线路状态寄存器，检查发送缓冲区是否为空；在接受数据前，检查是否有数据就绪。
- 数据读写：使用底层的 `inb` 和 `outb` 汇编指令。在 Rust 中，这通常被安全地封装成了 `Port::read()` 和 `Port::write()` 方法。

3.2.1.3. 与硬件交互的过程中，需要考虑哪些并发问题？

答：

- 多核/多线程竞争：必须使用互斥锁将整个驱动对象包裹起来。获取锁才能发送，发完才能释放。
- 中断引发的死锁：在获取串口锁之前，必须暂时关闭当前 CPU 的本地中断。等打印完毕释放锁后，再恢复中断状态。

3.2.1.4. 驱动需要向内核提供哪些接口？

答：优秀的驱动程序应该把端口计算和硬件轮询藏在里面，向内核提供干净、高级的抽象接口：

- 生命周期接口

`init()`：供内核在极早期的启动阶段调用，唤醒并配置硬件。

- 底层原语接口

`send_byte(u8) / receive_byte() -> Option`：提供最基础的单字节吞吐能力。

- 高级格式化接口

实现 `core::fmt::Write trait`：这是 Rust 内核生态的点睛之笔。只要实现了 `write_str` 方法，驱动就能与 Rust 强大的格式化引擎无缝对接。内核的其他模块只需要愉快地使用 `println!` 或 `info!("value: {}", x)`，底层的转换、缓冲、发往端口的操作全部由驱动自动代劳。

3.2.2. 完成 `uart16550` 驱动：

```

pub fn init(&self) {
    let mut line_control_port = Port::new(self.port + 3);
    let mut fifo_control_port = Port::new(self.port + 2);
    let mut interrupt_enable_port = Port::new(self.port + 1);
    let mut modem_control_port = Port::new(self.port + 4);

    unsafe {
        // 1. 禁用所有中断
        interrupt_enable_port.write(0x00u8);

        // 2. 设置波特率 (115200)
        // 开启 DLAB (Divisor Latch Access Bit)
        line_control_port.write(0x80u8);
        // 设置分频器 (115200 波特率的分频值为 1)
        Port::<u8>::new(self.port).write(0x01u8); // 低 8 位
        Port::<u8>::new(self.port + 1).write(0x00u8); // 高 8 位

        // 3. 设置数据格式: 8 数据位, 无校验, 1 停止位
        line_control_port.write(0x03u8);

        // 4. 启用并重置 FIFO 缓冲区
        fifo_control_port.write(0xC7u8);

        // 5. 设置调制解调器控制位 (DTR, RTS, Out2)
        modem_control_port.write(0x0Bu8);
    }
}

```

```

/// Sends a byte on the serial port.
pub fn send(&mut self, data: u8) {
    let mut status_port = Port::<u8>::new(self.port + 5);
    let mut data_port = Port::<u8>::new(self.port);

    unsafe {
        // 关键：轮询检查 Line Status Register (LSR)
        // 检查第 5 位 (0x20) 是否为 1 (Transmitter Holding Register Empty)
        // 只有当缓冲区为空时，才能安全地写入下一个字节
        while (status_port.read() & 0x20) == 0 {
            core::hint::spin_loop();
        }
        // 向数据寄存器写入字节
        data_port.write(data);
    }
}

```

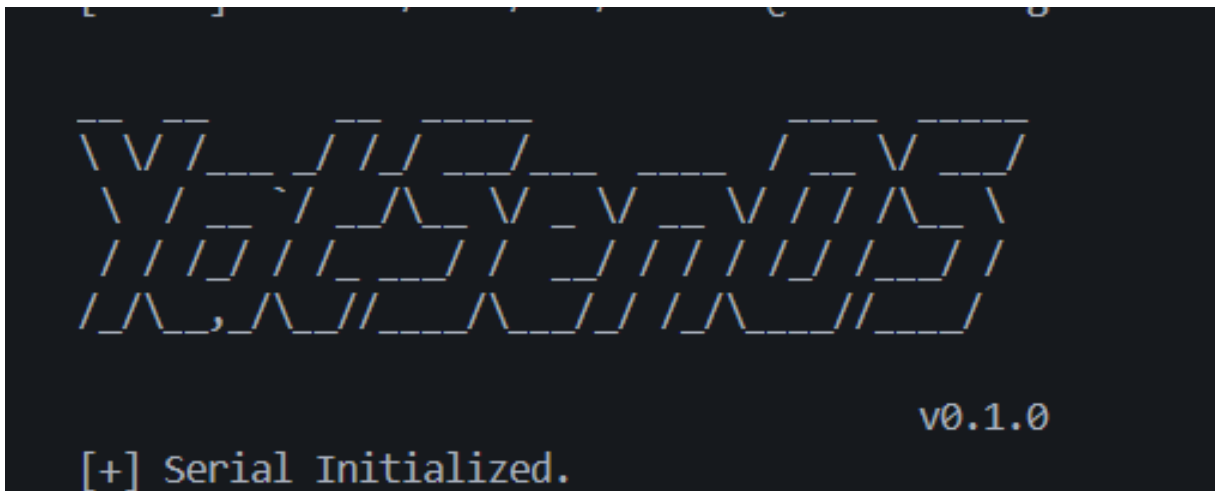
```

/// Receives a byte on the serial port no wait.
pub fn receive(&mut self) -> Option<u8> {
    let mut status_port = Port::<u8>::new(self.port + 5);
    let mut data_port = Port::<u8>::new(self.port);

    unsafe {
        // 检查第 0 位是否为 1 (Data Ready)
        if (status_port.read() & 1) == 0 {
            None
        } else {
            Some(data_port.read())
        }
    }
}

```

在调用 `drivers::serial::init()` 后，能正常看到 `[+] Serial Initialized.` 的输出，说明串口驱动已经成功初始化。



3.3. 步骤三：日志输出

补充下述代码：

```
use log::{Level, Metadata, Record};

pub fn init() {
    static LOGGER: Logger = Logger;
    log::set_logger(&LOGGER).unwrap();

    // 配置日志框架的最大输出级别：
    // 在内核开发的早期阶段，设置为 Trace，这样就能看到包括底层的每一步调试信息。
    log::set_max_level(log::LevelFilter::Trace);

    info!("Logger Initialized.");
}
```



```

struct Logger;

impl log::Log for Logger {
    fn enabled(&self, metadata: &Metadata) -> bool {
        // 判断当前拦截到的日志级别，是否低于或等于我们全局设定的最大级别
        metadata.level() <= log::max_level()
    }

    fn log(&self, record: &Record) {
        if self.enabled(record.metadata()) {
            // 为不同级别分配终端控制台颜色
            let color_code = match record.level() {
                Level::Error => 31,
                Level::Warn => 33,
                Level::Info => 32,
                Level::Debug => 36,
                Level::Trace => 90,
            };

            // 提取产生该日志的代码文件路径和行号
            // 使用 unwrap_or 提供默认值，防止偶尔获取不到信息导致内核 Panic
            let file = record.file_static().unwrap_or("unknown");
            let line = record.line().unwrap_or(0);

            // 组合并输出日志
            // \x1b[{}m : 告诉终端接下来的字用什么颜色显示
            // \x1b[0m : 打印完后，把终端颜色重置回默认状态，防止后续全变色
            // {:>5} : 让日志级别（如 INFO, WARN）右对齐，占5个字符宽度，显得整齐美观
            println!(
                "\x1b[{}m[{:>5}] [{}:{}] {} \x1b[0m",
                color_code,
                record.level(),
                file,
                line,
                record.args()
            );
        }
    }

    fn flush(&self) {}
}

```


当 kernel 在其 Cargo.toml 中引入 boot 时，如果不加上 `default-features = false`，Cargo 强大的特性合并（Feature Unification）机制就会把 boot 的 UEFI 依赖和执行逻辑全部打包进内核中。这将引发三大致命冲突：

💣 灾难一：编译目标（Target）不兼容

Bootloader 是编译给 `x86_64-unknown-uefi` 这个目标的，它依赖 UEFI 固件。而内核是编译给纯裸机目标 `x86_64-unknown-none` 的。

如果内核拉取了 boot 的默认功能，也就拉取了 UEFI 相关的 crate。而这些 crate 根本无法在 `none`（无操作系统环境）的目标下编译，直接导致内核在编译阶段报出一堆平台不兼容的错误。

💣 灾难二：“天无二日”的语义项冲突 (Lang Items Clash)

boot 包（默认开启时）为了自己能独立运行，内部可能定义了一套自己的 `panic_handler` 或引入了全局的内存机制。

kernel 包也定义了自己的 `panic_handler`（就是你刚改过的彩色报错）和 `global_allocator`（之前的代码里写的 `DummyAllocator`）。

如果在内核里引入了 boot 的默认特征，编译器会立刻崩溃，提示发现重复的 `Panic Handler` 或 `Allocator`。

💣 灾难三：运行时的“逻辑毒药”

正如之前在截图里看到的 `Exiting boot services...`，一旦 Bootloader 跳转到内核，UEFI 固件的服务就已经被彻底关闭并销毁了。如果内核里包含了 boot 中调用 UEFI 服务的代码，哪怕它只是个没被执行的死代码，一旦意外触发或链接器处理不当，CPU 就会试图去调用一个已经不存在的固件地址，导致瞬间且毫无征兆的系统死机。

4.2. 在 `crates/boot/src/main.rs` 中参考相关代码，聊聊 `max_phys_addr` 是如何计算的，为什么要这么做？

4.2.1. 什么是 `max_phys_addr`?

`max_phys_addr` 是处理器物理地址空间中，需要被直接映射覆盖的最高物理地址边界。计算 `max_phys_addr` 的根本目的，是为了在操作系统启动早期安全、高效地构建“物理内存的线性映射”页表体系。

计算它的必要性体现在以下三个方面：

- 确定线性映射的页表构建边界：在现代 `x86_64` 操作系统架构（如 Linux 以及你正在开发的 YatSenOS）中，内核通常会将整个可用的物理地址空间统一映射到一个固定的高位虚拟地址偏移处（即 $\text{Virtual Address} = \text{Physical Address} + \text{OFFSET}$ ）。

Bootloader 负责在跳转到内核主函数（`kernel_main`）之前，建立好这套基础的四级页表（PML4）。为了建立这套映射，代码需要执行内存分配并填写页表项（PTE）。`max_phys_addr` 充当了建表逻辑

辑的循环上限边界。它指示了 Bootloader 必须为 $[0, \text{max_phys_addr})$ 这个闭环范围内的所有物理地址分配虚拟地址映射。

- 防止内存资源的过度浪费：既然需要建立线性映射，为什么不直接把架构支持的理论最大值全部映射了？

在 x86_64 架构下，物理地址上限可以达到 48 位（256TB）甚至 52 位（4PB）。即使采用 2MB 或 1GB 的大页（Huge Pages）进行映射，盲目映射整个理论空间依然会消耗惊人数量的连续物理内存，仅仅用于存放页表本身。

通过动态解析 UEFI Memory Map 并计算出真实的 max_phys_addr ，Bootloader 能够做到按需建表。如果当前硬件环境的物理边界在 8GB，页表构建逻辑在映射完这 8GB 后就会主动停止。这避免了页表过度膨胀，将宝贵的物理内存保留给内核态的堆分配器（Heap Allocator）以及未来的用户态进程使用。

- 规避内核态缺页异常：如果 Bootloader 偷懒，不计算精确的最大值，而是随便估算一个较小的值（例如仅映射前 1GB），会导致灾难性后果。

内核在初始化各种底层外设驱动时，必须通过 $\text{OFFSET} + \text{物理基址}$ 形成的虚拟地址来操作设备寄存器（如 PCIe 设备、显卡、APIC 等高位 MMIO 区域）。如果线性映射因为没有精确计算 max_phys_addr 而未能覆盖这些高位物理地址，当 CPU 的指令指针执行到读写设备寄存器的代码时，MMU（内存管理单元）将在页表树中找不到对应的有效映射（Present Bit 为 0）。

这会立刻触发缺页异常（Page Fault）。在内核启动初期，如果触发由于映射不全导致的缺页异常，通常属于致命错误，会直接导致内核崩溃（Panic）。精确计算 max_phys_addr 是确保内核对所有硬件拥有合法访问权的前提。

4.2.2. max_phys_addr 如何计算？

```
let mmap = uefi::boot::memory_map(..); // 从主板 UEFI 固件获取物理内存分布表

let max_phys_addr = mmap
    .entries() // 1. 遍历内存映射表中的每一个条目 (Memory Descriptor)
    .map(|m| m.phys_start + m.page_count * 0x1000) // 2. 计算每块内存的结束地址
    .max() // 3. 找出所有块中，结束地址最大的那一个
    .unwrap_or(0x1_0000_0000) // 4. 防御性编程：如果没拿到数据，默认给 4GB
    .max(0x1_0000_0000); // 5. 强制托底：就算最大物理地址不足 4GB，也强行拔高到 4GB
```

4.2.3. 为什么要这么做？

- 为了建立完整的“线性物理映射”：内核开启分页后，为了能任意分配和读写物理内存，Bootloader 会利用一个极高虚拟地址作为偏移量（ $\text{physical_memory_offset}$ ），将整块物理内存一口气映射进虚拟地址空间。

但是，如果不遍历 UEFI 内存表，Bootloader 怎么知道物理内存到底有多大呢？

如果映射少了，内核以后分配到高地址的物理页帧时，就会因为没有映射而触发 Page Fault 崩溃。因此，必须找出真正的最高物理地址，一次性映射到底。

- 为什么要强制拉高到至少 4GB？

这是 x86 架构极其特殊的一点：物理内存地址空间 不等于 真实的内存条（RAM）空间。

在 4GB 以下的物理地址空间里，主板偷偷藏了很多硬件设备的寄存器。这种技术叫做 MMIO (Memory-Mapped I/O, 内存映射 I/O)。

例如，高级可编程中断控制器 (APIC) 的默认物理地址在 0xFEE00000。

PCIe 设备的配置空间、显卡的显存 (Framebuffer) 等，通常都被插在 3GB 4GB 之间的这段“空洞” (PCI Hole) 里。

如果你在 QEMU 里只给虚拟机分配了 128MB 的内存，那么 mmap 算出来的实际最大 RAM 地址也就是一百多兆。如果不强制拉高到 4GB，Bootloader 就只会映射这 128MB 的范围。结果就是：当你的内核启动后，想要去读写 APIC 中断控制器或者向显存里画图时，它去访问 3GB 多位置的物理地址，直接就 Page Fault 炸了。

4.3. 串口驱动是在进入内核后启用的，那么在进入内核之前，显示的内容是如何输出的？

在进入内核之前（也就是 Bootloader 运行阶段），你能在终端看到的那些白色的 [INFO] 日志，完全是依赖于 UEFI 固件 (Firmware) 提供的标准服务。

在计算机上电后、你的内核正式接管硬件之前，主板上的 UEFI 固件其实已经为你搭建好了一个临时的、功能相对完备的“微型运行环境”。在这个阶段，日志输出的技术实现路径与内核态截然不同：

1. UEFI System Table (系统表) 与 ConOut

当你的 Bootloader 被 UEFI 固件加载并启动时（即进入 `efi_main` 函数时），固件会向 Bootloader 传递一个极其核心的数据结构：UEFI System Table。

这个系统表里封装了大量固件级别的服务指针，其中就包含了一个名为 ConOut (Console Output) 的指针。该指针指向了 UEFI 规范中定义的 Simple Text Output Protocol (简单文本输出协议)。

2. OutputString 函数调用

在这个协议中，UEFI 提供了一个名为 `OutputString` 的标准函数。在你的 `crates/boot/src/main.rs` 中，当你调用 `info!("Running UEFI bootloader...");` 时，Rust 的 `uefi` crate 在底层做了这样一件事：它将 Rust 的字符串转换成 UEFI 规范要求的 UTF-16 字符数组，然后调用 UEFI 固件提供的 `ConOut->OutputString` 接口。

此时,真正负责和硬件打交道(比如把字符渲染到屏幕像素上,或者将数据转发给 QEMU 终端)的,是主板固件本身(在你的实验环境中,就是 QEMU 加载的那个 OVMF.fd 虚拟固件文件)。Bootloader 只是一个“发号施令者”,并不直接操作底层硬件。

3. “过河拆桥”: Exit Boot Services

这种“借用固件功能”的便利是临时且脆弱的。

在 Bootloader 的最后,你会看到这样一行极其关键的代码:

```
uefi::boot::exit_boot_services(...)
```

这行代码的系统级语义是:“Bootloader 的引导任务已完成,操作系统内核即将全面接管物理内存和硬件权限。”

一旦调用这个函数,UEFI 固件就会执行自我销毁与内存回收。包括 ConOut 在内的绝大多数 UEFI 服务及其占用的内存,都会被瞬间剥离。从这一毫秒开始,OutputString 函数彻底不复存在。

4.4. 在 QEMU 中,我们通过指定 `-nographic` 参数来禁用图形界面,这样 QEMU 会默认将串口输出重定向到主机的标准输出。

4.4.1. 假如我们将 Makefile 中取消该选项,QEMU 的输出窗口会发生什么变化?请观察指令 `make run QEMU_OUTPUT=` 的输出,结合截图分析对应现象。

当你在 Makefile 中取消该选项(相当于剥离了 QEMU 的无头模式)后,整个系统的输出流向会发生根本性的重定向。你会观察到以下两个极其显著的现象:

弹出一个独立的图形化窗口(VGA 显示器):QEMU 会启动其默认的图形前端(如 GTK、SDL 或 Cocoa)。在这个弹出的窗口中,你会看到 UEFI 固件(TianoCore/OVMF)的启动 Logo,随后可能会闪过 Bootloader 阶段调用 UEFI 接口打印的白色文本。然而,当屏幕上打印出 `Exiting boot services...` 并正式跃迁入内核后,这个图形界面的画面将永远定格,或者直接黑屏。

宿主机终端陷入死寂:你敲下启动指令的那个原始终端里,将再也看不到之前那绚丽的带有 ANSI 颜色的内核日志。除了 QEMU 启动时的一些基础提示外,终端没有任何后续输出。

结合内核原理的深度分析:产生这种割裂现象的核心原因在于输出通道的不匹配。

图形窗口为什么卡死?当系统退出 UEFI 服务后,Bootloader 阶段借用的屏幕绘制接口被彻底销毁。由于你的 YatSenOS 目前只写了串口驱动,还没有编写显卡驱动(Framebuffer/VGA),内核根本不知道如何往屏幕的像素阵列里写字。因此,QEMU 模拟的显示器永远停留在 UEFI 移交控制权前的那一帧画面。

终端为什么没字了?内核的 `println!` 是硬编码写入物理端口 `0x3F8`(串口 COM1)的。在带有 `-nographic` 参数时,QEMU 会贴心地把 COM1 的数据线自动接到你启动它的物理终端上。但一旦

移除了该参数，QEMU 默认会把虚拟机的串口映射到一个虚拟控制台（Virtual Console，通常可以在图形窗口按 Ctrl+Alt+2 或 Ctrl+Alt+3 切换查看），切断了与宿主机终端的标准输出（stdout）的联系。你的内核依然在拼命往串口发数据，只是你盯着的物理终端没有插上接收的“数据线”。

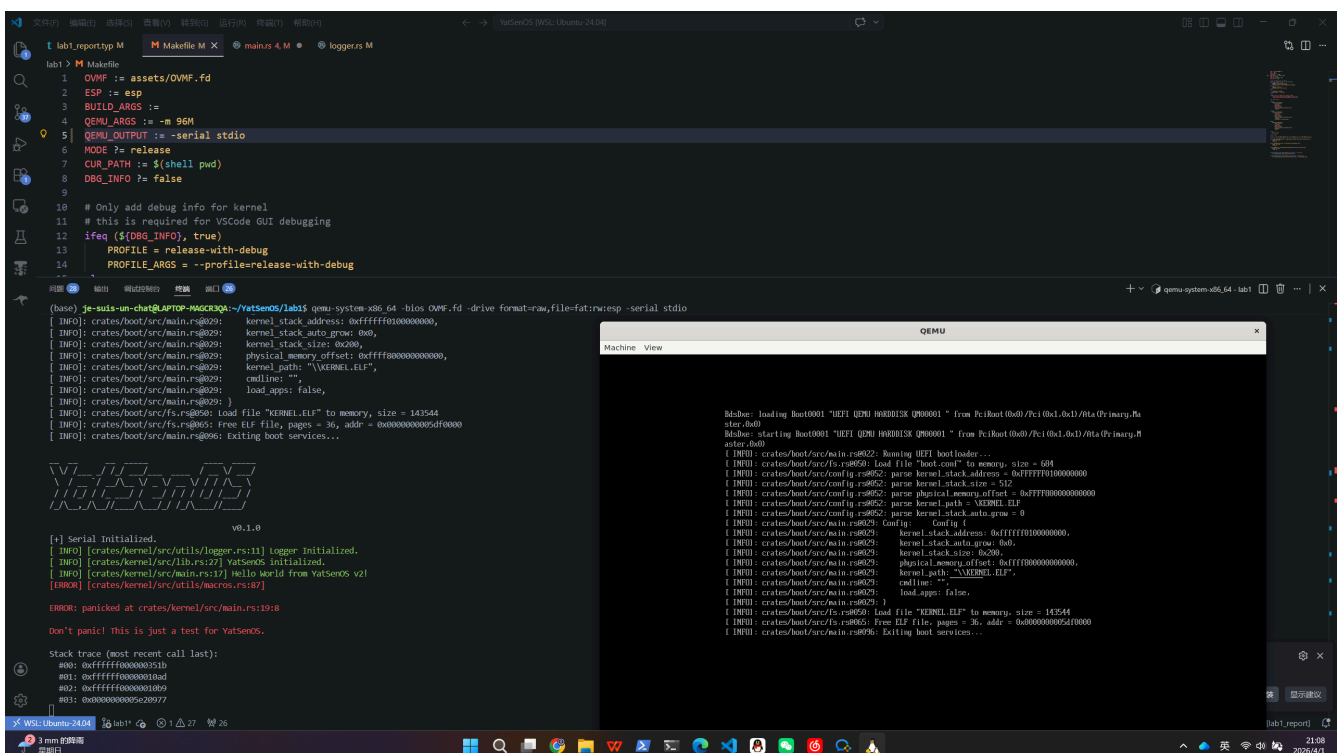
4.4.2. 在移除 -nographic 的情况下，如何依然将串口重定向到主机的标准输入输出？请尝试自行构造命令行参数，并查阅 QEMU 的文档，进行实验。

1. 把 Makefile 文件的 `QEMU_OUTPUT := -nographic` 改成 `QEMU_OUTPUT := -serial stdio`

2. 执行 QEMU 启动指令：

```
qemu-system-x86_64 -bios OVMF.fd -drive format=raw,file=fat:rw:esp -serial stdio
```

3. 可以观察到两种并发的系统状态：宿主机桌面环境和宿主机终端都会显示调试界面。



```
lab1 > Makefile
1 OVMF := assets/OVMF.fd
2 ESP := esp
3 BUILD_ARGS :=
4 QEMU_ARGS := -m 96M
5 QEMU_OUTPUT := -serial stdio
6 MODE ?= release
7 CUR_PATH := $(shell pwd)
8 DBG_INFO ?= false
9
10 # Only add debug info for kernel
11 # this is required for VSCode GUI debugging
12 ifeq ($(DBG_INFO), true)
13     PROFILE = release-with-debug
14     PROFILE_ARGS = --profile=release-with-debug
15 else
16     PROFILE = release
17     PROFILE_ARGS =
18 endif
19
20 # Only add debug info for kernel
21 # this is required for VSCode GUI debugging
22 ifeq ($(DBG_INFO), true)
23     PROFILE = release-with-debug
24     PROFILE_ARGS = --profile=release-with-debug
25 else
26     PROFILE = release
27     PROFILE_ARGS =
28 endif
```

```
(base) je-suis-un-chat@APTDP-HAGC83QA:~/VatSenOS/lab1$ qemu-system-x86_64 -bios OVMF.fd -drive format=raw,file=fat:rw:esp -serial stdio
[INFO] crates/boot/src/main.rs@29: kernel_stack address: 0xffffffff00000000,
[INFO] crates/boot/src/main.rs@29: kernel_stack auto grow: 0x0,
[INFO] crates/boot/src/main.rs@29: kernel_stack size: 0x200,
[INFO] crates/boot/src/main.rs@29: physical_memory.offset: 0xffff800000000000,
[INFO] crates/boot/src/main.rs@29: kernel_path: "\\VXENEL.ELF",
[INFO] crates/boot/src/main.rs@29: cmdline: "",
[INFO] crates/boot/src/main.rs@29: load_apps: false,
[INFO] crates/boot/src/main.rs@29: }
[INFO] crates/boot/src/main.rs@29: }
[INFO] crates/boot/src/main.rs@29: Load file "KERNEL.ELF" to memory, size = 143544
[INFO] crates/boot/src/main.rs@29: Free ELF file, pages = 36, addr = 0x0000000000000000
[INFO] crates/boot/src/main.rs@29: Exiting boot services...
```

4.4.3. 如果你使用 yyos.py 来启动 qemu，可以尝试修改 -o 选项来实现上述功能。

1. 用 `python yyos.py launch --help` 来查看 `-o` 参数，结果显示如下：


```
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1$ python yyos.py launch --help
usage: yyos.py [-h] [-d] [-i] [-m MEMORY] [-o OUTPUT] [-p {release,debug}] [-v] [--dry-run] [--bios BIOS] [--boot
BOOT]
               [--debug-listen DEBUG_LISTEN] [--vfat_disabled]
               {build,clean,launch,run,clippy}
```

Build script for YYOS

positional arguments:

{build,clean,launch,run,clippy}

Task to execute

options:

```
-h, --help      show this help message and exit
-d, --debug     Enable debug for qemu
-i, --intdbg    Enable interrupt output for qemu
-m, --memory MEMORY Set memory size for qemu, default is 96M
-o, --output OUTPUT Set output for qemu, default is -nographic
-p, --profile {release,debug}
                Set build profile for kernel
-v, --verbose   Enable verbose output
--dry-run      Enable dry run
--bios BIOS    Set BIOS path
--boot BOOT    Set boot path
--debug-listen DEBUG_LISTEN
                Set listen address for gdbserver
--vfat_disabled QEMU doesn't support vfat
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/lab1$
```

2.在终端中执行：

```
python yyos.py launch -o "-serial stdio" --vfat_disabled
```

即可实现将串口重定向到主机的标准输入输出。



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验二：中断处理

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 了解中断的作用、中断的分类、中断的处理过程。
2. 启用基于 APIC 的中断，注册 IDT 中断处理程序，实现时钟中断。
3. 注册内核堆分配器。(不实现内存分配算法，使用现有代码赋予内核堆分配能力)
4. 实现串口驱动的输入能力，尝试进行基础的 IO 操作和交互。

二. 实验内容

1. 合并实验代码
2. GDT 与 TSS
3. 注册中断处理程序
4. 初始化 APIC
5. 时钟中断
6. 串口输入中断
7. 用户交互

三. 实验过程

1. 合并实验代码

为了保持每个实验的相对独立性同时实现代码的继承性，我在原有的实验文件夹上新建了一个 Git 分支，并将实验二的代码和上一次实验的代码在新分支上进行合并。这样可以保证实验的连贯性，同时可以随时回退到完成某一次实验时的状态。

值得注意的是，我的整个实验文件夹是 YatSenOS,具体的实验代码在 yatsenos 文件夹下，一开始我是在 YatSenOS 中进行 `cp` 命令的，导致文件结构完全乱套。于是我用 `rm` 命令将拷错的文件删掉，再在子目录中把官方代码拉进来。

这一步可以用 `git status` 监测到：

```
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS$ git status
On branch lab2
Your branch is up to date with 'origin/lab2'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   yatsenos/Cargo.lock
    modified:   yatsenos/crates/kernel/src/memory/gdt.rs

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    yatsenos/lab2_report.typ
```

现在，我可以基于上个实验和官方文档的代码进一步进行操作系统的开发了。

2. GDT 与 TSS

2.1. 步骤一：知识学习

2.1.1. 全局描述符表(GDT)

全局描述符表是 x86 架构下的一种数据结构，用于存储段描述符、定义和管理内存段的访问权限和属性。它是操作系统内核与硬件之间的桥梁，允许操作系统精确地控制不同段的访问权限。

在整个操作系统中，全局描述符表 GDT 只有一张(一个处理器对应一个 GDT)，GDT 可以被放在内存的任何位置，但 CPU 必须知道 GDT 的入口，也就是知道基地址放在哪里，intel 设计者们提供了一个寄存器 GDTR 用来存放 GDT 的入口地址，程序员将 GDT 设定在内存中某个位置之后，可以通过 LGDT 指令将 GDT 的入口地址装入此寄存器。从此以后，CPU 就根据此寄存器中的内容作为 GDT 的入口来访问 GDT 了。

2.1.2. 任务状态段(TSS)

任务状态段(Task State Segment,TSS)也是一种数据结构，它存储有关任务的相关数据。

在 32 位保护模式下，它主要用于存储与任务和中断处理相关的信息。TSS 包含了处理器在任务切换时需要保护和恢复的一些状态信息。每个任务都有一个相应的 TSS，通过任务寄存器(TR)来引用。

在 64 位长模式下，TSS 的结构与 32 位不同，它并不直接与任务切换挂钩，但是它仍然被用于存储特权级栈和中断栈。

2.1.3. 中断描述符表(IDT)

中断描述符表(Interrupt Descriptor Table,IDT)是用于存储中断门描述符的数据结构。

使用 IDT 时，需要为每个可能的中断分配一个唯一的 interrupt 描述符，对于 x86_64 架构，前 32 个中断号被 intel 保留，用于处理 CPU 异常，之后的描述符是用户自定义的中断处理程序，可以被操作系统自定义。

中断门描述符中的地址字段指向相应中断处理程序的入口地址、中断栈等信息。在中断发生时，中断上下文的信息将会被保留在中断栈中，并将处理程序的地址放置到 RIP 寄存器中来进行调用。这一过程遵守了 x86_64 的相关调用约定，因此中断处理程序也需要遵守相关的 ABI 约定，在 x86_64 中，应当通过 `iretq` 指令来结束中断调用。

2.2. 步骤二：参考上下文，在 `src/memory/gdt.rs` 中补全 TSS 的中断栈表，代码如下：

```

lazy_static! {
    static ref TSS: TaskStateSegment = {
        let mut tss = TaskStateSegment::new();

        // initialize the TSS with the static buffers
        // will be allocated on the bss section when the kernel is load
        //
        // DO NOT MODIFY THE FOLLOWING CODE
        tss.privilege_stack_table[0] = {
            const STACK_SIZE: usize = IST_SIZES[0];
            static mut STACK: [u8; STACK_SIZE] = [0; STACK_SIZE];
            let stack_start = VirtAddr::from_ptr(addr_of_mut!(STACK));
            let stack_end = stack_start + STACK_SIZE as u64;
            info!(
                "Privilege Stack : 0x{:016x}-0x{:016x}",
                stack_start.as_u64(),
                stack_end.as_u64()
            );
            stack_end
        };
        // 设置 Double Fault 专用栈
        tss.interrupt_stack_table[DOUBLE_FAULT_IST_INDEX as usize] = {
            const STACK_SIZE: usize = IST_SIZES[1];
            static mut STACK: [u8; STACK_SIZE] = [0; STACK_SIZE];
            let stack_start = VirtAddr::from_ptr(unsafe { addr_of_mut!(STACK) });
            let stack_end = stack_start + STACK_SIZE as u64;
            info!(
                "Double Fault IST : 0x{:016x}-0x{:016x}",
                stack_start.as_u64(),
                stack_end.as_u64()
            );
            stack_end
        };
    };
}

```

```

// 设置 Page Fault 专用栈
tss.interrupt_stack_table[PAGE_FAULT_IST_INDEX as usize] = {
    const STACK_SIZE: usize = IST_SIZES[2];
    static mut STACK: [u8; STACK_SIZE] = [0; STACK_SIZE];
    let stack_start = VirtAddr::from_ptr(unsafe { addr_of_mut!(STACK) });
    let stack_end = stack_start + STACK_SIZE as u64;
    info!(
        "Page Fault IST : 0x{:016x}-0x{:016x}",
        stack_start.as_u64(),
        stack_end.as_u64()
    );
    stack_end
};

tss
};
}

```

3. 注册中断处理程序

3.1. 在 `exception.rs` 中,为各种 CPU 异常注册中断处理程序并完成中断处理函数

- 注册中断处理程序：

```

pub unsafe fn register_idt(idt: &mut InterruptDescriptorTable) {
    idt.divide_error.set_handler_fn(divide_error_handler);
    idt.double_fault
        .set_handler_fn(double_fault_handler)
        .set_stack_index(gdt::DOUBLE_FAULT_IST_INDEX);
    idt.page_fault
        .set_handler_fn(page_fault_handler)
        .set_stack_index(gdt::PAGE_FAULT_IST_INDEX);

    // TODO: you should handle more exceptions here
    // especially general protection fault (GPF)
    // see: https://wiki.osdev.org/Exceptions

    idt.general_protection_fault.set_handler_fn(general_protection_fault_handler)
        .set_stack_index(gdt::GENERAL_PROTECTION_FAULT_IST_INDEX);

    idt.breakpoint.set_handler_fn(breakpoint_handle);

    idt.invalid_opcode.set_handler_fn(invalid_opcode_handle);
}

```

- 完成中断处理函数：


```

fn parse_gp_error(code: u64) -> &'static str{
    if code == 0{
        "Null Selector or General Violation"
    }else if code & 0b1 != 0{
        "External Event (Hardware)"
    }else {
        match (code >>1) & 0b11 {
            0b00 => "GDT Violation",
            0b01 => "IDT Violation",
            0b10 => "LDT Violation",
            0b11 => "IDT Violation (with TI set)",
            _ => "Unknown Segment Violation",
        }
    }
}

pub extern "x86-interrupt" fn general_protection_fault_handler(
    stack_frame: InterruptStackFrame,
    err_code: u64,
){
    panic!(
        "EXCEPTION: GENERAL_PROTECTION_FAULT,ERROR_CODE: {:#x}\n\nDescription: {}\n\nStack_Frame: {:#?}",
        err_code,
        parse_gp_error(err_code),
        stack_frame
    );
}

pub extern "x86-interrupt" fn breakpoint_handle(stack_frame: InterruptStackFrame){
    info!("EXCEPTION: BREAKPOINT\n\nStack_Frame: {:#?}",stack_frame)
}

pub extern "x86-interrupt" fn invalid_opcode_handle(stack_frame: InterruptStackFrame){
    panic!(
        "EXCEPTION: INVALID_OPCODE\n\nStack_Frame: {:#?}",
        stack_frame
    );
}

```

特别地,由于这一步中为 General protection Fault 设置了专用栈,比如修改 gdt.rs 文件,在 TSS 中补全内存分配。

```
pub const GENERAL_PROTECTION_FAULT_IST_INDEX: u16 = 2;

tss.interrupt_stack_table[GENERAL_PROTECTION_FAULT_IST_INDEX as usize] = {
    const STACK_SIZE: usize = IST_SIZES[3];
    static mut STACK: [u8; STACK_SIZE] = [0; STACK_SIZE];
    let stack_start = VirtAddr::from_ptr(unsafe {
        addr_of_mut!(STACK)
    });
    let stack_end = stack_start + STACK_SIZE as u64;
    info!(
        "General Protection Fault IST :0x{:016x}-0x{:016x}",
        stack_start.as_u64(),
        stack_end.as_u64()
    );
    stack_end
};
```

3.1.0.1. 初始化中断系统

```
pub fn init() {
    IDT.load();

    unsafe {
        //初始化本地APIC
        //映射物理地址到虚拟地址并初始化

        let mut lapic = XApic::new(physical_to_virtual(LAPIC_ADDR));
        lapic.init();

        //启用串口中断
        //串口COM1通常对应 IOAPIC 的 IRQ4
        //将其路由至CPU 0(主核)
        enable_irq(4,0);

        //启用时钟中断
        enable_irq(0,0);
        //开启CPU的中断响应开关
        x86_64::instructions::interrupts::enable();
    }

    info!("Interrupts Initialized.");
}
```

4. 初始化 APIC

4.1. 步骤一：知识学习之 APIC 可编程中断控制器

1. 什么是 APIC

APIC(高级可编程中断控制器)是一种关键的硬件组件，旨在管理和协调系统内的中断请求。

APIC 不仅简单地分配中断向量，还提供了更为复杂的功能，如中断优先级、中断屏蔽、中断向量分发等。这使得它成为多处理器系统中协调中断处理的理想选择，并在大型、高性能的计算机系统中发挥关键作用。APIC 的作用不仅仅局限于中断处理，它还协助处理器间通信、同步和系统管理。通过提供多处理器系统中的高级中断控制和协同工作机制，APIC 极大地推动了操作系统和应用程序在复杂环境下的性能表现。

2. APIC 的初始化与编程

在基于 APIC 的系统中，每个 CPU 都由一个本地 APIC(LAPIC)控制。LAPIC 通过 MMIO 方式映射到物理内存中的某个地址空间，这个地址空间称为 LAPIC 寄存器空间。

同时，系统中还有一个 I/O APIC，它是一个独立芯片，负责管理系统中所有 I/O 设备的中断请求。I/O APIC 也通过 MMIO 方式映射到物理内存中的某个地址空间。

x2APIC 是 xAPIC 的变体和扩展，主要改进解决了支持的 CPU 数量和接口性能问题，它们都属于 LAPIC 的实现。在本实验中，我们将使用 xAPIC 来实现 LAPIC 的初始化和编程，在之后的描述中，出现的 APIC 均代指 xAPIC。

4.2. 步骤二：补全检查和初始化函数代码

• 补全 support 检查：

```
impl LocalApic for XApic {  
    fn support() -> bool {  
        CpuId::new()  
            .get_feature_info()  
            .map(|f| f.has_apic())  
            .unwrap_or(false)  
    }  
    // ...  
}
```

• 补全 cpu_init 初始化逻辑

```
fn cpu_init(&mut self) {  
    unsafe {  
        // 1. 启用 Local APIC 并设置伪中断向量 (Spurious Interrupt Vector)  
        // 寄存器 0xF0: 位 8 是软件启用位，0-7 是向量号  
        let spurious_vector = 0xFF; // 通常使用 0xFF 作为伪中断向量  
        self.write(0xF0, spurious_vector | (1 << 8));  
    }  
}
```

```

// 2. 配置 LVT Timer (时钟)
// 寄存器 0x3E0: 设置分频器。0x0B (1011b) 表示 1分频
self.write(0x3E0, 0x0B);

// 寄存 crate::interrupts::consts 里的向量号
let timer_vec = 0x20; // 假设时钟向量号为 0x20
// 寄存器 0x320: 位 17:18 为模式 (01b 是 Periodic)，位 16 是屏蔽位 (0 表示开启)
self.write(0x320, timer_vec | (1 << 17));

// 寄存器 0x380: 设置初始计数值，计数到 0 时触发中断
self.write(0x380, 1000000);

// 3. 禁用不需要的 LVT 线路 (LINT0, LINT1, PCINT, Error)
// 将这些寄存器的位 16 (Mask) 设为 1
self.write(0x350, 1 << 16); // LINT0
self.write(0x360, 1 << 16); // LINT1
self.write(0x340, 1 << 16); // Performance Counter

// 将错误中断映射到特定向量并启用
let error_vec = 0x31; // 假设错误中断向量为 0x31
self.write(0x370, error_vec);

// 4. 清除错误状态寄存器 (ESR)
// 必须连续写入两次才能清除
self.write(0x280, 0);
self.write(0x280, 0);

// 5. 确认并清除所有挂起的中断 (EOI)
self.eoi();

// 6. 发送 Init Level De-assert 信号 (同步仲裁 ID)
// 这是为了在多核环境下同步 APIC 状态
self.write(0x310, 0); // 写 ICR 高 32 位 (目标 CPU 为 0)
// 写 ICR 低 32 位: Level De-assert (位 15=0), All Excl Self (位 18:19=11b)
self.write(0x300, 0x000C8500);
while self.read(0x300) & (1 << 12) != 0 {
    core::hint::spin_loop();
}

// 7. 设置任务优先级寄存器 (TPR)
// 允许所有优先级的中断进入 CPU
self.write(0x080, 0);
}
}

```

在上次实验中，由于内核没有内存分配器无法编译通过，我在 `main.rs` 中定义了一个虚假的分配器 `DummyAllocator`，本次实验中 `lib.rs` 已经调用了 `memory::allocator::init()`，意味着其已经实现了一个真正的、可用的全局堆内存分配器。于是在 `make run` 之前，我需要把之前的 `DummyAllocator` 相关代码注释掉，这样代码就运行成功了，结果如下：

```
000
[ INFO]: crates/boot/src/main.rs@096: Exiting boot services...

__ __
\ \ / _ _ \ / / _ _ \ _ _ \ / _ _ \
 \ / _ _ \ / _ _ \ \ / _ _ \ \ / _ _ \
 / / / / / / / / / / / / / / / / / /
 / \ _ , _ \ _ / / _ _ \ / / _ _ \ / / _ _ \

                                v0.2.0

[+] Serial Initialized.
[ INFO] [crates/kernel/src/utils/logger.rs:11] Logger Initialized.
[ INFO] [crates/kernel/src/memory/address.rs:9] Physical Offset : 0xffff800000000000
[ INFO] [crates/kernel/src/memory/gdt.rs:29] Privilege Stack : 0xffffffff0000812128-0
xxxxxxxxxxxxxx0000813128
[ INFO] [crates/kernel/src/memory/gdt.rs:43] Double Fault IST : 0xffffffff0000813128-0
xxxxxxxxxxxxxx0000814128
[ INFO] [crates/kernel/src/memory/gdt.rs:57] Page Fault IST : 0xffffffff0000814128-0
xxxxxxxxxxxxxx0000815128
[ INFO] [crates/kernel/src/memory/gdt.rs:72] GP Fault IST : 0xffffffff0000815128-0
xxxxxxxxxxxxxx0000816128
[ INFO] [crates/kernel/src/memory/gdt.rs:135] Kernel IST Size : 16.000 KiB
[ INFO] [crates/kernel/src/memory/gdt.rs:136] GDT Initialized.
[DEBUG] [crates/kernel/src/memory/allocator.rs:26] Kernel Heap : 0xffffffff0000011
000-0xffffffff0000811000
[ INFO] [crates/kernel/src/memory/allocator.rs:33] Kernel Heap Size : 8.000 MiB
[ INFO] [crates/kernel/src/memory/allocator.rs:35] Kernel Heap Initialized.
[TRACE] [crates/kernel/src/interrupt/apic/ioapic.rs:74] Enable IOApic: IRQ=4, CPU=0
[TRACE] [crates/kernel/src/interrupt/apic/ioapic.rs:74] Enable IOApic: IRQ=0, CPU=0
.[ INFO] [crates/kernel/src/interrupt/mod.rs:46] Interrupts Initialized.
.[ INFO] [crates/kernel/src/memory/mod.rs:26] Physical Memory : 12.093 GiB
[ INFO] [crates/kernel/src/memory/mod.rs:29] Free Usable Memory : 35.016 MiB
.[ INFO] [crates/kernel/src/memory/mod.rs:38] Frame Allocator initialized.
.[ INFO] [crates/kernel/src/lib.rs:50] Interrupts Enabled.
[ INFO] [crates/kernel/src/lib.rs:52] YatSenOS initialized.
[ INFO] [crates/kernel/src/main.rs:14] Hello World from YatSenOS v2!
.[ERROR] [crates/kernel/src/utils/macros.rs:87]

ERROR: panicked at crates/kernel/src/main.rs:16:8
```

5. 时钟中断

5.1. 步骤一：创建 src/interrupt/clock.rs 文件，补全代码，为 Timer 设置中断处理程序：

```
use super::consts::*;
// 引入原子类型和内存排序规则
use core::sync::atomic::{AtomicU64, Ordering};
use x86_64::structures::idt::{InterruptDescriptorTable, InterruptStackFrame};

pub unsafe fn register_idt(idt: &mut InterruptDescriptorTable) {
    idt[Interrupts::IrqBase as u8 + Irq::Timer as u8]
        .set_handler_fn(clock_handler);
}

pub extern "x86-interrupt" fn clock_handler(_sf: InterruptStackFrame) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        if inc_counter() % 0x10000 == 0 {
            info!("Tick! @{}", read_counter());
        }
        super::ack();
    });
}

static COUNTER: AtomicU64 = AtomicU64::new(0);

#[inline]
pub fn read_counter() -> u64 {
    // 使用 Relaxed 内存序读取当前值
    COUNTER.load(Ordering::Relaxed)
}

#[inline]
pub fn inc_counter() -> u64 {
    // fetch_add 会将变量加 1，并返回相加【之前】的旧值。
    // 为了让 inc_counter 返回最新的值，我们在后面 + 1
    COUNTER.fetch_add(1, Ordering::Relaxed) + 1
}
```

实验结果：

程序每隔一个固定的时间间隔会输出：

```
[ INFO ] [crates/kernel/src/interrupt/clock.rs:14] Tick! @65536
```

5.2. 步骤二：回答问题

如果想要中断的频率减半，应该如何修改？

答：设置一个计数变量 `count`，当 `count % 2 == 0` 时，触发需要执行的定时任务。再修改日志打印的频率，将原本的 `inc_counter() % 0x10000 == 0` 改为 `inc_counter() % 0x20000 == 0`。

6. 串口输入中断

6.1. 步骤一：创建 `crates/kernel/src/drivers/input.rs` 文件

```

use alloc::string::String;
use crossbeam_queue::ArrayQueue;

// 串口输入的是 ASCII 字节
pub type Key = u8;

lazy_static! {
    static ref INPUT_BUF: ArrayQueue<Key> = ArrayQueue::new(128);
}

#[inline]
pub fn push_key(key: Key) {
    if INPUT_BUF.push(key).is_err() {
        warn!("Input buffer is full. Dropping key '{}'", key);
    }
}

#[inline]
pub fn try_pop_key() -> Option<Key> {
    INPUT_BUF.pop()
}

// 3. 实现阻塞式的 pop_key
pub fn pop_key() -> Key {
    loop {
        if let Some(key) = try_pop_key() {
            return key;
        }
        // 如果没有数据，使用 cpu 提示指令降低功耗，自旋等待
        core::hint::spin_loop();
    }
}

// 4. 实现 get_line 函数（处理回车和退格）
pub fn get_line() -> String {
    let mut s = String::with_capacity(128);
    loop {
        let key = pop_key();

```



```

match key {
    b'\r' | b'\n' => {
        // 遇到回车/换行：回显换行，并返回字符串
        if let Some(mut serial) = crate::serial::get_serial() {
            serial.send(b'\n');
        }
        return s;
    }
    0x08 | 0x7F => {
        // 退格键（Backspace 或 Delete）
        if !s.is_empty() {
            s.pop(); // 从字符串中删除
            // 屏幕回显删除：退格(0x08) -> 打印空格盖住 -> 再退格
            if let Some(mut serial) = crate::serial::get_serial() {
                serial.send(0x08);
                serial.send(b' ');
                serial.send(0x08);
            }
        }
    }
    _ => {
        // 普通字符：追加到字符串并回显
        s.push(key as char);
        if let Some(mut serial) = crate::serial::get_serial() {
            serial.send(key);
        }
    }
}
}
}

```

6.2. 步骤二：实现串口中断逻辑

```

use x86_64::structures::idt::{InterruptDescriptorTable, InterruptStackFrame};
use super::{ack, consts::SERIAL_INTERRUPT_VEC};
use crate::serial::get_serial;

pub fn register_idt(idt: &mut InterruptDescriptorTable) {
    // 依然使用你之前在 consts.rs 里定义的正确常量
    idt[SERIAL_INTERRUPT_VEC].set_handler_fn(serial_handler);
}

pub extern "x86-interrupt" fn serial_handler(_st: InterruptStackFrame) {
    receive(); // 调用下方的接收逻辑
    ack();    // 结束中断
}

```

```

/// 仅仅从 UART 读取字符，放入 INPUT_BUFFER (Top Half)
fn receive() {
    if let Some(mut serial) = get_serial() {
        while let Some(byte) = serial.receive() {
            // 将收到的字节推入刚刚写好的缓冲队列
            crate::drivers::input::push_key(byte);
        }
    }
}

```

6.3. 步骤三：开启硬件设备的串口中断

在串口初始化的尾部添加：

```

let mut ier_port: Port<u8> = Port::new(0x3F8 + 1);
unsafe {
    ier_port.write(0x01); // 开启 Data Available Interrupt
}

```

7. 用户交互

修改 crates/kernel/src/main.rs：

```

#![no_std]
#![no_main]

use yyos::*;
use yyos_kernel as yyos;

extern crate alloc;

boot::entry_point!(kernel_main);

pub fn kernel_main(boot_info: &'static boot::BootInfo) -> ! {
    yyos::init(boot_info);

    loop {
        print!("> ");
        let input = input::get_line();

        match input.trim() {
            "exit" => break,
            _ => {
                println!("You said: {}", input);
                println!("The counter value is {}", interrupt::clock::read_counter());
            }
        }
    }

    yyos::shutdown();
}

```

实验结果：

```
(base) je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/yatsenos$ make run
[ INFO]: crates/boot/src/fs.rs:65: Free ELF file, pages = 30, addr = 0x00000000407d000
[ INFO]: crates/boot/src/main.rs:96: Exiting boot services...

      \V//_/_/_/_/_/_/_/_/_/_\V//_/_/_/_/_
       \|/_/_/_/_/_/_/_/_/_\_ \|/_/_/_/_/_
        ///_/_/_/_/_/_/_/_/_// ///_/_/_/_/_
         /\,_/\_/_/_/_/_/_/_/_/\_/_/_/_/_

                                v0.2.0

[+] Serial Initialized.
[ INFO] [crates/kernel/src/utils/logger.rs:11] Logger Initialized.
[ INFO] [crates/kernel/src/memory/address.rs:9] Physical Offset   : 0xffff800000000000
[ INFO] [crates/kernel/src/memory/gdt.rs:29] Privilege Stack    : 0xffffffff0000814368-0xffffffff0000815368
[ INFO] [crates/kernel/src/memory/gdt.rs:43] Double Fault IST   : 0xffffffff0000815368-0xffffffff0000816368
[ INFO] [crates/kernel/src/memory/gdt.rs:57] Page Fault IST    : 0xffffffff0000816368-0xffffffff0000817368
[ INFO] [crates/kernel/src/memory/gdt.rs:72] GP Fault IST     : 0xffffffff0000817368-0xffffffff0000818368
[ INFO] [crates/kernel/src/memory/gdt.rs:135] Kernel IST Size   : 16.000 KiB
[ INFO] [crates/kernel/src/memory/gdt.rs:136] GDT Initialized.
[DEBUG] [crates/kernel/src/memory/allocator.rs:26] Kernel Heap      : 0xffffffff0000013218-0xffffffff0000813218
[ INFO] [crates/kernel/src/memory/allocator.rs:33] Kernel Heap Size : 8.000 MiB
[ INFO] [crates/kernel/src/memory/allocator.rs:35] Kernel Heap Initialized.
[TRACE] [crates/kernel/src/interrupt/apic/ioapic.rs:74] Enable IOApic: IRQ=4, CPU=0
[TRACE] [crates/kernel/src/interrupt/apic/ioapic.rs:74] Enable IOApic: IRQ=0, CPU=0
[ INFO] [crates/kernel/src/interrupt/mod.rs:46] Interrupts Initialized.
[ INFO] [crates/kernel/src/memory/mod.rs:26] Physical Memory      : 12.093 GiB
[ INFO] [crates/kernel/src/memory/mod.rs:29] Free Usable Memory   : 35.008 MiB
[ INFO] [crates/kernel/src/memory/mod.rs:38] Frame Allocator initialized.
[ INFO] [crates/kernel/src/lib.rs:50] Interrupts Enabled.
[ INFO] [crates/kernel/src/lib.rs:52] YatSenOS initialized.
> hello world
You said: hello world
The counter value is 12098
> I'm calling from lab2
You said: I'm calling from lab2
The counter value is 35712
> exit
[ INFO] [crates/kernel/src/lib.rs:56] YatSenOS shutting down.
```

8. 思考题

8.1. 为什么需要在 `clock_handler` 中使用 `without_interrupts` 函数？如果不使用它，可能会发生什么情况？

答：在 `clock_handler` 中使用 `without_interrupts` 的根本原因是为了创建一个“临界区”，防止嵌套中断引发的并发灾难。

简单来说，`without_interrupts` 会在它执行期间向 CPU 发送指令，强行屏蔽掉所有其他的外部硬件中断。等里面的代码执行完，再把中断开关恢复原样。

如果不适用它，内核可能会发生以下三种情况：

1. 经典死锁 (Deadlock)：内核直接卡死

这是在编写 Rust OS（如 YatSenOS 或 Phil Opp 的 `blog_os`）时最常遇到的崩溃原因。

`clock_handler` 里调用了 `info!("Tick!...")`。在内核中，为了防止多行日志打印时字符互相穿插，`info!` 宏底层一定会使用一个自旋锁（Spinlock）来锁住串口设备（Serial Port）。

如果不关中断，考虑以下灾难场景：

主程序（或者另一个优先级低的中断）正在打印日志，它刚刚获取了串口的锁。

就在这时，时钟中断（IRQ0）触发了！CPU 立刻暂停主程序，跳进 `clock_handler`。

在 `clock_handler` 中，也调用了 `info!`，试图获取同一个串口的锁。

死锁诞生：`clock_handler` 发现锁被占用了，于是开始死循环等待（自旋）；但占用锁的主程序必须等 `clock_handler` 执行完才能恢复运行去释放锁。两者永远互相等待，整个操作系统瞬间卡死。

2. 数据竞争与状态破坏 (Data Race)

`clock_handler` 中调用了 `super::ack()` 来告诉中断控制器（APIC 或 PIC）本次中断处理完毕，同时也可能修改其他全局变量。

如果中断没有被屏蔽，当时钟处理函数执行到一半时，突然键盘被按下了，触发了键盘中断。键盘中断的处理程序如果刚好也要读取/修改与时钟共享的内核状态（比如某些时间戳记录、进程调度队列），两边的数据就会交叉覆盖，导致难以复现的幽灵 Bug。

3. 栈溢出 (Stack Overflow)

虽然时钟中断的频率很高，但处理速度通常也很快。然而，如果中断不被屏蔽：如果在时钟中断还没处理完的时候，又来了一个新的中断（甚至是下一个时钟中断），CPU 就会把当前的状态再次压入栈中，去执行新的中断。如果系统负载极高，中断像俄罗斯套娃一样无限嵌套，内核栈（通常只有十几 KB）很快就会被耗尽，导致系统直接崩溃引发 Double Fault（双重异常）。

8.2. 考虑时钟中断进行进程调度的场景，时钟中断的频率应该如何设置？太

快或太慢的频率会带来什么问题？请分别回答。

答：

1. 时钟中断频率应该如何设置？

时钟中断频率没有一个“放之四海而皆准”的固定值，它取决于操作系统的应用场景。

权衡原则：设置频率的本质是在“任务执行效率”与“交互响应速度”之间寻找平衡点。

常见范围：

桌面系统（如 Ubuntu, Windows）：通常设置为 250Hz - 1000Hz（即每 1ms 到 4ms 触发一次中断）。高频率能保证鼠标点击、窗口拖动等交互操作非常流畅。

服务器系统：通常设置为 100Hz（每 10ms 触发一次）。服务器更看重吞吐量，稍微牺牲一点响应时间以换取更高的计算效率。

嵌入式/实时系统：可能根据具体硬件实时性要求设置得更高（如 1000Hz+），以确保任务在严格的截止时间内完成。

2. 频率设置“太快”（太高）会带来什么问题？

如果时钟中断频率过高（例如设置到 10000Hz，即每 0.1ms 一次中断），会产生以下严重后果：

巨大的 CPU 开销（Overhead）：每次中断发生时，CPU 都必须强制停止当前工作，保存寄存器现场，进入内核态执行中断处理程序。如果频率太快，CPU 会浪费大量比例的时间在“进出中断”这件事上，而不是执行有意义的用户代码。

上下文切换频繁：高频时钟意味着时间片（Time Slice）更短，进程会被频繁切换。频繁的上下文切换会导致 CPU 缓存（L1/L2 Cache）污染，因为刚缓存好的数据还没怎么用，进程就被切走了，新进程又要重新加载缓存，大幅降低执行效率。

功耗增加（电量杀手）：对于移动设备或笔记本，频繁唤醒 CPU 处理中断会阻止 CPU 进入深度的休眠省电模式，导致电池寿命急剧缩短。

3. 频率设置“太慢”（太低）会带来什么问题？

如果频率过低（例如设置为 10Hz，即每 100ms 才一次中断），系统会表现得非常“迟钝”：

交互响应延迟（Latency）：假设你点击了一下鼠标，系统可能需要等当前这个漫长的 100ms 时钟周期结束才能触发调度器处理你的点击。用户会感觉到明显的“卡顿”或输入延迟。

进程调度粒度太粗：调度器对进程运行时间的控制变得非常不精确。比如一个进程只需要运行 5ms，但由于 100ms 才有一次“检查点”，它可能会平白无故占用 CPU 很久，导致其他急需运行的小任务排队等待。

系统时间精度下降：依赖时钟滴答计数的计时器（如 sleep 函数）会变得很不准。如果你想睡眠 10ms，但在 100ms 一跳的系统里，你可能不得不睡满 100ms 才能被唤醒。

8.3. 在进行 receive 操作的时候，为什么无法进行日志输出？如果强行输出日志，会发生什么情况？谈谈你对串口、互斥锁的认识。

答：

一、为什么不能在 receive 中输出日志？强行输出会怎样？

在你的 YatSenOS 中，info! 宏的作用是将字符串打印到屏幕或终端，它的底层必然调用了串口设备（Serial Port）的发送功能。这意味着，串口不仅负责接收键盘输入，还负责输出内核日志，它是一个全局共享的硬件资源。

为了防止多条日志同时打印导致字符乱码拼凑，内核通常会用一个全局互斥锁（在 no_std 内核中通常是自旋锁 Spinlock）将串口保护起来。

如果你在 receive 函数中强行调用 info!，几乎必然会触发以下“死亡连环套”：

场景预设：内核主程序正在愉快地运行，此时恰好执行到一句普通的 `info!("System running...")`。

获取锁：主程序成功获取了串口的全局锁，准备开始向硬件端口发送字符。

灾难降临（中断打断）：就在主程序刚发了两个字符时，你敲击了一下键盘。串口硬件立刻向 CPU 发送 IRQ4 中断请求。

强行挂起：CPU 响应硬件中断，立刻挂起主程序（此时主程序依然紧紧握着串口的锁没有释放），强行跳入你的 `serial_handler`，进而调用 `receive`。

死锁爆发：在 `receive` 函数内部，你强行写了一句 `info!("Received a key")`。`info!` 宏为了打印，试图去获取那个全局的串口锁。

万劫不复的死循环：`receive` 发现锁被占用了，于是开始原地死循环（自旋）等待锁被释放；然而，唯一能释放这个锁的主程序，正被 `receive` 所在的中断上下文无情地压制着，永远得不到 CPU 时间去执行解锁。

结论：强行输出会引发内核级死锁（Deadlock），整个操作系统瞬间卡死，画面定格，连 Panic 的机会都不会有。

二、对“串口（Serial Port）”的认识

在现代编程中，我们习惯了极快的 I/O 和无限的缓冲区，但在写 OS 底层驱动时，串口暴露出硬件最原始的模样：

极其缓慢的字符流：串口是串行通信，一个比特一个比特地发。相对于 CPU 动辄 GHz 的时钟频率，串口的速度慢得像蜗牛。这意味着 CPU 如果要等待串口发完一段长日志，会浪费海量的时钟周期。

极小硬件 FIFO 缓冲区：UART 16550 芯片内部通常只有一个 16 字节的 FIFO（先进先出队列）。如果 CPU 不赶紧把收到的字节读走，或者串口被其他高优先级任务霸占太久，后续敲击的字符就会直接被硬件丢弃（Overrun Error）。

既是矛也是盾的双重身份：如前文所述，它既负责输入也负责输出。这使得它成为了内核中最容易发生冲突的瓶颈点。

这也就是为什么实验指导书让你使用 Top Half & Bottom Half（上半部与下半部）原则：在中断里（Top Half）绝不恋战，不打印日志，甚至不作逻辑处理，以最快的速度把硬件 FIFO 里的字节捞出来，扔进你刚刚写的无锁队列 `crossbeam_queue` 中就赶紧退出。复杂的打印和回显（Bottom Half），交给普通的内核线程去慢慢做。

三、对“互斥锁（Mutex / Spinlock）”的认识

普通的 Mutex（如标准库中的 `std::sync::Mutex`）在获取不到锁时，会让当前线程休眠，让出 CPU 给别的线程。

但在内核中断里，绝对不能休眠！中断没有自己的独立线程上下文，如果在中断里休眠，整个 CPU 核心就跟着睡死了。

因此，在 OS 开发中，我们使用的是自旋锁（Spinlock）。获取不到锁时，它就原地死循环（`while !locked {}`）。

为了安全地在中断和主程序之间使用自旋锁，OS 理论给出了一个铁律（Locking Rule）：

“当一段普通代码试图获取一个【可能在中断处理程序中被使用】的锁时，必须先在硬件层面彻底关闭中断！”

回想一下你在上一个实验中学习的 `without_interrupts(|| { ... })` 闭包。这就是为什么你在修改时钟计数器、向 APIC 发送 `ack()` 时，必须把它包裹在关中断的环境中。

主程序调用 `without_interrupts` 关闭中断（清零 IF 标志）。

主程序获取自旋锁。

执行操作（由于硬件中断被屏蔽，绝对不可能出现中断跑出来抢锁的情况）。

释放自旋锁。

恢复中断标志。

8.4. 输入缓冲区在什么情况下会满？如果缓冲区满了，用户输入的数据会发生什么情况？

答：

一、输入缓冲区在什么情况下会满？

缓冲区的本质是用来“吸收速度差”的。当生产者的生产速度 > 消费者的消费速度，并且持续一段时间后，缓冲区就会满。具体到你的操作系统中，通常有以下几种场景：

1. 突发的大量输入 (Burst Input)

人类打字的速度再快，通常也无法在消费者读取前打满 128 个字符。但如果用户在主机终端上复制并粘贴了一大段文本，或者使用脚本向虚拟机的串口狂发数据。此时，硬件中断会像暴风雨一样密集触发，Top Half 瞬间向队列里塞入几百个字符，瞬间撑爆 128 的容量。

2. 消费者“罢工”或被阻塞 (Consumer Blocked)

如果你的内核主逻辑在忙其他事情，根本没空去调用 `pop_key()` 读取数据。例如：

你的操作系统正在执行一个极其复杂的数学计算（耗时好几秒）。

系统发生了死锁，或者进入了一个没有调用 `get_line()` 的死循环。在这期间，用户哪怕只是慢条斯理地敲了 129 下键盘，缓冲区也会因为“只进不出”而被填满。

3. 频繁且长时间的“关中断”操作

如果你的内核中有很多包裹在 `without_interrupts(|| { ... })` 中的长耗时操作，这会阻止调度器切换到负责读取输入的进程，导致输入数据在缓冲区中不断积压。

二、如果缓冲区满了，用户输入的数据会怎样？

答案是非常残酷但又符合系统设计的：数据会永久丢失（Data Dropped）。

丢失分为两个层面：

1. 软件层面的主动丢弃 (Software Drop)

回顾一下 input.rs 中的这行核心代码：

```
if INPUT_BUF.push(key).is_err() {  
    warn!("Input buffer is full. Dropping key '{}'", key);  
}
```

ArrayQueue 是一个固定大小的队列。当里面已经有 128 个未读字符时，第 129 个字符尝试 push 时会返回 Err。此时，你的代码逻辑捕获了这个错误，打印了一条警告日志，然后什么也没做就结束了函数。这代表着这个字符被内核无情地抛弃了。在用户的视角里，就是“我明明按了键盘，但屏幕上死活不出现那个字母”（即键盘“丢键”）。

2. 硬件层面的溢出错误 (Hardware Overrun Error)

更极端的情况是，如果系统不仅软件缓冲区满了，甚至连中断都被长时间屏蔽，导致 Top Half 的 receive() 都没机会执行。UART 16550 串口芯片内部通常只有一个极其微小的 16 字节硬件 FIFO(先进先出队列)。一旦这 16 个字节被填满，而 CPU 还没有把它们读走，当第 17 个字节从线缆里传过来时，硬件芯片会直接丢弃这个新字节，并在寄存器中悄悄标记一个 Overrun Error(溢出错误)。

8.5. 进行下列尝试，并在报告中保留对应的触发方式及相关代码片段：

8.5.1. 尝试用你的方式触发 Triple Fault, 开启 intdbg 对应的选项，在 QEMU 中查看调试信息，分析 Triple Fault 的发生过程。

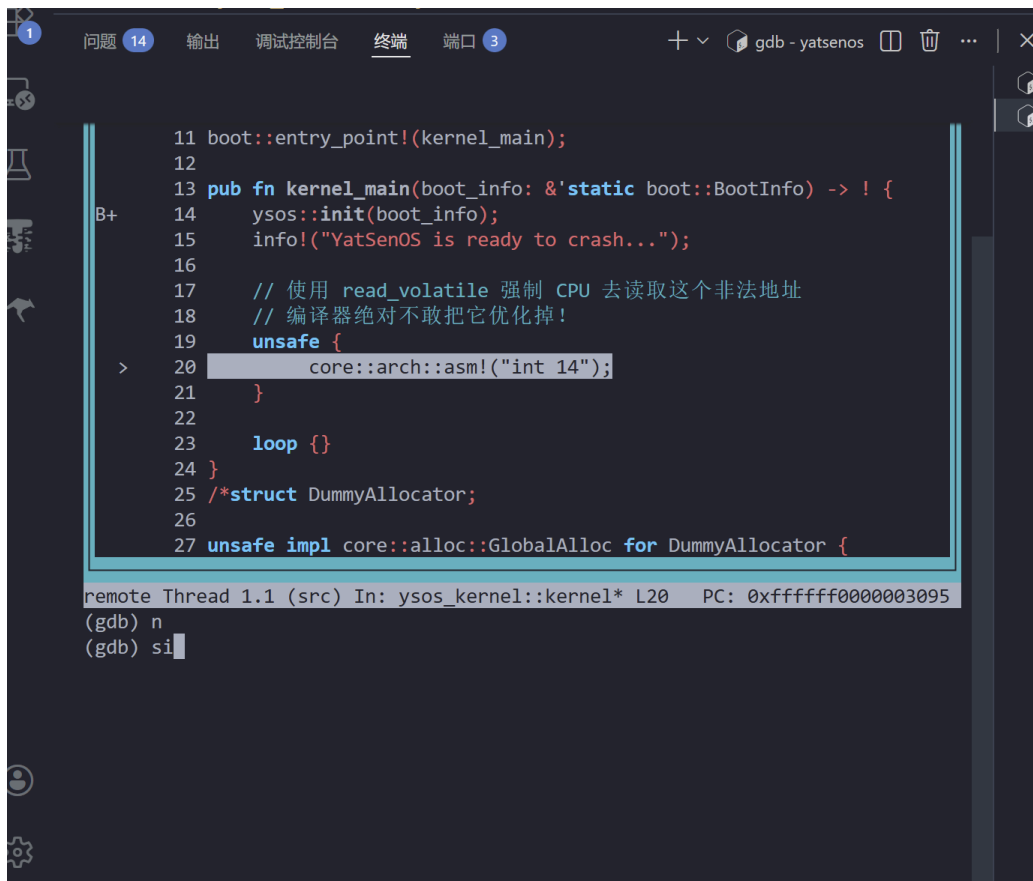
为了触发 triple fault, 我移除了 double fault 和 page fault 处理程序, 并在 kernel_main 里面故意去访问一个不存在的物理地址, 触发缺页异常。

在终端运行 `make intdbg` 之后，终端出现了以下日志信息：

```
check_exception old: 0x8 new 0xb  
Triple fault
```

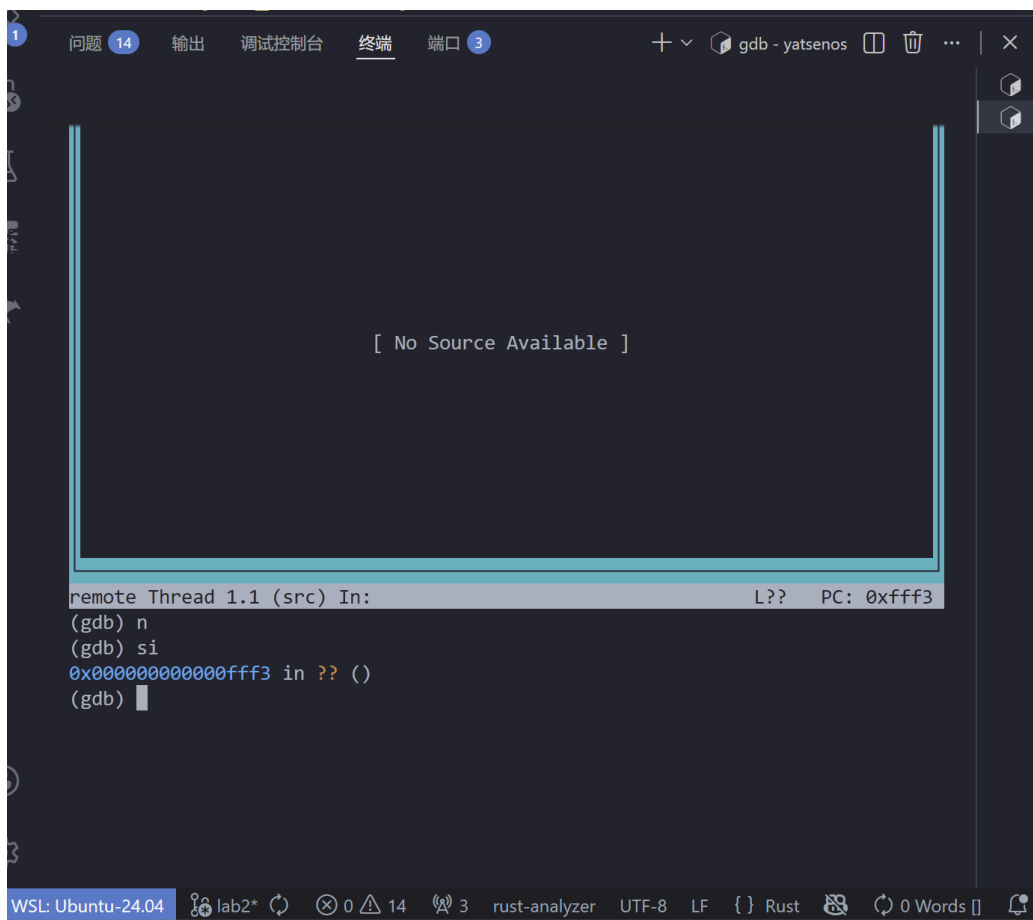
证明我成功触发了 triple Fault。

用 QEMU 调试:



```
1 问题 14 输出 调试控制台 终端 端口 3 + v gdb - yatsenos [icon] [icon] ... [X]
11 boot::entry_point!(kernel_main);
12
13 pub fn kernel_main(boot_info: &'static boot::BootInfo) -> ! {
14     ysos::init(boot_info);
15     info!("YatSenOS is ready to crash...");
16
17     // 使用 read_volatile 强制 CPU 去读取这个非法地址
18     // 编译器绝对不敢把它优化掉!
19     unsafe {
20         core::arch::asm!("int 14");
21     }
22
23     loop {}
24 }
25 /*struct DummyAllocator;
26
27 unsafe impl core::alloc::GlobalAlloc for DummyAllocator {
```

remote Thread 1.1 (src) In: ysos_kernel::kernel* L20 PC: 0xffffffff000003095
(gdb) n
(gdb) si



```
1 问题 14 输出 调试控制台 终端 端口 3 + v gdb - yatsenos [icon] [icon] ... [X]
[ No Source Available ]

remote Thread 1.1 (src) In: L?? PC: 0xffff3
(gdb) n
(gdb) si
0x00000000000000ff3 in ?? ()
(gdb) 
```

WSL: Ubuntu-24.04 lab2* 0 14 3 rust-analyzer UTF-8 LF {} Rust 0 Words

易知,程序在运行到 `core::arch::asm!("int 14")` 这句时崩溃,这时候打开另一个运行 `make debug` 的终端可见到 Triple Fault 警告。

分析 Triple Fault 的发生过程:

第一阶段:初始异常的爆发 (First Fault)

触发:内核在运行过程中遭遇了常规的异常事件(在本次实验中,是我们通过 int 14 汇编指令强制触发的 Page Fault 缺页异常)。

机制:CPU 暂停当前执行流,试图去中断描述符表(IDT)中寻找 14 号中断的处理函数 (page_fault_handler)。

阻碍:由于我们在 IDT 中故意去除了 Page Fault 的注册信息(或者在真实场景中遭遇了严重的栈溢出/页表损坏),CPU 无法成功调用该处理程序。

第二阶段:防线退守与二次沦陷 (Double Fault)

触发:按照 x86 硬件规范,如果在调用异常处理程序的过程中本身又发生了异常,CPU 会立即放弃当前的救救行动,转而呼叫底线救援机制——触发 8 号 Double Fault(双重异常)。

机制:CPU 试图将引发危机的状态压入栈中,并去 IDT 寻找 8 号中断的处理函数 (double_fault_handler)。

绝境:在本次实验中,我们同样拆除了 Double Fault 的 IDT 表项。CPU 发现连这道“最后的防线”也是无效的。

第三阶段:三重异常与硬件重置 (Triple Fault & Hardware Reset)

触发:在试图处理 Double Fault 时,再次遭遇了无法解决的异常(如表项缺失)。

结局:面对“处理异常的异常处理过程也发生了异常”这种逻辑死局,CPU 内部硬件逻辑判定系统状态已处于彻底不可知的混沌之中,软件层面失去任何恢复可能。

拉闸:为了保护硬件,CPU 立即停止取指执行,并向主板总线发出硬件重置信号(拉低 RESET 引脚)。在物理机上表现为电脑瞬间黑屏重启;在 QEMU 模拟器中,表现为触发 CPU Reset 并退出进程。

8.5.2. 尝试触发 Double Fault,观察 Double Fault 的发生过程,尝试通过调试器定位 Double Fault 发生时使用的栈是否符合预期。

把 page fault 的处理函数注释掉,再试图访问一个超过范围的页面,就会触发 double fault.

用调试器定位发生 double fault 的语句:

```
问题 11 输出 调试控制台 终端 端口 3
crates/kernel/src/main.rs
7 use ysos_kernel as ysos;
8
9 extern crate alloc;
10
11 boot::entry_point!(kernel_main);
12
13 pub fn kernel_main(boot_info: &'static boot::BootInfo) -> !{
14     ysos::init(boot_info);
15     info!("YatSenOS is ready to crash...");
16
17     // 1. 触发一次 Page Fault
18     // 由于你没注册 Page Fault Handler, CPU 会试图触发 Double Fault
19     unsafe {
20         info!("I wanna trigger a double fault.");
21         core::ptr::read_volatile(0x0000_1234_5678_9000 as *const u64);
22         info!("double fault finished.");
23     }
}

remote Thread 1.1 (src) In: ysos kernel::kernel_main L21 PC: 0xffffffff000000311c
Note: breakpoint 2 also set at pc 0xffffffff000000300b.
Breakpoint 3 at 0xffffffff000000300b: file crates/kernel/src/main.rs, line 14.
(gdb) c
Continuing.

Breakpoint 2, ysos_kernel::kernel_main (boot_info=0x5e93ea0) at crates/kernel/src/main.rs:14
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb)

Ubuntu-24.04 lab2* 0 11 3 rust-analyzer Update is ready, click to restart. 空格: 2 UTF-8 LF {} 纯文本 0 Words
```

```
问题 11 输出 调试控制台 终端 端口 3
crates/kernel/src/interrupt/exceptions.rs
31 idt.invalid_opcode.set_handler_fn(invalid_opcode_handler);
32 }
33
34 pub extern "x86-interrupt" fn divide_error_handler(stack_frame: InterruptStackFrame) {
35     panic!("EXCEPTION: DIVIDE ERROR\n\n{:#?}", stack_frame);
36 }
37
38 pub extern "x86-interrupt" fn double_fault_handler(
39     stack_frame: InterruptStackFrame,
40     error_code: u64,
41 ) -> ! {
42     panic!(
43         "EXCEPTION: DOUBLE FAULT, ERROR_CODE: 0x{:016x}\n\n{:#?}",
44         error_code, stack_frame
45     );
46 }
47
48 pub extern "x86-interrupt" fn page_fault_handler(
49     stack_frame: InterruptStackFrame,
50     err_code: PageFaultErrorCode,
51 ) {
52     panic!(
53         "EXCEPTION: PAGE FAULT, ERROR_CODE: {:?}\n\nTrying to access: {:#x}\nStack_Frame: {:#?}",
54         err_code,
55         Cr2::read().unwrap_or(VirtAddr::new_truncate(0xdeadbeef)),
56         stack_frame
57     );
58 }
59
60 fn parse_gp_error(code: u64) -> &'static str{

remote Thread 1.1 (src) In: ysos kernel::interrupt::exceptions::double_fault_handler L42 PC: 0xffffffff000000500b
(gdb) n
(gdb) n
ysos_kernel::interrupt::exceptions::double_fault_handler (
    stack_frame=error reading variable: Cannot access memory at address 0xffffffff0000000000, error_code=99171920)
    at crates/kernel/src/interrupt/exceptions.rs:38
(gdb) n

Breakpoint 1, ysos_kernel::interrupt::exceptions::double_fault_handler (stack_frame=..., error_code=0)
    at crates/kernel/src/interrupt/exceptions.rs:42
(gdb) n
```

程序在运行到触发缺页异常的语句的时候，由于找不到缺页异常处理函数，触发了 double fault.

在另一个终端中可见到 double fault 处理函数的输出：

```
[ERROR] [crates/kernel/src/utils/macros.rs:87]

ERROR: panicked at crates/kernel/src/interrupt/exceptions.rs:42:6

EXCEPTION: DOUBLE FAULT, ERROR_CODE: 0x0000000000000000

InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0xffffffff00000311c,
  ),
  code_segment: SegmentSelector {
    index: 1,
    rpl: Ring0,
  },
  cpu_flags: RFlags(
    INTERRUPT_FLAG | SIGN_FLAG | PARITY_FLAG | 0x2,
  ),
  stack_pointer: VirtAddr(
    0xffffffff001fff58,
  ),
  stack_segment: SegmentSelector {
    index: 0,
    rpl: Ring0,
  },
}

Stack trace (most recent call last):
#00: 0xffffffff00000dedb
#01: 0xffffffff000005044
#02: 0x0000123456789000
#03: 0xffffffff0000031b9
#04: 0x00000000040a7977
```

在调试器中可以查看异常发生前后的栈指针大小：

```
Breakpoint 2, ysos_kernel::kernel_main (boot_info=0x5e93ea0) at crates/kernel/src/main.rs:14
(gdb) n
(gdb) n
(gdb) n
(gdb) info reg rsp
rsp             0xffffffff001fff58  0xffffffff001fff58
(gdb) n
ysos_kernel::interrupt::exceptions::double_fault_handler (
  stack_frame=<error reading variable: Cannot access memory at address 0xffffffff00200000>, error_code=99171920)
  at crates/kernel/src/interrupt/exceptions.rs:38
(gdb) n

Breakpoint 1, ysos_kernel::interrupt::exceptions::double_fault_handler (stack_frame=..., error_code=0)
  at crates/kernel/src/interrupt/exceptions.rs:42
(gdb) info reg rsp
rsp             0xffffffff000813048  0xffffffff000813048 <<ysos_kernel::memory::gdt::TSS as core::ops::deref::Deref>::deref::__
static_ref_initialize::STACK+3912>
(gdb)
```

该结果表明，我的栈切换完全正确，且符合预期。

8.5.3. 通过访问非法地址触发 Page Fault，观察 Page Fault 的发生过程。分析 Cr2 寄存器的值，并尝试回答为什么 Page Fault 属于可恢复的异常。

缺页异常发生前后的寄存器值：

```
Breakpoint 1, 0xffffffff000003000 in yyos_kernel::kernel_main ()
(gdb) monitor info registers

CPU#0
RAX=ffff00000031b0 RBX=000000000200000 RCX=0000000005e93ea0 RDX=ffff01001fff8
RSI=ffff800000000000 RDI=0000000005e93ea0 RBP=ffff01001fffe8 RSP=ffff01001fffe0
R8 =0000000000000000 R9 =0000000000000501 R10=00000000056e6880 R11=0000000000000868
R12=00000000040ad040 R13=ffffffffffa0 R14=0000000005e96798 R15=00000000055ec018
RIP=ffff0000003000 RFL=00000082 [--S---] CPL=0 II=0 A20=1 SMM=0 HLT=0
ES =0030 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
CS =0038 0000000000000000 ffffffff 00af9a00 DPL=0 CS64 [-R-]
SS =0030 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
DS =0030 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
FS =0030 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
GS =0030 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
LDT=0000 0000000000000000 0000ffff 00008200 DPL=0 LDT
TR =0000 0000000000000000 0000ffff 00008b00 DPL=0 TSS64-busy
GDT= 00000000055dc000 00000047
IDT= 0000000005138018 00000fff
CR0=80010033 CR2=0000000000000000 CR3=0000000005801000 CR4=00000668
DR0=0000000000000000 DR1=0000000000000000 DR2=0000000000000000 DR3=0000000000000000
DR6=00000000ffff0ff0 DR7=0000000000000400
EFER=0000000000000d00
FCW=037f FSW=0000 [ST=0] FTW=00 MXCSR=00001f80
FPR0=0000000000000000 0000 FPR1=0000000000000000 0000
FPR2=0000000000000000 0000 FPR3=0000000000000000 0000
FPR4=0000000000000000 0000 FPR5=0000000000000000 0000
FPR6=0000000000000000 0000 FPR7=8000000000000000 4006
XMM00=0000000000000000 0000000000000000 XMM01=0000000000000000 0000000000000000
XMM02=0000000000000000 0000000000000000 XMM03=0000000000000000 0000000000000000
XMM04=0000000000000000 0000000000000000 XMM05=0000000000000000 0000000000000000
XMM06=0000000000000000 0000000000000000 XMM07=0000000000000000 0000000000000000
XMM08=0000000000000000 0000000000000000 XMM09=0000000000000000 0000000000000000
XMM10=0000000000000000 0000000000000000 XMM11=0000000000000000 0000000000000000
XMM12=0000000000000000 0000000000000000 XMM13=0000000000000000 0000000000000000
XMM14=0000000000000000 0000000000000000 XMM15=0000000000000000 0000000000000000
```

```

(gdb) n
Single stepping until exit from function _RNvCs6uHWCmPRqIz_11yyos_kernel11kernel_main,
which has no line number information.
0xfffff000000e40 in yyos_kernel::interrupt::exceptions::page_fault_handler ()
(gdb) n
Single stepping until exit from function
_RNvNtNtCsI7ZoszOsIzC_11yyos_kernel9interrupt10exceptions18page_fault_handler,
which has no line number information.
0xfffff000000e320 in core::panicking::panic_fmt ()
(gdb) monitor info registers

CPU#0
RAX=fffff00000073a0 RBX=fffff00000009b1 RCX=0000123456789000 RDX=fffff00000010458
RSI=fffff0000008162a0 RDI=fffff00000001385 RBP=fffff000000816320 RSP=fffff000000816290
R8 =000000000000003fd R9 =0000000000000000d R10=fffff00000012000 R11=ffffffffff8
R12=00000000040ad040 R13=fffff1ffaafc0 R14=fffff000000817388 R15=00000000055ec018
RIP=fffff000000e320 RFL=00000046 [---Z-P-] CPL=0 II=0 A20=1 SMM=0 HLT=0
ES =0000 0000000000000000 00000000 00000000
CS =0008 0000000000000000 ffffffff 00af9b00 DPL=0 CS64 [-RA]
SS =0000 0000000000000000 00000000 00000000
DS =0010 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
FS =0000 0000000000000000 00000000 00000000
GS =0000 0000000000000000 00000000 00000000
LDT=0000 0000000000000000 0000ffff 00008200 DPL=0 LDT
TR =0018 fffff0000008132a0 00000067 00008900 DPL=0 TSS64-avl
GDT= fffff000000813310 00000027
IDT= fffff000000812290 00000fff
CR0=80010033 CR2=0000123456789000 CR3=0000000005801000 CR4=00000668
DR0=0000000000000000 DR1=0000000000000000 DR2=0000000000000000 DR3=0000000000000000
DR6=00000000ffff0fff DR7=0000000000000400
EFER=0000000000000d00
FCW=037f FSW=0000 [ST=0] FTW=00 MXCSR=00001f80
FPR0=0000000000000000 0000 FPR1=0000000000000000 0000
FPR2=0000000000000000 0000 FPR3=0000000000000000 0000
FPR4=0000000000000000 0000 FPR5=0000000000000000 0000
FPR6=0000000000000000 0000 FPR7=8000000000000000 4006
XMM0=0000000000000000 0000000000000000 XMM01=0000000000000000 0000000000000000
XMM02=0000000000000000 0000000000000000 XMM03=0000000000000000 0000000000000000
XMM04=0000000000000000 0000000000000000 XMM05=0000000000000000 0000000000000000
XMM06=0000000000000000 0000000000000000 XMM07=0000000000000000 0000000000000000
XMM08=0000000000000000 0000000000000000 XMM09=0000000000000000 0000000000000000
XMM10=0000000000000000 0000000000000000 XMM11=0000000000000000 0000000000000000
XMM12=0000000000000000 0000000000000000 XMM13=0000000000000000 0000000000000000
XMM14=0000000000000000 0000000000000000 XMM15=0000000000000000 0000000000000000
(gdb)

```

 寄存器前后对比分析

1. 核心控制寄存器：CR2（决定性的证据）

异常前：CR2=0000000000000000

异常后：CR2=0000123456789000

分析：这是最关键的变化。当 read_volatile 尝试访问 0x123456789000 时，硬件立刻将其填入 CR2。它直接指证了是哪个虚拟地址导致了“地图（页表）”查询失败。

2. 指令指针与栈指针：RIP & RSP

RIP（执行到哪了）：

前：fffff0000003000（kernel_main 入口）。

后：fffff000000e320（已经跳进了 panic_fmt）。

RSP（栈在哪里）：

前：fffff01001ffe0（初始内核栈）。

后：fffff0000816290（注意变化！地址从 01 开头变成了 00 开头）。

分析：由于 register_idt 可能为异常配置了 IST（或者跳转到了特定的异常处理栈），RSP 发生了跳变，确保异常处理程序在一个干净、安全的栈上运行。

3. 段寄存器与状态：TR & GDT

异常前：TR=0000，GDT=00000000055dc000（地址在低半区，这是 Bootloader 留下的旧环境）。

异常后：TR=0018，GDT=fffff0000813310（地址在高半区，且 TR 加载了 0x18 偏移量）。

分析：这说明在两次快照之间，你的 yyos::init() 成功运行了，它切换了全局描述符表并加载了任务状态段（TSS）。

 为什么缺页异常（Page Fault）是可恢复的？

在 x86 架构中，异常被分为三类：Fault（错误）、Trap（陷阱）和 Abort（中止）。缺页异常属于 Fault，它的设计初衷就是为了“挽救”。

1. 硬件设计的“后悔药”机制

这是最核心的一点：当 Page Fault 发生时，压入栈中的返回地址（RIP）指向的是触发异常的那条指令本身，而不是下一条。

逻辑：如果操作系统能够修复页表（比如从硬盘加载数据到内存），执行 IRETQ 返回后，CPU 会重新执行刚才失败的那条指令。只要页表修好了，第二次执行就会成功，程序完全察觉不到自己曾经“摔倒”过。

2. 它是虚拟内存管理的基础

如果 Page Fault 不可恢复，现代计算机将无法运行：

按需分页 (Demand Paging)：为了省内存，系统只在程序真正访问某页时才分配物理内存。

交换空间 (Swap)：内存不够时把数据挪到硬盘，访问时再通过补回映射来恢复执行。

共享内存 (CoW)：两个进程共享同一块只读内存，只有当有人尝试写入并触发异常时，才分配新页并恢复运行。

3. 为什么你的程序“死”了？

虽然硬件支持恢复，但软件决定了命运。代码里，`page_fault_handler` 最终调用了 `panic!`。这意味着你的内核告诉 CPU：“我不知道怎么修复这个地址，直接判死刑吧。”于是它就变成了一个不可恢复的错误。

8.6. 如果在 TSS 中为中断分配的栈空间不足，会发生什么情况？请分析 CPU 异常的发生过程，并尝试回答什么时候会发生 Triple Fault。

答：

1. 栈空间不足引发的连锁反应

假设你为 Double Fault 分配了 4KB 的 IST 栈，但你的处理程序（Handler）逻辑复杂，消耗了超过 4KB 的空间：

第一阶段：静默破坏（Stack Overflow）

当 CPU 执行到 Handler 内部时，随着函数调用加深，RSP 会不断减小，最终越过你分配的 STACK 静态数组边界。由于内核空间通常是连续的，溢出的栈会覆盖掉相邻的内核数据或代码。

第二阶段：触发第二次异常（Secondary Exception）

当溢出的 RSP 最终指向了一个未映射的内存区域，或者试图向只读代码段压栈时，CPU 会立即触发一个新的异常（通常是 Page Fault 或 General Protection Fault）。

第三阶段：异常升级（Exception Promotion）

此时 CPU 陷入了绝境：它正在处理一个异常，结果在处理过程中又产生了一个新的异常。

根据 Intel 手册，如果在处理异常（如 Double Fault）时又发生了异常，且该异常无法被嵌套处理，CPU 会判定为 Triple Fault。

2. 什么时候会发生 Triple Fault？

Triple Fault（三重故障）是 CPU 的“终极放弃”。当 CPU 无法调用异常处理程序来处理当前的异常时，它就会停止工作，并向系统发送一个信号，通常表现为虚拟机瞬间重启。

发生 Triple Fault 的典型场景包括：

A. 栈切换失败（最常见原因）

这是你目前正在攻克的难关。当异常发生，CPU 尝试通过 TSS 切换栈时，如果发生以下情况，会直接触发 Triple Fault：

无效地址：TSS 中填写的 IST 地址是一个未在页表中映射的虚空地址（如你之前遇到的高位补全失败）。

权限错误：IST 地址指向的页面标记为“只读”或“不存在”。

递归崩溃：CPU 尝试向 IST 栈压入现场，但该操作本身触发了 Page Fault，而 CPU 此时已经认为自己在处理双重错误。

B. IDT 配置错误

如果异常发生时，CPU 试图去查 IDT 表，但：

IDT 越界：异常向量号超过了 IDTR.limit。

描述符无效：IDT 里的 Gate 描述符被设为了 Present = 0。

CS 错误：IDT 描述符里填写的代码段选择子在 GDT 中不存在。

C. 递归异常无法处理

如果发生了一个异常（如 Page Fault），CPU 试图调用 Handler，但 Handler 没注册。于是升级为 Double Fault。如果 Double Fault 也没注册或者其 Handler 本身又崩了，CPU 就彻底死心，进入 Triple Fault 关机重启。

3. CPU 异常发生的底层路径图

我们可以把这个过程看作是一个三级跳：

一级异常 (First-level Exception)：

例如：你代码里的 `read_volatile(0xdeadbeef)` 触发 Page Fault。

CPU 尝试压栈保存现场。

二级异常 (Double Fault)：

如果压栈失败，或者 IDT 里没对应的门。

CPU 查找 IDT 第 8 号向量。

关键点：如果 double fault 设置了 IST，CPU 强行跳到新栈。

三级异常 (Triple Fault)：

如果 CPU 在尝试进入 double fault Handler 的过程中再次失败（如：TSS 基址非法、IST 栈指针所在的页表没映射）。

结局：CPU 停机 (Halt)，并触发硬件重置信号。

💡 总结与建议

栈空间不足：不会立即重启，但会破坏周围数据，最终因访问非法内存而导致系统崩溃。

Triple Fault：通常发生在异常跳转的过渡瞬间。如果你看到 QEMU 瞬间重启，90% 的概率是你的 TSS 地址、GDT 权限或者页表映射（对应栈空间）没写对。

8.7. 在未使用 `set_stack_index` 函数时，中断处理程序的栈可能哪里？尝试结

合 gdb 调试器，找到中断处理程序的栈，并验证你的猜想是否正确。

答：如果没有使用这个函数的话，中断处理程序可能会原地压栈。我把缺页异常处理的 `set_stack_index` 函数注释掉了，然后触发缺页异常之后查看栈指针，结果：

```
(gdb) n
Single stepping until exit from function _RNvCs6uHWcmPRqIz_11ysos_kernel11kernel_main,
which has no line number information.
0xffffffff0000006e40 in ysos_kernel::interrupt::exceptions::page_fault_handler ()
(gdb) info reg rsp
rsp             0xffffffff01001fff20  0xffffffff01001fff20
(gdb)
```

实验数据分析

当前 RSP 值：0xffffffff01001fff20

对比之前的 Double Fault 实验：

在之前的实验中，触发 Double Fault 且开启了 IST 切换后，RSP 跳到了 0xffffffff0000813048（那个专门分配的高半区 TSS 栈）。

而在注释掉 set_stack_index 后的这次实验，RSP 依然停留在 0xffffffff01... 这个地址段。

 结论：猜想验证成功

没有发生栈切换：

此时的 RSP 指针（01 开头）与触发异常前的内核主栈是在同一个内存区域的。这证明：如果没有显式指定 IST 索引，CPU 默认会直接使用发生中断那一刻的栈指针。

原地压栈的行为：

当 Page Fault 发生时，硬件直接在当前的内核栈（Ring 0 栈）上压入了 SS, RSP, RFLAGS, CS, RIP 以及 Error Code。

 这种做法的“危险性”

既然能跑通，为什么我们还要大费周章地写那个函数？

现在的 page_fault_handler 正在使用主内核栈。如果是因为栈溢出（Stack Overflow）导致的缺页，那么：

当前的 RSP 已经越界，指向了一个非法区域。

CPU 尝试在“坏掉”的 RSP 处压入现场。

二次崩溃：压栈操作本身触发了第二次 Page Fault。

由于没有独立的 IST 逃生通道，CPU 无法处理这种递归异常，会直接导致 Triple Fault（虚拟机瞬间重启）。

 总结

实验清楚地展示了：

不调用该函数：中断处理程序与被中断的代码共用同一个栈，这在正常情况下没问题，但在处理严重错误（如 Double Fault）时极度危险。

调用该函数：中断处理程序会强制切换到独立、安全的 IST 栈，确保即便主栈烂透了，内核也能稳稳地接住异常并输出调试信息。



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验三：内核线程与缺页异常

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 了解进程与线程的概念、相关结构和实现。
2. 实现内核线程的创建、调度、切换。(栈分配、上下文切换)
3. 了解缺页异常的处理过程，实现进程的栈增长。

二. 实验内容

1. 进程模型设计

1.1. 进程控制块

在实验中使用 `Process` 结构体表示一个进程，它含有 `pid` 和 `inner` 两个字段，分别表示进程的 ID 和内部数据。`inner` 字段是一个 `RwLock` 类型的字段，它包含了一个 `ProcessInner` 结构体，这个结构体包含了进程的其他信息，包括进程的状态、调度计次、退出返回值、内存空间、父子关系、中断上下文、文件描述符表等等

1.2. 进程上下文

在抢占式操作系统中，进程的调度是通过中断来实现的。当一个进程的时间片用完后，操作系统会触发一个时钟中断，进程调度器会被唤醒，它会根据进程的状态和调度策略来决定下一个要运行的进程。在调度器决定好下一个要运行的进程后，它会将当前进程的上下文保存起来，然后将下一个进程的上下文恢复，从而使得下一个进程得以运行。

在之前的实验中，已经描述过在 `x86_64` 架构下的中断发生时，CPU 会将当前的一部分上下文保存到内核栈中，然后跳转到中断处理函数。这些上下文包括：

`instruction_pointer`：指令指针，保存了中断发生时 CPU 正在执行的指令的地址。
`code_segment`：代码段寄存器，保存了当前正在执行的代码段的选择子。
`cpu_flags`：CPU 标志寄存器，保存了中断前的 CPU 标志状态。
`stack_pointer`：栈指针，保存了中断前的栈指针。
`stack_segment`：栈段寄存器，保存了中断前的栈段选择子，在 `x86_64` 下总是为 0。而在进行进程切换时，通常还需要保存和恢复更多的上下文，这些内容主要包括通用寄存器和浮点寄存器。为了简化实现，实验在编译架构选项中禁用了浮点寄存器，因此只需要保存和恢复通用寄存器即可。

1.3. 进程页表

进程的页表是通过向 Cr3 寄存器写入页表的物理地址来实现控制的，因此在进程切换时，需要将进程的页表物理地址写入 Cr3 寄存器。

除了内核进程的页表在启动时被初始化外，其他进程的页表都是在进程创建时被初始化的。它们通常通过克隆内核进程来实现，这样做的目的是当程序陷入中断时，CPU 能够正常访问到内核的代码和数据，从而能够正常的进行系统调用。

1.4. 进程调度

进程调度的工作目的简单来说就是从就绪队列中选取一个进程，并将其分配给选定的 CPU 核心运行。在实验中，为了实现简单，YSOS 自始至终只会使用一个 CPU 核心。

在实验中，将实现一个简单的 FIFO 调度器，它会将就绪队列中的第一个进程选出来，然后将其分配给 CPU 核心运行，当进程的时间片用完后，调度器会将其重新放回就绪队列的末尾。

1.4.1. 内核进程

在初始化进程管理器时，为了让内核始终和其他进程有一样的获取时间片的机会，需要将内核进程作为第一个进程加入进程列表。

之后的每次切换时，内核进程也会被视为一个普通的进程，从而被调度器选中、保存、执行。

1.4.2. 时钟中断的处理

时钟中断发生时，进程调度器被运行，CPU 通过 IDT 调用中断处理程序，完成进程的切换。

在一次进程切换的过程中，需要关闭中断，之后完成以下几个步骤：

- 保存当前进程的上下文

经过上述进程上下文的描述可知，作为可变引用参数被传入的 `ProcessContext` 中存储了进程切换时需要保存的所有寄存器的值，并且其中的内容将会在进程切换完成后被恢复，进而真正实现进程的切换。

因此，进程切换的第一步就是将 `context` 保存至当前进程的 `ProcessInner` 中，以便下次恢复运行状态。

- 更新当前进程的状态

进程切换时，若它当前的状态并非 `Dead`，则当前进程的状态会被更新为 `Ready`。

同时，为了记录进程的执行时间，一般也会记录进程的调度次数，这里使用 `usize` 类型的 `ticks_passed` 字段来记录进程的调度次数。

- 将当前进程放入就绪队列

当前进程被切换出去后，它会被放入就绪队列的末尾，等待下一次调度。

- 从就绪队列中选取下一个进程

进程调度器会从就绪队列中选取第一个进程，检查进程的状态，如果进程处于可调度状态，就将其状态更新为 `Running`，并将其 `PID` 写入 `Processor` 中。

- 切换进程上下文和页表

进程调度器会将选中的进程的上下文重新加载，并将新进程的页表物理地址写入 Cr3 寄存器，从而完成进程的切换。

1.5. 进程的内存布局

在开启了虚拟内存的情况下，操作系统拥有巨大的地址空间，可以赋予它们不同的功能，管理这些内存如何使用。

在已经进行过的实验中，笔者为大家预设了一些内存布局，你应该或多或少接触过这些地址：

- 物理内存偏移：0xFFFF800000000000

通过定义物理内存偏移，借助 2MB 的页映射，将物理内存线性映射到了这一偏移量所对应的地址空间中。在内核中，当需要访问一个物理内存地址，如 0x1000 时，就可以通过 0xFFFF800000001000 来访问。

- 内核空间起始地址：0xFFFFF00000000000

在操作系统中，一般会将内核地址映射到高偏移，而将用户地址映射到低偏移。在实验中，内核空间的起始地址被定义在了 0xFFFFF00000000000，这相当于为内核预留了 1TiB 的地址空间。

通过 kernel.ld 链接器脚本，将内核的起始地址设置为了这一地址，实验中编译的操作系统内核会被链接到从这一地址开始的地址空间中。同时，通过 bootloader 中的内核加载函数，读取 ELF 文件的描述，并将这些内容加载到了对应的地址空间中。

- 内核栈地址：0xFFFFF01000000000

内核栈的起始地址通过配置文件被定义在了 0xFFFFF01000000000，距离内核起始地址 4GiB。默认大小为 512 个 4KiB 的页面，即 2MiB。

在虚拟内存的规划中，任意进程的栈地址空间大小为 4GiB。以内核为例，内核栈所对应的内存区域的起始地址为 0xFFFFF01000000000，结束地址为 0xFFFFF02000000000。

2. 合并实验代码

3. 进程管理器的初始化

4. 进程调度的实现

5. 进程信息的获取

5.1. 环境变量

5.2. 进程返回值

6. 内核线程的创建

7. 缺页异常的处理

8. 进程的退出

三. 实验过程

1. 合并实验代码

我在 lab2 的基础上新建了 lab3 分支，然后将官方文档下载到根目录下的 `src/0x03/crates/kernel` 文件夹中，再在 YatSenOS 根目录下运行以下命令：

```
cp -rf src/0x03/crates/kernel/* yatsenos/
```

这样就完成了实验代码的合并。

2. 进程管理器的初始化

在 `src/proc/mod.rs` 中，补全 `init` 函数，这个函数将内核包装成进程，并将其传递给 `ProcessManager`，使其成为第一个进程。


```

/// init process manager
pub fn init(boot_info: &'static boot::BootInfo) {
    let proc_vm = ProcessVm::new(PageTableContext::new()).init_kernel_vm();

    trace!("Init kernel vm: {:?}", proc_vm);

    let mut proc_data = ProcessData::new();

    // 从 boot.conf 配置中获取的信息（硬编码或从配置解析）
    proc_data.set_env("KERNEL_STACK_ADDR", "0xFFFFFFFF00000000");
    proc_data.set_env("KERNEL_STACK_SIZE", "512");
    proc_data.set_env("KERNEL_PATH", "\\KERNEL.ELF");

    // 从 BootInfo 获取的信息
    proc_data.set_env("PHYSICAL_MEM_OFFSET", format!("{:x}", boot_info.physical_memory_offset).as_str());
    proc_data.set_env("SYSTEM_TABLE", format!("{:x}", boot_info.system_table.as_ptr() as u64).as_str());

    // 从运行时获取的信息
    proc_data.set_env("KERNEL_HEAP_SIZE", format!("{}", HEAP_SIZE).as_str());

    // kernel process
    let kproc = {
        /* FIXME: create kernel process */
        Process::new(
            String::from("kernel"),
            None,
            Some(proc_vm),
            Some(proc_data),
        )
    };
    manager::init(kproc);

    info!("Process Manager Initialized.");
}

```

3. 进程调度的实现

3.1. 修改时间中断的内容

```

use super::consts::*;
// 引入原子类型和内存排序规则
use core::sync::atomic::{AtomicU64, Ordering};
use x86_64::structures::idt::{InterruptDescriptorTable, InterruptStackFrame};
use crate::as_handler;
use crate::proc::{switch, ProcessContext};
use crate::memory::gdt::CLOCK_IST_INDEX;

as_handler!(clock);
pub unsafe fn register_idt(idt: &mut InterruptDescriptorTable) {
    idt[Interrupts::IrqBase as u8 + Irq::Timer as u8]
        .set_handler_fn(clock_handler)
        .set_stack_index(CLOCK_IST_INDEX);
}

static COUNTER: AtomicU64 = AtomicU64::new(0);

#[inline]
pub fn read_counter() -> u64 {
    // 使用 Relaxed 内存序读取当前值
    COUNTER.load(Ordering::Relaxed)
}

#[inline]
pub fn inc_counter() -> u64 {
    // fetch_add 会将变量加 1，并返回相加【之前】的旧值。
    // 为了让 inc_counter 返回最新的值，我们在后面 + 1
    COUNTER.fetch_add(1, Ordering::Relaxed) + 1
}

pub fn clock(context: &mut ProcessContext) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        /*if inc_counter() % 0x100 == 0 {
            info!("Tick! @{}", read_counter());
        }*/
        switch(context);
        super::ack();
    });
}

```

修改时间中断函数，在 TSS 中声明一块新的中断处理栈，并将它加载到时钟中断的 IDT 中。在时间中断处调用进程切换函数，再用 `as_handler` 宏重新定义中断处理函数，它会生成一个函数，先保存寄存器然后利用 `call` 指令调用原函数，再恢复寄存器，使用 `iretq` 指令返回。

3.2. 补全 `switch`、`save_current` 和 `switch_next` 函数的实现

```

pub fn switch(context: &mut ProcessContext) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        // FIXME: switch to the next process
        get_process_manager().save_current(context);

        let current = get_process_manager().current();
        if current.read().status() == ProgramStatus::Ready {
            get_process_manager().push_ready(current.pid());
        }
        get_process_manager().switch_next(context);
    });
}

pub fn save_current(&self, context: &ProcessContext) {
    // FIXME: update current process's tick count
    let cur = self.current();
    // FIXME: save current process's context
    cur.write().tick();
    cur.write().save(context);
    cur.write().pause();
}

pub fn switch_next(&self, context: &mut ProcessContext) -> ProcessId {
    // 1. 从就绪队列获取下一个进程
    let next_pid = loop {
        let pid = self.ready_queue.lock().pop_front();

        if let Some(pid) = pid {
            let proc = self.get_proc(&pid);

            // 2. 检查进程是否存在且就绪
            if let Some(proc) = proc {
                if proc.read().is_ready() {
                    break pid;
                }
            }
        }
    }
    // 如果进程不就绪，继续循环获取下一个
} else {
    // 就绪队列空，返回当前进程 PID（无切换）
    return processor::get_pid();
}
};

```

4. 进程信息的获取

4.1. 环境变量

补全 env 函数：

```
pub fn env(key: &str) -> Option<String> {
    x86_64::instructions::interrupts::without_interrupts(|| {
        let current = get_process_manager().current();
        let inner = current.read();

        // Rust 自动插入 Deref 调用：
        // inner.env(key)
        // ↓ 自动解引用 RwLockReadGuard → &ProcessInner
        // (*inner).env(key)
        // ↓ ProcessInner 的 Deref 实现 → &ProcessData
        // ProcessData::env(&**inner, key)

        inner.env(key)
    })
}
```

4.2. 进程返回值

补全 wait 函数：

```

pub fn new_stack_test_thread() {
    let pid = spawn_kernel_thread(
        func::stack_test,
        alloc::string::String::from("stack"),
        None,
    );

    // wait for progress exit
    wait(pid);
} // 创建一个栈进程就要立马停下来等进程退出才结束创建过程
fn wait(pid: ProcessId) {
    loop {
        // FIXME: try to get the status of the process
        if let Some(proc) = get_process_manager().get_proc(&pid)
        { // HINT: it's better to use the exit code
            let EXIT_CODE = proc.read().exit_code();
            if EXIT_CODE.is_some() {
                break; // exit_code 不等于 none 就说明该进程已经退出了
            } else {
                x86_64::instructions::hlt();
            }
        } else {
            break;
        }
    }
}

```

5. 内核线程的创建

//在 src/proc/mod.rs 中定义的 spawn_kernel_thread 函数。它关闭中断，之后将函数转化为地址以使其能够赋值给 rip 寄存器，之后将进程的信息传递给 ProcessManager，使其创建所需进程。

```
pub fn spawn_kernel_thread(entry: fn() -> !, name: String, data: Option<ProcessData>) -> ProcessId {
    x86_64::instructions::interrupts::without_interrupts(|| {
        let entry = VirtAddr::new(entry as usize as u64);
        get_process_manager().spawn_kernel_thread(entry, name, data)
    })
}
```

//src/proc/process.rs 中，根据内存布局预设和当前进程的 PID，为其分配初始栈空间。

```
pub fn spawn_kernel_thread(
    &self,
    entry: VirtAddr,
    name: String,
    proc_data: Option<ProcessData>,
) -> ProcessId {
    let kproc = self.get_proc(&KERNEL_PID).unwrap();
    let page_table = kproc.read().clone_page_table();
    let proc_vm = Some(ProcessVm::new(page_table));
    let proc = Process::new(name, Some(Arc::downgrade(&kproc)), proc_vm, proc_data);

    // alloc stack for the new process base on pid
    let stack_top = proc.alloc_init_stack();
    let pid = proc.pid();
    // FIXME: set the stack frame
    proc.write().init_stack_frame(entry, stack_top);
    // FIXME: add to process map
    self.add_proc(pid, proc);
    // FIXME: push to ready queue
    self.push_ready(pid);
    // FIXME: return new process pid
    pid
}
```

6. 缺页异常的处理

在本实验设计中，实现为栈空间自动扩容来作为缺页异常的处理。

完善缺页异常的相关处理函数：

- 1.在 ProcessManager 中，检查缺页异常是否包含越权访问或其他非预期的错误码。
 - 2.如果缺页异常是由于非预期异常导致的，或者缺页异常的地址不在当前进程的栈空间中，直接返回 false。
 - 3.如果缺页异常的地址在当前进程的栈空间中，把缺页异常的处理委托给当前的进程。
- 需要为 ProcessInner 和 ProcessVm 添加用于分配新的栈、更新进程存储信息的函数。

4.在进程的缺页异常处理函数中：

分配新的页面、更新页表、更新进程数据中的栈信息。

processmanager 中有关页错误的处理函数：

```
pub fn handle_page_fault(&self, addr: VirtAddr, err_code: PageFaultErrorCode) -> bool {
    // FIXME: handle page fault
    // 1. 检查保留位违规 - 硬件错误或严重问题
    if err_code.contains(PageFaultErrorCode::MALFORMED_TABLE) {
        return false;
    }
    // 2. 检查地址是否为空指针或接近空指针
    if addr.as_u64() < 0x1000{
        return false;
    }
    // 3. 检查地址是否在有效的用户空间范围内
    if !is_canonical(addr.as_u64() as usize){
        return false;
    }

    // 7. 检查是否在保护违规情况下访问内核空间
    let user_mode = err_code.contains(PageFaultErrorCode::USER_MODE);
    let protection = err_code.contains(PageFaultErrorCode::PROTECTION_VIOLATION);

    // 用户态尝试访问内核空间
    if user_mode && addr.as_u64() >= 0xffff_8000_0000_0000 && protection {
        return false; // 非法访问内核空间 - 非预期
    }

    let current = self.current();
    let proc = current.read();

    drop(proc);
    if current.write().handle_page_fault(addr) {
        return true; // 成功处理 - 预期异常 (如栈增长)
    }

    false
}
```

Page Fault 文件有关页处理的核心逻辑：

```

pub struct Stack {
    range: PageRange<Size4KiB>,
    usage: u64,
}

impl Stack {
    pub fn handle_page_fault(
        &mut self,
        addr: VirtAddr,
        mapper: MapperRef,
        alloc: FrameAllocatorRef,
    ) -> bool {
        if !self.is_on_stack(addr) {
            return false;
        }
        if let Err(m) = self.grow_stack(addr, mapper, alloc) {
            error!("Grow stack failed: {:?}", m);
            return false;
        }
        true
    }

    fn is_on_stack(&self, addr: VirtAddr) -> bool {
        let addr = addr.as_u64();
        let cur_stack_bot = self.range.start.start_address().as_u64();
        trace!("Current stack bot: {:#x}", cur_stack_bot);
        trace!("Address to access: {:#x}", addr);
        // Is it within the STACK_MAX_SIZE capacity?
        let max_stack_top = (cur_stack_bot & STACK_START_MASK) + STACK_MAX_SIZE;
        addr >= (cur_stack_bot & STACK_START_MASK) && addr < max_stack_top
    }

    fn grow_stack(
        &mut self,
        addr: VirtAddr,
        mapper: MapperRef,
        alloc: FrameAllocatorRef,
    ) -> Result<(), MapToError<Size4KiB>> {
        debug_assert!(self.is_on_stack(addr), "Address is not on stack.");
        // FIXME: grow stack for page fault
        let fault_page = Page::containing_address(addr);
        let current_bot = self.range.start;
        if fault_page >= current_bot {
            return Ok(());
        }
        let new_pages_count = current_bot - fault_page;
        let new_usage = self.usage + new_pages_count;
        if new_usage > STACK_MAX_PAGES {
            error!("Stack overflow: requested {} pages, max is {}",
                new_usage, STACK_MAX_PAGES);
            return Err(MapToError::FrameAllocationFailed);
        }
    }
}

```



```

let flags = PageTableFlags::PRESENT | PageTableFlags::WRITABLE | PageTableFlags::USER_ACCESSIBLE;

for page in Page::range(fault_page, current_bot){
    let frame = alloc.allocate_frame().ok_or(MapToError::FrameAllocationFailed)?;

    unsafe {
        mapper.map_to(page, frame, flags, alloc)?.flush();
    }
}

self.range = Page::range(fault_page, self.range.end);
self.usage = new_usage;

Ok(())
}

pub fn memory_usage(&self) -> u64 {
    self.usage * crate::memory::PAGE_SIZE
}
}

```

processVM 中有关页错误处理的函数：

```

pub fn handle_page_fault(&mut self, addr: VirtAddr) -> bool {
    let mapper = &mut self.page_table.mapper();
    let alloc = &mut *get_frame_alloc_for_sure();

    self.stack.handle_page_fault(addr, mapper, alloc)
}

```

7. 进程的退出

实现进程的退出要完成如下几件事：

- 在进程退出时，将进程的状态设置为 Dead，并删除进程运行时需要的部分数据，如 ProcessData。
- 确保进程不会被再次调度，这可以在切换时添加检查、防止进程再次进入就绪队列来实现。
- 存储进程的返回值，以便其他进程可以利用它来查询进程的退出状态。

```
pub fn kill(&mut self, ret: isize) {
    // FIXME: set exit code
    self.exit_code = Some(ret);
    // FIXME: set status to dead
    self.status = ProgramStatus::Dead;
    // FIXME: take and drop unused resources
    if let Some(cur_proc_data) = self.proc_data.take()
    {
        drop(cur_proc_data);
    }
    if let Some(cur_proc_vm) = self.proc_vm.take()
    {
        drop(cur_proc_vm);
    }
    self.children.clear();

    trace!("Process {} killed with exit code {}", self.name, ret);
}
}
```

8. 进程调度内核运行成果展示

为了给本实验添加个性化，我将实验中本内核的名字由 yatsenos 改成了 yyos,由我的名字命名，我修改了配置文件和项目文件夹名以及内核启动时输出的系统 logo，并用 cargo clean 删除旧的编译产物，重新编译成功。

以下是本次实验的终端输出结果：

```

  _ _ _ _ _ U _ _ u _ _ _
  \ \ / \ \ / \ V " _ V / _ " | u
  \ V / \ V /   | | | | < \ _ _ V
  U _ " | _ u U _ " | _ u _ _ | | | u _ _ ) |
  | | | | | \ ) - \ _ / | _ _ / > >
  _ _ / | ( _ _ _ / | ( _ _ \ \ ) ( _ _
  \ ) ( _ ) \ ) ( _ ) ( _ ) ( _ )
```

By GYY0.2.0

[+] Serial Initialized.

[INFO] [crates/kernel/src/utils/logger.rs:11] Logger Initialized.

```

[ INFO] [crates/kernel/src/memory/address.rs:9] Physical Offset : 0xffff800000000000
[ INFO] [crates/kernel/src/memory/gdt.rs:34] Privilege Stack : 0xfffff00000213c8-0xfffff00000223c8
[ INFO] [crates/kernel/src/memory/gdt.rs:49] Privilege Stack : 0xfffff00000223c8-0xfffff00000233c8
[ INFO] [crates/kernel/src/memory/gdt.rs:61] Privilege Stack : 0xfffff00000233c8-0xfffff00000243c8
[ INFO] [crates/kernel/src/memory/gdt.rs:73] Privilege Stack : 0xfffff00000243c8-0xfffff00000253c8
[ INFO] [crates/kernel/src/memory/gdt.rs:85] Privilege Stack : 0xfffff00000253c8-0xfffff00000263c8
[ INFO] [crates/kernel/src/memory/gdt.rs:148] Kernel IST Size : 12.000 KiB
[ INFO] [crates/kernel/src/memory/gdt.rs:150] GDT Initialized.
[DEBUG] [crates/kernel/src/memory/allocator.rs:26] Kernel Heap : 0xfffff00000263c8-0xfffff0000020263c8
[ INFO] [crates/kernel/src/memory/allocator.rs:33] Kernel Heap Size : 32.000 MiB
[ INFO] [crates/kernel/src/memory/allocator.rs:35] Kernel Heap Initialized.
[TRACE] [crates/kernel/src/interrupt/apic/ioapic.rs:74] Enable IOApic: IRQ=4, CPU=0
[TRACE] [crates/kernel/src/interrupt/apic/ioapic.rs:74] Enable IOApic: IRQ=0, CPU=0
[ INFO] [crates/kernel/src/interrupt/mod.rs:46] Interrupts Initialized.
[ INFO] [crates/kernel/src/memory/mod.rs:26] Physical Memory : 12.093 GiB
[ INFO] [crates/kernel/src/memory/mod.rs:29] Free Usable Memory : 10.906 MiB
[ INFO] [crates/kernel/src/memory/mod.rs:38] Frame Allocator initialized.
[TRACE] [crates/kernel/src/proc/mod.rs:36] Init kernel vm: ProcessVm {
    stack: Stack {
        top: 0xfffff01ffff000,
        bot: 0xfffff01ffe00000,
    },
    memory_usage: "2 MiB",
    page_table: PageTable {
        addr: PhysFrame[4KiB](0x5801000),
        flags: Cr3Flags(
            0x0,
        ),
    },
}
[TRACE] [crates/kernel/src/proc/process.rs:72] New process kernel#1 created.

```

```

[TRACE] [crates/kernel/src/proc/manager.rs:43] Init Process {
  pid: 1,
  name: "kernel",
  parent: None,
  status: Running,
  ticks_passed: 0,
  children: Map {
    iter: Iter(
      [],
    ),
  },
  status: Running,
  context: StackFrame {
    stack_top: VirtAddr(
      0x2000,
    ),
    cpu_flags: RFlags(
      0x0,
    ),
    instruction_pointer: VirtAddr(
      0x1000,
    ),
    regs: Registers
    r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
    r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
    r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi: 0x0000000000000000,
    rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
    rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
  },
  vm: Some(
    ProcessVm {
      stack: Stack {
        top: 0xfffff01ffff000,
        bot: 0xfffff01ffe00000,
      },
      memory_usage: "2 MiB",
      page_table: PageTable {
        addr: PhysFrame[4KiB](0x5801000),
        flags: Cr3Flags(
          0x0,
        ),
      },
    ),
  ),
}

```

```

[ INFO] [crates/kernel/src/proc/mod.rs:64] Process Manager Initialized.
[ INFO] [crates/kernel/src/lib.rs:53] Interrupts Enabled.
[ INFO] [crates/kernel/src/lib.rs:55] YYOS initialized.
[ INFO] [crates/kernel/src/main.rs:15] 开始创建进程 :
[TRACE] [crates/kernel/src/proc/process.rs:72] New process #0_test#2 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB](0x3ffdffff000), end: Page[4KiB](0x3ffe0000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0xffffffff000000c0a0,
    ),
    code_segment: SegmentSelector {
        index: 1,
        rpl: Ring0,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ffdffffff8,
    ),
    stack_segment: SegmentSelector {
        index: 2,
        rpl: Ring0,
    },
}

[TRACE] [crates/kernel/src/proc/process.rs:72] New process #1_test#3 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB](0x3ffcffff000), end: Page[4KiB](0x3ffd0000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0xffffffff000000c0a0,
    ),
    code_segment: SegmentSelector {
        index: 1,
        rpl: Ring0,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ffcffffff8,
    ),
    stack_segment: SegmentSelector {
        index: 2,
        rpl: Ring0,
    },
}
}

```

```

[TRACE] [crates/kernel/src/proc/process.rs:72] New process #2_test#4 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB](0x3ffbffff000), end: Page[4KiB](0x3ffc00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0xffffffff000000c0a0,
  ),
  code_segment: SegmentSelector {
    index: 1,
    rpl: Ring0,
  },
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  stack_pointer: VirtAddr(
    0x3ffbffff8,
  ),
  stack_segment: SegmentSelector {
    index: 2,
    rpl: Ring0,
  },
}
[TRACE] [crates/kernel/src/proc/process.rs:72] New process #3_test#5 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB](0x3ffaffff000), end: Page[4KiB](0x3ffb00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0xffffffff000000c0a0,
  ),
  code_segment: SegmentSelector {
    index: 1,
    rpl: Ring0,
  },
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  stack_pointer: VirtAddr(
    0x3ffaffff8,
  ),
  stack_segment: SegmentSelector {
    index: 2,
    rpl: Ring0,
  },
}

```

```

[TRACE] [crates/kernel/src/proc/process.rs:72] New process #4_test#6 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB](0x3ff9ffff000), end: Page[4KiB](0x3ffa00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0xfffff000000c0a0,
  ),
  code_segment: SegmentSelector {
    index: 1,
    rpl: Ring0,
  },
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  stack_pointer: VirtAddr(
    0x3ff9fffff8,
  ),
  stack_segment: SegmentSelector {
    index: 2,
    rpl: Ring0,
  },
}
[INFO] [crates/kernel/src/main.rs:20] Created 5 test threads, scheduler is running...
[>] ps
  PID | PPID | Process Name | Ticks | Status
# 2 | # 1 | #0_test      | 661 | Ready
# 1 | # 0 | kernel       | 666 | Running
# 4 | # 1 | #2_test      | 659 | Ready
# 5 | # 1 | #3_test      | 658 | Ready
# 3 | # 1 | #1_test      | 660 | Ready
# 6 | # 1 | #4_test      | 657 | Ready
Heap   : 7.281 KiB used / 31.993 MiB free / 32.000 MiB total
Queue  : [2, 3, 4, 5, 6]
CPUs   : [0: 1]
[>] test
[TRACE] [crates/kernel/src/proc/process.rs:72] New process #5_test#7 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB](0x3ff8ffff000), end: Page[4KiB](0x3ff900000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0xfffff000000c0a0,
  ),
  code_segment: SegmentSelector {
    index: 1,
    rpl: Ring0,
  },
}

```

```

cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
),
stack_pointer: VirtAddr(
    0x3ff8fffff8,
),
stack_segment: SegmentSelector {
    index: 2,
    rpl: Ring0,
},
}
[>] ps
PID | PPID | Process Name | Ticks | Status
# 2 | # 1 | #0_test | 1353 | Ready
# 1 | # 0 | kernel | 1358 | Running
# 4 | # 1 | #2_test | 1351 | Ready
# 5 | # 1 | #3_test | 1350 | Ready
# 3 | # 1 | #1_test | 1352 | Ready
# 6 | # 1 | #4_test | 1349 | Ready
# 7 | # 1 | #5_test | 287 | Ready
Heap : 7.977 KiB used / 31.992 MiB free / 32.000 MiB total
Queue : [2, 3, 4, 5, 6, 7]
CPUs : [0: 1]
[>] stack
[TRACE] [crates/kernel/src/proc/process.rs:72] New process stack#8 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB](0x3ff7ffff000), end: Page[4KiB](0x3ff800000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0xfffff000000c090,
    ),
    code_segment: SegmentSelector {
        index: 1,
        rpl: Ring0,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ff7fffff8,
    ),
    stack_segment: SegmentSelector {
        index: 2,
        rpl: Ring0,
    },
}

```


[TRACE] [crates/kernel/src/proc/vm/stack.rs:104] Current stack bot: 0x3ff7ffff000

[TRACE] [crates/kernel/src/proc/vm/stack.rs:105] Address to access: 0x3ff7ffff7f88

Huge stack testing...

0x000 == 0x000

0x100 == 0x100

0x200 == 0x200

0x300 == 0x300

0x400 == 0x400

0x500 == 0x500

0x600 == 0x600

0x700 == 0x700

0x800 == 0x800

0x900 == 0x900

0xa00 == 0xa00

0xb00 == 0xb00

0xc00 == 0xc00

0xd00 == 0xd00

0xe00 == 0xe00

0xf00 == 0xf00

[TRACE] [crates/kernel/src/proc/manager.rs:202] Kill Process {

pid: 8,

name: "stack",

parent: Some(

1,

),

status: Running,

ticks_passed: 3,

children: Map {

iter: Iter(

[],

),

},

status: Running,

context: StackFrame {

stack_top: VirtAddr(

0x3ff7ffff7f68,

),

cpu_flags: RFlags(

IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG | 0x2,

),

instruction_pointer: VirtAddr(

0xffffffff0000005c46,

),

```

regs: Registers
r15: 0xffffffff000020030, r14: 0xffffffff0000001d13, r13: 0xffffffff0000018af0,
r12: 0xffffffff0000018af0, r11: 0x0000000000000000, r10: 0x0000000000000002,
r9 : 0x0000000000000000d, r8 : 0x000000000000003fd, rdi: 0x000000000000003f8,
rsi: 0xffffffff0000001d29, rdx: 0x000000000000003f8, rcx: 0xffffffff0000001d29,
rbx: 0x00003ff7ffff90, rax: 0x0000000000000000, rbp: 0x00003ff7fffff80,
},
vm: Some(
  ProcessVm {
    stack: Stack {
      top: 0x3ff800000000,
      bot: 0x3ff7ffff7000,
    },
    memory_usage: "36 KiB",
    page_table: PageTable {
      addr: PhysFrame[4KiB](0x1f000),
      flags: Cr3Flags(
        0x0,
      ),
    },
  },
),
}
}
[DEBUG] [crates/kernel/src/proc/process.rs:84] Killing process stack#8 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:189] Process stack killed with exit code 0
[>] ps
  PID | PPID | Process Name | Ticks | Status
# 5 | # 1 | #3_test | 1930 | Ready
# 1 | # 0 | kernel | 1938 | Running
# 2 | # 1 | #0_test | 1933 | Ready
# 4 | # 1 | #2_test | 1931 | Ready
# 6 | # 1 | #4_test | 1929 | Ready
# 3 | # 1 | #1_test | 1932 | Ready
# 7 | # 1 | #5_test | 867 | Ready
Heap : 8.438 KiB used / 31.992 MiB free / 32.000 MiB total
Queue : [2, 3, 4, 5, 6, 7]
CPUs : [0: 1]
[>] test
[TRACE] [crates/kernel/src/proc/process.rs:72] New process #6_test#9 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB](0x3ff6ffff000), end: Page[4KiB](0x3ff700000000) }(1)

```

```
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0xffffffff000000c0a0,
  ),
  code_segment: SegmentSelector {
    index: 1,
    rpl: Ring0,
  },
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  stack_pointer: VirtAddr(
    0x3ff6ffff8,
  ),
  stack_segment: SegmentSelector {
    index: 2,
    rpl: Ring0,
  },
}
[>] ps
PID | PPID | Process Name | Ticks | Status
# 5 | # 1 | #3_test | 2664 | Ready
# 1 | # 0 | kernel | 2672 | Running
# 2 | # 1 | #0_test | 2667 | Ready
# 9 | # 1 | #6_test | 396 | Ready
# 4 | # 1 | #2_test | 2665 | Ready
# 6 | # 1 | #4_test | 2663 | Ready
# 3 | # 1 | #1_test | 2666 | Ready
# 7 | # 1 | #5_test | 1601 | Ready
Heap : 9.500 KiB used / 31.991 MiB free / 32.000 MiB total
Queue : [2, 3, 4, 5, 6, 7, 9]
CPUs : [0: 1]
[>] QEMU: Terminated
```

9. 实验结果分析

本次实验成功展示了 *YatSenOS* (YYOS) 内核的初始化及多任务管理能力。以下是对实验输出结果的详细描述：

1. 内核环境初始化：系统成功进入内核入口，完成了串口 (Serial)、日志系统 (Logger) 及全局描述符表 (GDT) 的初始化。内核堆空间 (Kernel Heap) 设置为 `32 MiB`，物理内存识别为 `12.093 GiB`。中断控制器 (IOAPIC) 正确启用了 `IRQ 0` (时钟) 与 `IRQ 4` (串口) 中断。
2. 多任务创建与调度：
 - 内核成功创建了初始进程 `kernel (PID 1)`，并初始化其虚拟内存空间。

- 通过 Shell 指令多次触发测试线程创建，系统成功孵化了从 `#0_test` 到 `#6_test` 的多个内核线程。
 - `ps` 命令显示所有线程状态正常，`Ticks` 计数持续增长，证明基于时钟中断的抢占式调度器运行稳定，就绪队列（Queue）管理正确。
3. 内存管理与大栈测试（**Huge Stack Test**）：在执行 `stack` 命令时，系统创建了 `stack (PID 8)` 进程。实验记录显示：
- 按需分页：当进程尝试访问 `0x3ff7fff7f88` 等超出初始物理映射的地址时，触发了页错误处理机制。
 - 栈动态增长：系统成功捕获异常并分配物理页，输出了从 `0x000` 到 `0xf00` 的连续内存访问记录，验证了栈空间的自动扩展功能。
4. 进程回收机制：`stack` 进程在完成测试后，通过内核函数正确触发了进程终止逻辑。日志显示 `Kill Process` 动作清理了该进程的上下文及虚拟内存资源，返回退出码 `0`，PID 被系统回收。
- 结论：实验输出结果符合预期。内核已具备稳定的物理/虚拟内存管理、可扩展的内核栈机制、以及基本的进程生命周期管理功能。

10. 思考题

10.1. 为什么在初始化进程管理器时需要将它置为正在运行的状态？能否通过将它置为就绪状态并放入就绪队列来实现？这样的实现可能会遇到什么问题？

进程管理器初始状态设计分析：

在初始化进程管理器时，将第一个进程（如 kernel 进程）直接置为 Running 状态而非 Ready 状态，是基于操作系统底层引导逻辑与硬件约束的必然选择。以下从三个维度进行深度分析：

10.1.1. 现状描述的客观性

内核在执行 `rust_main` 逻辑时，物理 CPU 已经在运行当前的代码指令。此时创建 kernel 进程并将其设为 Running，是对当前物理事实的软件同步：

- 若设为 Ready，则意味着软件层面认为 CPU 处于空闲或尚未开始该任务，这会导致逻辑上的自相矛盾。
- 只有置为 Running，进程管理器才能正确地将当前的执行上下文（寄存器、栈）与 PID 1 绑定。

10.1.2. 避免“引导死锁”（Bootstrapping Deadlock）

若尝试将初始进程放入就绪队列实现，会面临“先有鸡还是先有蛋”的问题：

- 中断依赖性:调度通常依赖时钟中断触发。若当前没有进程处于 Running 状态,当中断发生时,硬件尝试压栈保存现场将失去合法的栈支撑,导致系统触发 Double Fault 重启。
- 切换逻辑局限:标准的上下文切换函数 switch(from, to) 要求必须有一个“当前正在运行”的来源 (from)。如果没有 Running 进程,该函数将因找不到有效的旧栈指针而崩溃。

10.1.3. 可能遇到的风险与问题

如果强行通过“放入就绪队列再启动”的模式实现,可能会遇到以下技术难题:

潜在问题	详细描述
上下文丢失	初始化代码中包含大量“仅限一次”的硬件配置,若在配置中途被切换走,可能导致硬件状态不一致。
栈空间非法	引导阶段使用的是临时栈,若不立即将其合法化为运行进程,该段空间可能被后续分配逻辑污染。
空队列恐慌	若就绪队列因某种异常未及时更新,调度器可能从空队列中抓取任务,导致内核进入不可恢复的 Panic 状态。

结论:将初始进程直接标记为 Running,是为了建立一个合法的执行基准,使得后续的“中断 -> 调度 -> 切换”链条能够平滑启动,是保证内核从静态引导转向动态多任务运行的关键。

10.2. 在 src/proc/process.rs 中,有两次实现 Deref 和一次实现 DerefMut 的代码,它们分别是为了什么?使用这种方式提供了什么便利?

在 [process.rs](yyos/crates/kernel/src/proc/process.rs) 中,存在以下 Deref/DerefMut 实现:
第一次 Deref (Process → RwLock):

```
impl Deref for Process {
    type Target = RwLock<ProcessInner>;
    fn deref(&self) -> &Self::Target { &self.inner }
}
```

允许直接调用 process.read() / process.write() 而非 process.inner.read()。

第二次 Deref (ProcessInner → ProcessData):

```
impl Deref for ProcessInner {
    type Target = ProcessData;
    fn deref(&self) -> &Self::Target {
        self.proc_data.as_ref().expect(...)
    }
}
```

允许直接访问 `ProcessData` 的方法，如 `inner.env(key)`。

`DerefMut (ProcessInner → ProcessData)`：

```
impl DerefMut for ProcessInner {  
    fn deref_mut(&mut self) -> &mut Self::Target {  
        self.proc_data.as_mut().expect(...)  
    }  
}
```

允许可变访问 `ProcessData`，如 `inner.set_env(key, val)`。

这种链式 `Deref` 设计实现了「智能指针」语义，在保持封装性的同时简化了多层嵌套结构的访问语法，体现了 Rust 中「组合优于继承」的典型实践。

10.3. 中断的处理过程默认是不切换栈的，即在中断发生前的栈上继续处理中断过程，为什么在处理缺页异常和时钟中断时需要切换栈？如果不为它们切换栈会分别带来哪些问题？请假设具体的场景、或通过实际尝试进行回答。

缺页异常与时钟中断中的栈切换必要性分析：

在 [x86_64] 架构下，中断发生时默认不切换栈（除非特权级发生改变）。但在处理缺页异常（Page Fault）和时钟中断（Timer Interrupt）时，内核通常会显式配置 [IST (Interrupt Stack Table)] 或手动切换栈。其必要性分析如下：

10.3.1. 处理缺页异常（Page Fault）时的栈切换

原因：防止内核栈溢出导致的双重异常（**Double Fault**）

缺页异常可能在任何地方发生，包括内核自身正在进行深度函数调用或在栈上分配大型局部变量时。

- 假设场景：内核函数正在执行一个深递归操作，此时栈指针 [RSP] 距离页边界仅剩 [16 字节]。此时函数尝试压入一个新变量，触发了缺页异常。
- 不切换栈的问题：若不切换栈，CPU 会尝试在当前已经触底的 [RSP] 下方压入 [Interrupt Stack Frame]（约 40 字节）。由于此时栈已经没有空间且触发异常的正是“栈写入”，CPU 会在处理“缺页异常”的过程中再次触发“缺页异常”，最终导致 [Double Fault]，硬件直接重启，内核无法打印任何调试信息。
- 解决方案：通过 [IST] 为缺页异常分配专门的、干净的栈空间，确保即便内核栈已经爆满，异常处理程序依然能安全运行并尝试修复映射或优雅地 Panic。

10.3.2. 处理时钟中断（Timer Interrupt）时的栈切换

原因：支持多任务并发与内核抢占（**Preemption**）

时钟中断是任务调度的引擎。处理它时切换栈不是因为硬件限制，而是为了软件架构的解耦。

- 假设场景：进程 A 正在内核态运行，时钟中断发生。此时调度器决定切换到进程 B 运行。
- 不切换栈的问题：若复用当前栈，时钟中断的处理逻辑、寄存器现场以及后续调度器的局部变量都会压在进程 A 的内核栈上。
 1. 逻辑耦合：进程 A 的栈将不得不保留“调度器”的状态，导致栈空间碎片化。
 2. 嵌套风险：如果在处理时钟中断（还在 A 的栈上）时又发生了一个高优先级中断，栈的深度将变得不可控。
 3. 隔离性差：若所有 CPU 的时钟处理都混杂在当前进程栈中，内核将难以实现统一的中断处理模型。
- 实际尝试观察：在 [YYOS] 实验中，如果不为中断配置独立的 [Privilege Stack] 或 [IST]，在频繁输入指令（Shell）同时触发时钟中断时，你会观察到内核经常报出 [Stack Overflow] 或者上下文恢复后寄存器数值错乱，这正是因为不同任务的执行流在同一个物理栈上发生了“踩踏”。

10.3.3. 总结对比

中断类型	不切换栈的后果	切换栈带来的收益
缺页异常	导致 [Double Fault]，系统直接死锁/重启。	保证异常处理逻辑始终有可用栈空间。
时钟中断	任务状态污染，增加栈溢出风险，难以实现内核抢占。	实现中断上下文与进程上下文的解耦，支持平滑调度。

结论：栈切换是内核“防御性编程”的一种体现。对于异常，它是为了生存（避免硬件崩溃）；对于时钟中断，它是为了秩序（支持复杂的任务切换）。



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验四：用户程序与系统调用

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 理解用户态程序的编译、链接、装载与运行过程，掌握 ELF 用户程序的加载方法。
2. 理解 x86_64 特权级机制，为用户进程建立独立页表、用户栈和 Ring 3 执行上下文。
3. 掌握系统调用的调用约定，使用 `int 0x80` 实现从用户态到内核态的受控切换。
4. 实现用户态动态内存分配、标准输入输出以及进程创建、等待和退出等基础服务。
5. 编写用户态 Shell，并使用递归阶乘程序验证系统调用、进程调度和栈按需增长功能。

二. 实验环境

项目	配置
操作系统	YatSenOS / YYOS
开发语言	Rust (<code>no_std</code> 、 <code>no_main</code>)
目标架构	x86_64
引导环境	UEFI + OVMF
虚拟机	QEMU
报告工具	Typst

三. 实验内容

本次实验在实验三的进程调度与缺页异常处理基础上，进一步加入用户态执行环境。整体实验可以分为以下部分：

1. 在工作区中加入用户程序、用户态库和系统调用定义三个 crate。
2. 由 Bootloader 从 ESP 分区的 `APP` 目录读取用户 ELF，并通过 `BootInfo` 传递给内核。
3. 为用户进程克隆内核页表，映射用户 ELF 和用户栈，构造 Ring 3 中断返回现场。
4. 注册 `0x80` 号软件中断，并实现系统调用参数提取、分发与返回值传递。
5. 在用户态库中封装输入输出、堆分配、进程创建、等待和退出功能。
6. 编写 Shell 与阶乘测试程序，完成用户程序的交互运行。

四. 实验原理

1. 用户态与内核态

x86_64 使用特权级保护关键资源，Ring 0 拥有最高权限，Ring 3 用于运行普通用户程序。用户程序不能直接执行特权指令，也不能直接访问未设置 `USER_ACCESSIBLE` 标志的内核页面。需要内核服务时，用户程序通过系统调用进入 Ring 0，内核检查参数并完成相应操作，随后恢复用户上下文。

用户进程的页表由内核页表的最高级页表克隆而来，因此内核映射仍存在于该地址空间中，系统调用和中断发生后内核代码可以继续执行；但内核页没有用户访问权限，所以 Ring 3 程序不能直接读写内核空间。

2. 用户程序的 ELF 装载

用户程序使用独立目标文件 `x86_64-unknown-yyos.json` 和链接脚本 `app.ld` 编译为 ELF。由于当前内核尚未实现磁盘文件系统，Bootloader 在退出 UEFI Boot Services 之前完成应用文件读取，并把应用名称和 `ElfFile` 保存到 `BootInfo.loaded_apps`。

内核创建用户进程时，需要完成以下步骤：

1. 从应用列表中按名称找到 ELF。
2. 克隆内核进程的页表顶层结构。
3. 根据 ELF Program Header 映射代码段和数据段，并添加 `USER_ACCESSIBLE` 权限。
4. 分配用户栈，设置栈顶地址。
5. 将入口地址、用户栈、用户代码段和用户数据段选择子写入进程上下文。
6. 将进程加入进程表和就绪队列，等待调度。

3. 系统调用约定

本实验采用与 x86_64 Linux 相近的寄存器传参方式：

寄存器	用途
<code>rax</code>	系统调用号，同时保存返回值
<code>rdi</code>	第一个参数
<code>rsi</code>	第二个参数

rdx	第三个参数
-----	-------

用户态库通过内联汇编执行 `int 0x80`。CPU 根据 IDT 找到系统调用处理程序，并从 Ring 3 切换到 Ring 0。内核从保存的 `ProcessContext` 中读取寄存器，构造 `SyscallArgs`，调用相应服务函数，最后把返回值写回 `rax`。

五. 实验过程

1. 步骤一：合并实验代码与配置工作区

本次实验新增了三个主要目录：

- `crates/app`：存放 `hello`、`shell` 和 `factorial` 等用户程序。
- `crates/lib`：提供用户态运行库、输入输出和堆分配接口。
- `crates/syscall`：统一定义内核与用户态共享的系统调用号和调用宏。

根目录 `Cargo.toml` 将这些 crate 加入 workspace，并声明本地依赖：

```
members = [
    "crates/app/*",
    "crates/boot",
    "crates/elf",
    "crates/kernel",
    "crates/lib",
    "crates/syscall",
]

[workspace.dependencies]
boot    = { path = "crates/boot", package = "yyos_boot" }
elf     = { path = "crates/elf", package = "yyos_elf" }
lib     = { path = "crates/lib", package = "yylib" }
syscall_def = { path = "crates/syscall", package = "yyos_syscall" }
```

同时在 `boot.conf` 中启用应用加载：

```
load_apps=1
```

`Makefile` 会遍历 `crates/app` 下的应用，使用用户程序目标配置完成编译，再将生成的 ELF 复制到 `esp/APP`：

```
$(ESP)/APP: target/x86_64-unknown-yyos/$(MODE)
@for app in $(APPS); do \
    mkdir -p $(ESP)/APP; \
    cp $</yyos_$$app $(ESP)/APP/$$app; \
done
```

2. 步骤二：由 Bootloader 加载用户程序

在 `boot/src/lib.rs` 中定义应用信息，并将应用列表加入 `BootInfo`：

```
pub struct App<'a> {
    pub name: ArrayString<16>,
    pub elf: xmas_elf::ElfFile<'a>,
}

pub type AppList = ArrayVec<App<'static>, 16>;

pub struct BootInfo {
    // ...
    pub loaded_apps: Option<AppList>,
}
```

`load_apps` 打开 ESP 分区中的 `\APP\` 目录，逐项过滤目录并读取普通文件，然后将文件内容解析为 ELF：

```
pub fn load_apps() -> AppList {
    let mut root = open_root();
    let mut buf = [0; 8];
    let path = uefi::CStr16::from_str_with_buf("\\APP\\", &mut buf).unwrap();

    let mut handle = match root
        .open(path, FileMode::Read, FileAttribute::empty())
        .expect("Failed to open APP dir")
        .into_type()
        .expect("Failed to get file type")
    {
        FileType::Dir(dir) => dir,
        _ => panic!("APP is not a directory"),
    };

    // 遍历目录，读取文件并构造 App { name, elf }
    // ...
}
```

Bootloader 根据 `load_apps` 配置决定是否加载应用，并在退出 Boot Services 后将列表传递给内核：

```
let loaded_apps = if config.load_apps {
    Some(load_apps())
} else {
    None
};

let bootinfo = BootInfo {
    memory_map: mmap_owned.entries().copied().collect(),
    physical_memory_offset: config.physical_memory_offset,
    system_table,
    loaded_apps,
};
```

这样，虽然内核暂时没有文件系统，也能获得用户程序的名称和 ELF 内容。

3. 步骤三：创建 Ring 3 用户进程

3.1. 保存应用列表

初始化进程管理器时，将 `BootInfo` 中的应用列表交给 `ProcessManager`：

```
let app_list = boot_info.loaded_apps.clone();
manager::init(kproc, app_list);
```

随后可以通过 `list_app` 遍历并输出应用名称，也可以由 `spawn` 按名称查找 ELF：

```
pub fn spawn(name: &str) -> Option<ProcessId> {
    let app = x86_64::instructions::interrupts::without_interrupts(|| {
        let app_list = get_process_manager().app_list()?;
        app_list.iter().find(|app| app.name.eq(name))
    })?;

    elf_spawn(name.to_string(), &app.elf)
}
```

将 `elf_spawn` 单独封装，可以让后续实验把 ELF 来源从 Bootloader 应用列表替换为内核文件系统，而不需要改变进程创建的核心逻辑。

3.2. 加载 ELF 与初始化用户栈

`ProcessManager::spawn` 克隆内核页表，创建 PCB，将用户 ELF 映射到新地址空间，并初始化用户栈：

```

pub fn spawn(
    &self,
    elf: &ElfFile,
    name: String,
    parent: Option<Weak<Process>>,
    proc_data: Option<ProcessData>,
) -> ProcessId {
    let kproc = self.get_proc(&KERNEL_PID).unwrap();
    let page_table = kproc.read().clone_page_table();
    let proc = Process::new(
        name,
        parent,
        Some(ProcessVm::new(page_table)),
        proc_data,
    );

    let entry_point;
    {
        let mut inner = proc.write();
        let mapper = &mut inner.vm_mut().page_table.mapper();
        let frame_alloc = &mut *get_frame_alloc_for_sure();
        let physical_offset = *crate::memory::PHYSICAL_OFFSET.get().unwrap();

        elf::load_elf(elf, physical_offset, mapper, frame_alloc, true)
            .expect("Failed to load ELF for new process");
        entry_point = VirtAddr::new(elf.header.pt2.entry_point());
    }

    let stack_top = proc.alloc_init_stack();
    proc.write().init_stack_frame(entry_point, stack_top);

    let pid = proc.pid();
    self.add_proc(pid, proc);
    self.push_ready(pid);
    pid
}

```

这里把 ELF 加载和栈分配分成两个作用域，及时释放进程写锁和帧分配器锁，避免后续再次申请相同锁时发生死锁。

用户 ELF 和栈页必须带有 `USER_ACCESSIBLE` 标志。用户堆还需要添加 `NO_EXECUTE`，避免把数据页作为代码执行：

```

let flags = PageTableFlags::PRESENT
    | PageTableFlags::WRITABLE
    | PageTableFlags::USER_ACCESSIBLE
    | PageTableFlags::NO_EXECUTE;

```

3.3. 配置 GDT 和用户上下文

GDT 中加入 Ring 3 代码段与数据段：

```
let user_code_selector = gdt.append(Descriptor::user_code_segment());
let user_data_selector = gdt.append(Descriptor::user_data_segment());
```

初始化进程上下文时，将 ELF 入口写入 `instruction_pointer`，将用户栈顶写入 `stack_pointer`，并设置 Ring 3 段选择子。调度器恢复该上下文并执行 `iretq` 后，CPU 即进入用户态执行。

4. 步骤四：注册并分发系统调用

4.1. 定义系统调用号

`crates/syscall` 同时被内核和用户态库依赖，避免两侧系统调用号不一致：

```
#[repr(usize)]
#[derive(Clone, Debug, FromPrimitive)]
pub enum Syscall {
    Read = 0,
    Write = 1,
    GetPid = 39,
    Spawn = 59,
    Exit = 60,
    WaitPid = 61,
    ListApp = 65531,
    Stat = 65532,
    Allocate = 65533,
    Deallocate = 65534,
    Unknown = 65535,
}
```

4.2. 注册 `int 0x80`

系统调用中断门设置为 Ring 3 可调用，并使用独立 IST 栈：

```
pub unsafe fn register_idt(idt: &mut InterruptDescriptorTable) {
    idt[consts::Interrupts::Syscall as u8]
        .set_handler_fn(syscall_handler)
        .set_stack_index(gdt::SYSCALL_IST_INDEX)
        .set_privilege_level(x86_64::PrivilegeLevel::Ring3);
}
```

如果没有把 DPL 设置为 3，用户态执行 `int 0x80` 会因为权限不足触发 General Protection Fault。

4.3. 参数分发

内核从进程上下文中提取寄存器并分发服务：

```
let args = SyscallArgs::new(
    Syscall::from(context.regs.rax),
    context.regs.rdi,
    context.regs.rsi,
    context.regs.rdx,
);

match args.syscall {
    Syscall::Read => context.set_rax(sys_read(&args)),
    Syscall::Write => context.set_rax(sys_write(&args)),
    Syscall::Spawn => context.set_rax(sys_spawn(&args)),
    Syscall::Exit => sys_exit(&args, context),
    Syscall::WaitPid => context.set_rax(sys_waitpid(&args) as usize),
    Syscall::Stat => context.set_rax(sys_stat()),
    Syscall::ListApp => context.set_rax(sys_list_app(&args)),
    // ...
}
```

`set_rax` 修改的是将要恢复的用户上下文，所以用户程序从中断返回后，可以直接从 `rax` 得到系统调用返回值。

5. 步骤五：实现用户态库

5.1. 系统调用宏

用户态库使用内联汇编准备寄存器并触发中断：


```

#[inline(always)]
pub fn syscall3(
    n: Syscall,
    arg0: usize,
    arg1: usize,
    arg2: usize,
) -> usize {
    let ret: usize;
    unsafe {
        asm!(
            "int 0x80",
            in("rax") n as usize,
            in("rdi") arg0,
            in("rsi") arg1,
            in("rdx") arg2,
            lateout("rax") ret,
        );
    }
    ret
}

```

在此基础上，`sys_write`、`sys_read`、`sys_spawn`、`sys_wait_pid` 和 `sys_exit` 等函数只需负责参数转换与返回值解释。

5.2. 动态内存分配

用户态全局分配器把 Rust 的 `alloc` 和 `dealloc` 操作转换为系统调用：

```

unsafe impl alloc::alloc::GlobalAlloc for KernelAllocator {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        crate::sys_allocate(&layout)
    }

    unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout) {
        crate::sys_deallocate(ptr, &layout);
    }
}

#[global_allocator]
static ALLOCATOR: KernelAllocator = KernelAllocator;

```

内核在 `0x4000_0000_0000` 建立 1 MiB 用户堆，并通过 `LockedHeap` 完成分配。当前实现由内核统一管理这一用户堆，结构较简单；后续可以改为每进程独立堆或实现 `brk`。

5.3. 标准输入输出

`println!` 最终调用 `sys_write(1, ...)`，内核把用户缓冲区转换为字节切片并输出。`stdin().read_line()` 则反复进行非阻塞读取，在用户态处理回车、退格、可打印字符和回显：

```
match ch {
  b'\r' | b'\n' => {
    sys_write(1, b"\n");
    return s;
  }
  0x08 | 0x7f => {
    if !s.is_empty() {
      s.pop();
      sys_write(1, b"\x08 \x08");
    }
  }
  0x20..=0x7e => {
    s.push(ch as char);
    sys_write(1, &[ch]);
  }
  _ => {}
}
```

控制字符在用户态处理，使内核 `read` 服务保持简单且非阻塞。

6. 步骤六：实现进程退出、创建与等待

用户程序入口宏调用 `main`，随后把返回值传给 `sys_exit`；发生 panic 时则以返回码 1 退出：

```
#[macro_export]
macro_rules! entry {
  ($fn:ident) => {
    #[unsafe(export_name = "_start")]
    pub extern "C" fn __impl_start() {
      let ret = $fn();
      $crate::sys_exit(ret);
    }
  };
}
```

内核处理 `Exit` 时，将当前进程设置为 `Dead`，记录返回码，释放 `ProcessData` 和 `ProcessVm` 的所有权，然后直接调度下一个进程：

```
pub fn exit(ret: isize, context: &mut ProcessContext) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        let manager = get_process_manager();
        manager.kill_current(ret);
        manager.switch_next(context);
    })
}
```

`Spawn` 根据应用名称创建进程并返回 PID；`WaitPid` 查询目标进程状态，进程尚未结束时返回负值，用户态继续轮询，结束后返回退出码。

内核入口创建 Shell 作为初始用户进程：

```
pub fn kernel_main(boot_info: &'static boot::BootInfo) -> ! {
    yyos::init(boot_info);
    yyos::wait(spawn_init());
    yyos::shutdown();
}

pub fn spawn_init() -> proc::ProcessId {
    proc::list_app();
    proc::spawn("shell").unwrap()
}
```

7. 步骤七：实现用户态 Shell

Shell 提供以下命令：

<code>help</code>	显示命令帮助；按照实验要求还应输出学号 <code>24312063</code>
<code>ls / apps</code>	列出 Bootloader 已加载的用户程序
<code>ps / stat</code>	显示进程表、就绪队列和内核堆状态
<code>run <name></code>	创建指定用户程序，等待退出并输出返回码
<code>clear</code>	使用 ANSI 转义序列清屏
<code>exit</code>	退出 Shell

核心运行逻辑如下：

```
fn cmd_run(name: &str) {
    let pid = sys_spawn(name);
    if pid == 0 {
        println!("Error: Failed to spawn '{}', name);
        return;
    }

    loop {
        let result = sys_wait_pid(pid);
        if result >= 0 {
            println!("Process {} exited with code {}", pid, result);
            break;
        }
        core::hint::spin_loop();
    }
}
```

Shell 既可以使用 `run hello`，也可以直接输入 `hello` 运行程序。

8. 步骤八：阶乘程序与栈增长测试

阶乘程序递归计算：

```
const MOD: u64 = 1000000007;

fn factorial(n: u64) -> u64 {
    if n == 0 {
        1
    } else {
        n * factorial(n - 1) % MOD
    }
}
```

程序从标准输入读取 `n`，调用 `sys_stat` 输出系统状态，再打印模意义下的阶乘结果。较大的递归深度会不断访问更低的栈地址，从而触发 Page Fault。实验三实现的缺页处理程序检查故障地址属于当前用户栈后，分配带有 `USER_ACCESSIBLE` 标志的新页面，使递归可以继续。

教程给出的最大测例为：

```
Input n: 999999
The factorial of 999999 under modulo 1000000007 is 128233642.
```

该测例预计占用约 3929 个页面，即约 15.3 MiB，可同时验证用户输入、系统调用、进程调度和栈动态增长。

六. 调试过程

1. 缺少类型与依赖导入

合并代码后曾出现 `Weak` 、 `Arc` 、 `Vec` 和 `ElfFile` 不在作用域，以及内核 `crate` 未链接 `xmas-elf` 等编译错误。对应处理为：

```
use alloc::{
    string::{String, ToString},
    sync::{Arc, Weak},
    vec::Vec,
};
use xmas_elf::ElfFile;
```

同时在 `crates/kernel/Cargo.toml` 中加入 `workspace` 的 `xmas-elf` 依赖。此问题说明 `no_std` 环境不会自动导入 `std` 中常用的容器和智能指针，必须显式从 `alloc` 引入。

2. ELF 加载参数不一致

为了区分内核映射和用户映射，`elf::load_elf` 增加了 `user_access: bool` 参数。所有调用点必须同步更新：

```
// 内核 ELF
elf::load_elf(&elf, offset, mapper, allocator, false);

// 用户 ELF
elf::load_elf(&elf, offset, mapper, allocator, true);
```

如果遗漏第五个参数会导致 `E0061`；如果用户程序使用 `false`，Ring 3 访问代码页时会立即触发保护违例。

3. 创建进程时的锁顺序

最初在持有 `ProcessInner` 写锁和帧分配器锁时继续调用栈分配函数，后者会再次申请这些锁，存在自锁风险。最终将 ELF 加载放入独立作用域，先释放锁，再初始化栈，从而保证锁顺序清晰。

4. 特权级与页面权限

用户程序能成功进入 Ring 3 需要同时满足：

- GDT 中存在用户代码段和用户数据段。
- 中断返回现场的段选择子 RPL 为 Ring 3。
- 用户 ELF、用户栈和用户堆页面具有 `USER_ACCESSIBLE`。
- `0x80` 中断门的 DPL 为 Ring 3。

任意一处遗漏都会表现为 General Protection Fault 或 Page Fault，因此调试时需要同时检查段选择子、页表标志和 IDT 权限。

七. 实验结果与分析

从实现结构上看，本次实验已经建立了用户程序执行链路的主要模块：

1. Bootloader 能够读取 `APP` 目录并把 ELF 列表传入内核。
2. 内核能够根据名称创建用户进程，映射 ELF、用户栈并设置 Ring 3 上下文。
3. 用户态通过 `int 0x80` 调用内核，内核完成读写、内存分配、进程管理和状态输出。
4. Shell 能够列出应用和进程、创建子进程、等待退出并显示返回值。
5. 运行结果展示：

```
[DEBUG] [crates/kernel/src/proc/mod.rs:168] Spawned process: shell#2
```

```
YYOS Shell v0.4
```

```
Type 'help' for available commands.
```

```
yyos> help
```

```
学号：24312063
```

```
Available commands:
```

```
help      - Show this help message
ls / apps  - List all available user programs
ps / stat  - List all running processes
run <name> - Run a user program by name
clear      - Clear the screen
exit       - Exit the shell
```

```
You can also type a program name directly to run it.
```

```
yyos> apps
```

```
Available applications:
```

```
[app] shell
[app] hello
[app] factorial
```

```

yyos> ps
--- Process Status ---
PID | PPID | Process Name | Ticks | Status
# 2 | # 1 | shell | 6300 | Running
# 1 | # 0 | kernel | 6304 | Ready
Heap : 4.133 KiB used / 31.996 MiB free / 32.000 MiB total
Queue : [1]
CPUs : [0: 2]
yyos> run hello
[TRACE] [crates/kernel/src/proc/process.rs:73] New process hello#3 created.
[TRACE] [crates/elf/src/lib.rs:80] Loading ELF file... (user_access=true)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100000000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100001000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100004000) with flags PageTableFlags(PRESENT
| WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:52] Page Range: PageRange { start: Page[4KiB](0x3ffcff000), end: Page[4KiB]
(0x3ffd00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0x111100001020,
  ),

  code_segment: SegmentSelector {
    index: 5,
    rpl: Ring3,
  },
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  stack_pointer: VirtAddr(
    0x3ffcff000000,
  ),
  stack_segment: SegmentSelector {
    index: 6,
    rpl: Ring3,
  },
}

```

```

[TRACE] [crates/kernel/src/proc/manager.rs:275] New Process {
  pid: 3,
  name: "hello",
  parent: Some(
    2,
  ),
  status: Ready,
  ticks_passed: 0,
  children: Map {
    iter: Iter(
      [],
    ),
  },
  status: Ready,
  context: StackFrame {
    stack_top: VirtAddr(
      0x3ffcffff8,
    ),
    cpu_flags: RFlags(
      IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    instruction_pointer: VirtAddr(
      0x111100001020,
    ),
    regs: Registers
    r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
    r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
    r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi: 0x0000000000000000,
    rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
    rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
  },
  vm: Some(
    ProcessVm {
      stack: Stack {
        top: 0x3ffd00000000,
        bot: 0x3ffcffff000,
      },
      memory_usage: "4 KiB",
      page_table: PageTable {
        addr: PhysFrame[4KiB](0x173000),
        flags: Cr3Flags(
          0x0,
        ),
      },
    ),
  ),
}

```



```

[DEBUG] [crates/kernel/src/proc/mod.rs:168] Spawned process: hello#3
Hello, world!!!
[TRACE] [crates/kernel/src/proc/manager.rs:203] Kill Process {
  pid: 3,
  name: "hello",
  parent: Some(
    2,
  ),
  status: Running,
  ticks_passed: 0,
  children: Map {
    iter: Iter(
      [],
    ),
  },
  status: Running,
  context: StackFrame {
    stack_top: VirtAddr(
      0x3ffcffffff8,
    ),
    cpu_flags: RFlags(
      IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    instruction_pointer: VirtAddr(
      0x111100001020,
    ),
    regs: Registers
    r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
    r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
    r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi: 0x0000000000000000,
    rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
    rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
  },
  vm: Some(
    ProcessVm {
      stack: Stack {
        top: 0x3ffd00000000,
        bot: 0x3ffcffff000,
      },
      memory_usage: "4 KiB",
      page_table: PageTable {
        addr: PhysFrame[4KiB](0x173000),
        flags: Cr3Flags(
          0x0,
        ),
      },
    ),
  ),
}

```

```

[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process hello#3 with ret code: 233
[TRACE] [crates/kernel/src/proc/process.rs:190] Process hello killed with exit code 233
Running 'hello' (PID: 3)...
Process 3 exited with code 233.
yyos> run factorial
[TRACE] [crates/kernel/src/proc/process.rs:73] New process factorial#4 created.
[TRACE] [crates/elf/src/lib.rs:80] Loading ELF file... (user_access=true)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100000000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100001000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100004000) with flags PageTableFlags(PRESENT
| WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:52] Page Range: PageRange { start: Page[4KiB](0x3ffbffff000), end: Page[4KiB]
(0x3ffc00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0x1111000012c0,
  ),
  code_segment: SegmentSelector {
    index: 5,
    rpl: Ring3,
  },
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  stack_pointer: VirtAddr(
    0x3ffbffff8,
  ),
  stack_segment: SegmentSelector {
    index: 6,
    rpl: Ring3,
  },
}

```

```

[TRACE] [crates/kernel/src/proc/manager.rs:275] New Process {
  pid: 4,
  name: "factorial",
  parent: Some(
    2,
  ),
  status: Ready,
  ticks_passed: 0,
  children: Map {
    iter: Iter(
      [],
    ),
  },
  status: Ready,
  context: StackFrame {
    stack_top: VirtAddr(
      0x3ffbffff8,
    ),
    cpu_flags: RFlags(
      IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    instruction_pointer: VirtAddr(
      0x111100012c0,
    ),
    regs: Registers
    r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
    r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
    r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi: 0x0000000000000000,
    rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
    rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
  },
  vm: Some(
    ProcessVm {
      stack: Stack {
        top: 0x3ffc00000000,
        bot: 0x3ffbffff000,
      },
      memory_usage: "4 KiB",
      page_table: PageTable {
        addr: PhysFrame[4KiB](0x181000),
        flags: Cr3Flags(
          0x0,
        ),
      },
    ),
  ),
}

```

```

[DEBUG] [crates/kernel/src/proc/mod.rs:168] Spawned process: factorial#4
Input n: Running 'factorial' (PID: 4)...
10
PID | PPID | Process Name | Ticks | Status
# 4 | # 2 | factorial | 3361 | Running
# 2 | # 1 | shell | 15662 | Ready
# 1 | # 0 | kernel | 15666 | Ready
Heap : 5.102 KiB used / 31.995 MiB free / 32.000 MiB total
Queue : [2, 1]
CPUs : [0: 4]
The factorial of 10 under modulo 1000000007 is 3628800.
[TRACE] [crates/kernel/src/proc/manager.rs:203] Kill Process {
  pid: 4,
  name: "factorial",
  parent: Some(
    2,
  ),
  status: Running,
  ticks_passed: 3362,
  children: Map {
    iter: Iter(
      [],
    ),
  },
  status: Running,
  context: StackFrame {
    stack_top: VirtAddr(
      0x3ffbffff70,
    ),
    cpu_flags: RFlags(
      IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG | 0x2,
    ),
    instruction_pointer: VirtAddr(
      0x111100001150,
    ),
    regs: Registers
    r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
    r12: 0x0000000000000000, r11: 0x00003ffbffff18, r10: 0x0000111100000344,
    r9 : 0x0000000000000002, r8 : 0x0000000000000002, rdi: 0x0000000000000000,
    rsi: 0x0000400000000080, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
    rbx: 0x0000400000000080, rax: 0x0000000000000000, rbp: 0x00003ffbffffe0,
  },
}

```

```

vm: Some(
  ProcessVm {
    stack: Stack {
      top: 0x3ffc00000000,
      bot: 0x3ffbffff000,
    },
    memory_usage: "4 KiB",
    page_table: PageTable {
      addr: PhysFrame[4KiB](0x181000),
      flags: Cr3Flags(
        0x0,
      ),
    },
  },
),
}
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process factorial#4 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:190] Process factorial killed with exit code 0
Process 4 exited with code 0.
yyos>

```

八. 思考题

1. 1. 是否可以在内核线程中使用系统调用，并实现相同的退出能力？

从指令层面看，Ring 0 代码可以执行 `int 0x80`，但这不是合适的内核设计。系统调用的主要作用是为低特权级程序提供受控的内核入口；内核线程本来就可以直接调用 `kill_current`、`process_exit` 等内核函数，再经过一次中断只会增加现场保存、IDT 分发和参数转换开销。

此外，系统调用入口通常假设调用者来自 Ring 3，并依赖特权级切换后的栈布局。Ring 0 调用时不会发生同样的 CPL 切换，若处理代码错误地假设存在用户栈字段，可能破坏上下文。因此可以专门兼容内核调用，但更合理的做法是让系统调用服务和内核 API 复用同一组底层函数。

2. 2. 为什么需要克隆内核页表？系统调用时使用哪一张页表？用户程序能否访问内核空间？

中断或系统调用发生后，CPU 不会自动更换 CR3，因此进入内核态后仍使用当前用户进程的页表。若用户页表中没有映射内核代码、内核数据和中断栈，CPU 刚进入中断处理函数就会再次缺页。

克隆内核页表的顶层映射，可以让每个用户进程都保留一致的内核高地址空间，同时拥有独立的用户地址空间。系统调用期间仍使用该用户进程自己的页表，只是 CPU 已切换到 Ring 0。

内核页面虽然存在于页表中，但没有 `USER_ACCESSIBLE` 标志。Ring 3 访问这些页面时，分页硬件会产生带有 `USER_MODE` 和 `PROTECTION_VIOLATION` 的 Page Fault，内核应当拒绝处理并终止或报告该非法访问。因此“映射存在”不等于“用户可访问”。

3.3. 为什么 `still_alive` 判断进程状态时需要关闭中断？

`still_alive` 需要读取进程表、取得进程引用并检查状态。时钟中断可能在这些步骤之间触发调度，退出系统调用也可能把进程改为 `Dead` 并释放其数据。如果不保护这一临界区，检查结果可能基于互相不一致的状态，甚至与进程清理过程交错。

在当前单核实验中，关闭中断可以阻止时钟调度和其他中断处理打断检查，使“查找进程并读取状态”成为一个原子操作。未来进入多核环境后，仅关闭本地中断还不够，还需要锁或原子变量来同步其他 CPU。

4.4. 解释普通 Linux C 程序从编译到退出的过程

对如下程序执行 `gcc hello.c -o hello` 时，预处理器先展开头文件和宏，编译器把 C 代码转换为汇编，汇编器生成目标文件，链接器再把目标文件、启动代码和所需动态库信息组合为 ELF。

Shell 运行 `./hello` 时，通常先通过 `fork` 或类似机制创建子进程，再由 `execve` 系统调用让内核读取 ELF。内核建立新地址空间，映射代码段、数据段、用户栈和动态链接器，将参数与环境变量放入栈中，然后从 ELF 入口 `_start` 开始执行。

`_start` 并不等于 `main`。C 运行库会初始化运行环境，再调用 `main`。程序中的 `printf` 先在用户态完成格式化与缓冲处理，最终通过 `write` 系统调用请求内核向标准输出文件描述符写数据。`main` 返回后，运行库执行清理并调用 `exit` 或 `exit_group`，内核记录退出状态、释放进程资源并唤醒等待它的父进程。Shell 通过 `wait` 系列系统调用取得返回码，然后重新显示提示符。

5.5. `hlt` 做了什么？为什么内核等待可以使用它，而用户态 `wait_pid` 不可以？

`hlt` 使当前 CPU 停止执行指令，直到出现可响应的中断、NMI 或复位。内核等待初始进程退出时，如果暂时无事可做，执行 `hlt` 可以避免忙等；时钟或设备中断到来后，CPU 被唤醒并继续调度。

`hlt` 是特权指令，Ring 3 执行会触发 General Protection Fault，所以用户态 `wait_pid` 不能直接使用。用户态可以轮询系统调用、主动让出 CPU，或在实现阻塞机制后由内核把等待进程置为 `Blocked`，待子进程退出时再唤醒。

6.6. 缺少 TSS 特权级栈为何会在串口输入时触发异常？

当 CPU 在 Ring 3 运行用户程序时，串口输入触发硬件中断。CPU 需要从 CPL 3 切换到 CPL 0，并在进入内核前保存用户态的 `SS`、`RSP`、`RFLAGS`、`CS` 和 `RIP`。新的 Ring 0 栈顶来自当前 TSS 的 `privilege_stack_table[0]`。

如果该项未初始化，CPU 可能取得 0 或其他非法地址作为内核栈，并尝试在该地址附近压入中断现场，于是触发 Page Fault。处理 Page Fault 时还需要继续压栈；若栈仍不可用，异常处理过程会升级为 Double Fault，Double Fault 也无法建立现场时最终形成 Triple Fault 并重启。

进程切换、普通内存分配或某些系统调用可能暂时没有暴露问题，是因为它们发生在 Ring 0，或使用了显式配置的 IST 栈；而串口中断恰好在用户程序运行期间到来，必须完成 Ring 3 到 Ring 0 的硬件栈切换，所以最先暴露 TSS 特权级栈缺失。调试日志中缺页前的中断向量以及接近零地址的异常栈地址，正是这一问题的线索。

九. 实验总结

本次实验把前几次实验中的启动、中断、分页、缺页处理和进程调度连接成了完整的用户程序运行环境。实现过程中最关键的认识是：系统调用并不只是一个中断处理函数，它依赖页表共享、权限隔离、TSS 特权级栈、进程上下文和用户态运行库共同配合。

通过 Shell 和阶乘程序，系统具备了基础的人机交互与用户进程生命周期管理框架。当前实现仍可继续改进，例如补齐 `help` 中的学号输出、为 `wait_pid` 增加真正的阻塞与唤醒机制、为每个进程提供独立用户堆、严格校验用户指针，以及完整释放页表和物理页。这些问题也为后续 `fork`、并发和文件系统实验奠定了基础。



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验五：fork、阻塞与并发

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 理解 `fork` 系统调用的语义，掌握父子进程上下文、栈空间、返回值和进程关系的维护方法。
2. 理解阻塞与唤醒机制，使用等待队列避免轮询带来的 CPU 浪费。
3. 理解竞态条件产生的原因，掌握自旋锁和信号量两种同步机制的实现与使用。
4. 在用户程序中使用 `fork` 创建并发执行流，并通过同步原语保护共享资源。
5. 完成多线程计数器、消息队列、哲学家就餐问题和 `fish` 同步输出程序，验证并发控制的正确性。

二. 实验环境

项目	配置
操作系统	YatSenOS / YYOS
开发语言	Rust (<code>no_std</code> 、 <code>no_main</code>)
目标架构	x86_64
引导环境	UEFI + OVMF
虚拟机	QEMU
报告工具	Typst

三. 实验内容概述

本次实验围绕“fork、阻塞与并发”展开。根据实验文档要求，`fork` 需要创建子进程，父进程返回子进程 PID，子进程返回 0；同时本实验中的 `fork` 不复制完整地址空间，而是共享代码段、数据段、堆和 bss 段，只为子进程提供独立的寄存器上下文和栈空间。因此，父子进程可以看到共享全局变量的修改，但各自的局部栈变量互不影响。

实验还要求实现进程阻塞和唤醒。对于等待某个事件的进程，若事件尚未发生，则将其放入等待队列并切换到其他就绪进程；当事件发生时，再把等待进程唤醒并放回就绪队列。信号量在此基础上实现 `new` 、 `remove` 、 `wait` 、 `signal` 四类操作，其中 `wait` 在资源不可用时阻塞当前进程，`signal` 在存在等待者时唤醒一个阻塞进程。

在用户态测试中，我实现了：

- `counter`：使用 `SpinLock` 和 `Semaphore` 分别保护计数器临界区。
- `mq`：使用 `EMPTY`、`FULL`、`MUTEX` 三个信号量实现生产者-消费者消息队列。
- `dinner`：使用筷子信号量与服务生信号量模拟并解决哲学家就餐问题。
- `fish`：创建三个子进程，分别只能输出 `>`、`<`、`_`，通过信号量保证输出总是 `<><_` 与 `><>_` 的组合。

四. 实验原理

1. `fork` 的进程复制模型

本实验中的 `fork` 更接近“共享地址空间的新执行流”。父子进程共享页表中的大部分用户映射，因此全局变量、静态变量和堆对象是共享的；但子进程需要独立栈空间，否则父子进程的局部变量和返回地址会互相覆盖。

实现时需要完成三件事：

1. 保存父进程当前上下文，使父进程之后能从系统调用返回。
2. 构造子进程 PCB，复制父进程上下文，并为子进程分配新的栈空间。
3. 设置不同返回值：父进程的 `rax` 为子进程 PID，子进程的 `rax` 为 0。

在我的实现中，`proc::fork` 先调用 `manager.save_current(context)` 保存父进程现场，再调用 `manager.fork()` 创建子进程，随后将父子进程都加入就绪队列，最后切换到下一个可运行进程：

```
pub fn fork(context: &mut ProcessContext) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        let manager = get_process_manager();
        let parent_pid = manager.current().pid();

        manager.save_current(context);

        let child_pid = manager.fork();

        manager.push_ready(child_pid);
        manager.push_ready(parent_pid);

        manager.switch_next(context);
    })
}
```

`Process::fork` 负责创建新的 PID，调用 `ProcessInner::fork` 复制内部状态，并将子进程加入父进程的 `children` 列表。父进程返回值通过 `inner.context.set_rax(child_pid.0 as usize)` 设置，子进程返回值在 `ProcessInner::fork` 中设置为 0。

2. 子进程栈空间重定位

由于父子进程共享地址空间，如果子进程继续使用父进程栈地址，会导致两个执行流读写同一段栈。因此子进程需要一段新的用户栈。

实现中根据父子 PID 计算 `stack_offset_count`：

```
let stack_offset_count = u16::from(child_pid) as u64 - u16::from(self.pid) as u64;
```

随后在 `ProcessInner::fork` 中调用 `ProcessVm::fork(stack_offset_count)`，复制进程虚拟内存结构并为子进程栈选择偏移位置。之后根据旧栈所在范围调用 `context.relocate_stack` 调整子进程上下文中的栈指针，使其恢复执行时使用新栈：

```
let old_rsp = context.stack_frame.stack_pointer.as_u64();
let old_stack_start = old_rsp & STACK_START_MASK;
let old_stack_end = old_stack_start + STACK_MAX_SIZE;
context.relocate_stack(old_stack_start, old_stack_end, stack_offset);
context.set_rax(0);
```

这样，父子进程共享全局数据，但局部变量位于不同栈空间中，满足实验中对 `fork` 行为的要求。

3. 阻塞、唤醒与等待队列

轮询等待会持续占用 CPU 时间。例如父进程不断查询子进程是否退出，在子进程真正退出前，这些查询都没有实际工作。因此实验引入等待队列，使等待事件的进程主动进入 `Blocked` 状态。

进程状态中增加了 `Blocked`：

```
pub enum ProgramStatus {
    Running,
    Ready,
    Blocked,
    Dead,
}
```

`ProcessManager` 中维护等待队列：

```
wait_queue: Mutex<HashMap<ProcessId, BTreeSet<ProcessId>, ahash::RandomState>>,
```

其含义是：键为被等待进程 PID，值为正在等待该进程退出的一组进程 PID。阻塞时，当前进程状态变为 `Blocked`，并调用 `switch_next` 切换到其他就绪进程；唤醒时，将被唤醒进程状态改回 `Ready`，并放入就绪队列。

信号量也复用了这一思想。`wait` 发现资源数为 0 时，不忙等，而是把当前进程放入信号量的等待队列；`signal` 发现有等待进程时，唤醒其中一个进程。

4. 自旋锁

自旋锁用于用户态中较短的临界区。我的 `SpinLock` 基于 `AtomicBool` 实现：

```
pub fn acquire(&self) {
    loop {
        if self.bolt
            .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)
            .is_ok()
        {
            break;
        }

        while self.bolt.load(Ordering::Relaxed) {
            core::hint::spin_loop();
        }
    }
}

pub fn release(&self) {
    self.bolt.store(false, Ordering::Release);
}
```

`compare_exchange(false, true, ...)` 保证“检查锁是否空闲”和“占有锁”是一个不可分割的原子操作。若获取失败，则使用 `spin_loop` 提示 CPU 当前处于自旋等待。释放时用 `Release` 顺序写回 `false`，保证临界区内的写入在释放锁前完成。

5. 信号量

信号量由资源计数和等待队列组成：

```
pub struct Semaphore {
    count: usize,
    wait_queue: VecDeque<ProcessId>,
}
```

`wait` 的语义为：

- 若 `count > 0`，说明资源可用，减少计数并返回 `Ok`。
- 若 `count == 0`，说明资源不可用，把当前进程加入等待队列并返回 `Block(pid)`。

`signal` 的语义为：

- 若等待队列非空，取出一个等待进程并返回 `WakeUp(pid)`。
- 若没有等待进程，则增加资源计数。

内核系统调用根据 `SemaphoreResult` 操作调度器。`sem_wait` 遇到 `Block(pid)` 时保存当前上下文、阻塞当前进程并切换；`sem_signal` 遇到 `WakeUp(pid)` 时设置其返回值、改为 `Ready` 并加入就绪队列。

用户态库中将信号量封装为：

```
pub struct Semaphore {
    key: u32,
}

impl Semaphore {
    pub const fn new(key: u32) -> Self { Self { key } }
    pub fn init(&self, value: usize) -> bool { sys_new_sem(self.key, value) }
    pub fn remove(&self) -> bool { sys_remove_sem(self.key) }
    pub fn wait(&self) -> bool { sys_sem_wait(self.key) }
    pub fn signal(&self) -> bool { sys_sem_signal(self.key) }
}
```

这样用户程序不需要关心系统调用参数，只需要使用 `init/wait/signal/remove` 即可完成同步。

五. 实验过程

1. 系统调用扩展

在 `crates/syscall/src/lib.rs` 中加入 `Fork` 和 `Sem` 等系统调用号，并在用户态库中封装 `sys_fork`、`sys_sem_wait`、`sys_sem_signal`、`sys_new_sem`、`sys_remove_sem` 等函数。

系统调用分发函数根据 `rax` 中的系统调用号执行对应服务：

```
match args.syscall {
    Syscall::GetPid => context.set_rax(sys_getpid()),
    Syscall::Fork => sys_fork(context),
    Syscall::Exit => sys_exit(&args, context),
    Syscall::WaitPid => context.set_rax(sys_waitpid(&args) as usize),
    Syscall::Sem => sys_sem(&args, context),
    _ => { /* ... */ }
}
```

其中 `Fork` 和 `Sem::wait` 都可能导致上下文切换，因此服务函数需要拿到可变的 `ProcessContext`。

2. 进程管理结构扩展

`Process` 中维护：

- `pid`：进程号。
- `parent`：父进程弱引用。
- `children`：子进程列表。
- `status`：运行状态。
- `context`：寄存器和中断栈帧。
- `exit_code`：退出码。
- `proc_data`：进程共享资源，包括环境变量、文件资源和信号量集合。
- `proc_vm`：进程虚拟内存。

`ProcessManager` 维护进程表、就绪队列、应用列表和等待队列：

```
pub struct ProcessManager {
    processes: RwLock<HashMap<ProcessId, Arc<Process>, ahash::RandomState>>,
    ready_queue: Mutex<VecDeque<ProcessId>>,
    app_list: Option<boot::AppList>,
    wait_queue: Mutex<HashMap<ProcessId, BTreeSet<ProcessId>, ahash::RandomState>>,
}
```

调度器只会从就绪队列中选择状态为 `Ready` 的进程。阻塞进程即使仍然存在于进程表中，也不会被调度执行。

3. 用户态同步库

用户态同步库位于 `crates/lib/src/sync.rs`。自旋锁用于纯用户态忙等待，信号量通过系统调用进入内核，能够真正阻塞当前进程。

为了便于创建一组信号量，实现了 `semaphore_array!` 宏：

```
static CHOPSTICK: [Semaphore; 5] = semaphore_array![0, 1, 2, 3, 4];
```

这在哲学家就餐问题中用于快速声明五根筷子对应的信号量。

六. 用户程序测试

1. 多线程计数器

`counter` 程序创建 8 个并发执行流，每个执行流累加 100 次共享计数器。 `inc_counter` 内部刻意插入 `delay`，使读取、修改、写回三个步骤更容易被调度打断，从而暴露竞态条件。

未加锁时，两个执行流可能同时读取同一个旧值。例如两个进程都读到 `COUNTER = 10`，随后分别写回 `11`，最终丢失一次加法。使用锁后，任意时刻只有一个执行流进入 `inc_counter`，最终结果稳定为 800。

自旋锁版本：

```
fn do_counter_inc_spin() {
    for _ in 0..100 {
        SPIN_LOCK.acquire();
        inc_counter();
        SPIN_LOCK.release();
    }
}
```

信号量版本：

```
fn do_counter_inc_semaphore() {
    for _ in 0..100 {
        SEMAPHORE.wait();
        inc_counter();
        SEMAPHORE.signal();
    }
}
```

运行结果中应观察到：

```
SpinLock counter: 800
Semaphore counter: 800
```

这说明两种同步机制均能正确保护临界区。

2. 消息队列

`mq` 程序实现生产者-消费者模型。父进程创建 16 个子进程，其中 8 个生产者、8 个消费者；每个生产者写入 10 条消息，每个消费者读取 10 条消息。队列容量分别测试 1, 4, 8, 16。

使用三个信号量：

- `MUTEX`：二元信号量，保护队列结构和输出信息。
- `EMPTY`：记录空槽数量，初值为队列容量。
- `FULL`：记录已有消息数量，初值为 0。

生产者逻辑：

```
EMPTY.wait();
MUTEX.wait();
push_message(msg);
MUTEX.signal();
FULL.signal();
```

消费者逻辑：

```
FULL.wait();
MUTEX.wait();
let msg = pop_message();
MUTEX.signal();
EMPTY.signal();
```

当队列满时，`EMPTY == 0`，生产者在 `EMPTY.wait()` 中阻塞；当队列空时，`FULL == 0`，消费者在 `FULL.wait()` 中阻塞。`MUTEX` 保证 `HEAD`、`TAIL`、`COUNT` 的修改是互斥的。

测试观察：

1. 队列容量为 1 时，生产者和消费者交替最明显。
2. 队列容量增大后，可以观察到多个生产者连续写入或多个消费者连续读取。
3. 每次测试结束后 `final queue count = 0`，说明生产和消费总量一致。
4. 输出中 `count = x/capacity` 始终满足 `0 <= x <= capacity`，说明队列没有越界写入或空读。

3. 哲学家就餐

`dinner` 程序创建 5 个并发执行流，每个哲学家循环进行“思考、拿筷子、吃饭、放筷子”。五根筷子分别由五个二元信号量保护：

```
static CHOPSTICK: [Semaphore; 5] = semaphore_array![
    0x2000, 0x2001, 0x2002, 0x2003, 0x2004
];
```

为了模拟不同情况，我设置了三种模式：

- `MODE = 0`：解决死锁版本。

- `MODE = 1` : 故意构造死锁。
- `MODE = 2` : 尝试观察饥饿或机会不均等。

死锁构造方式为：所有哲学家都先拿左筷子，再等待一段时间后尝试拿右筷子。此时可能出现五个哲学家各自持有一根筷子，并同时等待右侧筷子的情形，形成环路等待。

解决方式为引入“服务生”信号量 `ROOM`，初值为 `N - 1`：

```
static ROOM: Semaphore = Semaphore::new(0x2010);
assert!(ROOM.init(N - 1));
```

哲学家在拿筷子之前必须先进入 `ROOM`：

```
ROOM.wait();
// 再尝试拿左右筷子
```

由于最多只有 4 个哲学家同时竞争筷子，不可能出现 5 人同时各持有 1 根筷子的环路等待，因此破坏了死锁的必要条件之一。吃完后释放两根筷子并 `ROOM.signal()`，允许下一个哲学家进入竞争。

此外，程序使用基于 `pid`、哲学家编号和轮次的简单伪随机延迟：

```
fn random_delay(id: usize, round: usize, salt: usize) -> usize {
    let mut x = sys_get_pid() as usize;
    x ^= id * 1103515245;
    x ^= round * 12345;
    x ^= salt * 2654435761;
    x = x.wrapping_mul(1664525).wrapping_add(1013904223);
    x % 10 + 1
}
```

由于本实验环境中未实现 `sys_time`，这里使用 `sys_get_pid` 和循环变量作为种子来源，模拟不同执行流的竞争延迟。

实验现象：

- 在正常模式下，可以观察到不同哲学家输出 `eating round`，说明其同时获得左右筷子。
- 在死锁模式下，常见输出为多个哲学家均输出 `got left chopstick` 后停止，说明所有人都在等待右筷子。
- 在饥饿观察模式下，通过增加某个哲学家的思考延迟，可以观察其吃饭机会明显少于其他哲学家。

4. 加分项：fish 同步输出

`fish` 程序创建三个子进程：

- `gt_worker`：只能输出 `>`。

- `lt_worker` : 只能输出 `<` 。
- `under_worker` : 只能输出 `_` 。

三个子进程分别等待自己的信号量：

```
static SEM_GT: Semaphore = Semaphore::new(0x3000);
static SEM_LT: Semaphore = Semaphore::new(0x3001);
static SEM_UNDER: Semaphore = Semaphore::new(0x3002);
static SEM_DONE: Semaphore = Semaphore::new(0x3003);
```

父进程负责安排输出顺序。每次只唤醒一个子进程，并等待该子进程通过 `SEM_DONE` 告知打印完成：

```
fn print_one(sem: &Semaphore) {
    sem.signal();
    SEM_DONE.wait();
}
```

输出 `<><_` 的函数为：

```
fn print_group_lt_gt_lt() {
    print_one(&SEM_LT);
    print_one(&SEM_GT);
    print_one(&SEM_LT);
    print_one(&SEM_UNDER);
}
```

输出 `><>_` 的函数为：

```
fn print_group_gt_lt_gt() {
    print_one(&SEM_GT);
    print_one(&SEM_LT);
    print_one(&SEM_GT);
    print_one(&SEM_UNDER);
}
```

主循环交替调用这两个函数，因此输出总是 `<><_` 和 `><>_` 的组合。由于父进程在每个字符输出后等待 `SEM_DONE`，下一个字符不会提前输出；由于每个子进程代码中只调用 `put_char` 输出固定字符串，也满足“分别只能输出 `>`、`<` 和 `_`”的要求。

七. 编译与运行结果展示

1. counter 运行结果:

```

yyos> counter
[TRACE] [crates/kernel/src/proc/process.rs:73] New process counter#3 created.
[TRACE] [crates/elf/src/lib.rs:80] Loading ELF file... (user_access=true)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100000000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100001000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100004000) with flags PageTableFlags(PRESENT
| WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100006000) with flags PageTableFlags(PRESENT
| WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:52] Page Range: PageRange { start: Page[4KiB](0x3ffcffff000), end: Page[4KiB]
(0x3ffd00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:89] Init stack frame: InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0x1111000013f0,
    ),
    code_segment: SegmentSelector {
        index: 5,
        rpl: Ring3,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ffcfffff8,
    ),
    stack_segment: SegmentSelector {
        index: 6,
        rpl: Ring3,
    },
}
[TRACE] [crates/kernel/src/proc/manager.rs:291] New Process {
    pid: 3,
    name: "counter",
    parent: Some(
        2,
    ),
    status: Ready,
    ticks_passed: 0,
    children: Map {
        iter: Iter(
            [],
        ),
    },
}

```

```

status: Ready,
context: StackFrame {
  stack_top: VirtAddr(
    0x3ffcffffff8,
  ),
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  instruction_pointer: VirtAddr(
    0x1111000013f0,
  ),
  regs: Registers
  r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
  r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
  r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi: 0x0000000000000000,
  rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
  rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
},
vm: Some(
  ProcessVm {
    stack: Stack {
      top: 0x3ffd00000000,
      bot: 0x3ffcffff000,
    },
    memory_usage: "4 KiB",
    page_table: PageTable {
      addr: PhysFrame[4KiB](0x173000),
      flags: Cr3Flags(
        0x0,
      ),
    },
  },
),
}
[DEBUG] [crates/kernel/src/proc/mod.rs:169] Spawned process: counter#3
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#4 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#5 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4]
Running 'counter' (PID: 3)...
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#6 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#7 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5, 6]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#8 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5, 6, 7]

```



```

[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#19 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 15]
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#12 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#13 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#14 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#15 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#16 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#17 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#18 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#19 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
Semaphore counter: 800
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x1234>
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter#3 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter killed with exit code 0
[TRACE] [crates/kernel/src/proc/paging.rs:30] Releasing page table at PhysFrame[4KiB](0x173000)
Process 3 exited with code 0.
yyos>
yyos>

```

可看到，两种上锁的机制都正确保护了临界区的数据，最后输出结果都是预期的 800。

2. mq 运行结果

```

===== mq test capacity = 4 =====
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#37 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1]
[producer 0 pid 37] push msg 0, count = 1/4
[producer 0 pid 37] push msg 1, count = 2/4

```

```

[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#38 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 37]
[producer 0 pid 37] push msg 2, count = 3/4
[producer 1 pid 38] push msg 1000, count = 4/4
[producer 1 pid 38] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#39 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 37, 38]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#40 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 37]
[producer 0 pid 37] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#41 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 39]
[producer 2 pid 39] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#42 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 40]
[producer 3 pid 40] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#43 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 41]
[producer 4 pid 41] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#44 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 42]
[producer 5 pid 42] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#45 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 43]
[producer 6 pid 43] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#46 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 44]
[producer 7 pid 44] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#47 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 45]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#48 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 46]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#49 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 45]
[consumer 0 pid 45] pop msg 0, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#50 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 46]
[consumer 1 pid 46] pop msg 1, count = 2/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#51 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 48, 37, 46]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#52 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 45]

```

parent created 16 processes

PID | PPID | Process Name | Ticks | Status

```
# 49 | # 20 | mq_forked | 1 | Ready
# 20 | # 2 | mq | 297 | Running
# 1 | # 0 | kernel | 115243 | Ready
# 37 | # 20 | mq_forked | 5 | Blocked
# 2 | # 1 | shell | 115239 | Ready
# 41 | # 20 | mq_forked | 2 | Blocked
# 42 | # 20 | mq_forked | 2 | Blocked
# 46 | # 20 | mq_forked | 4 | Blocked
# 51 | # 20 | mq_forked | 1 | Blocked
# 48 | # 20 | mq_forked | 2 | Blocked
# 43 | # 20 | mq_forked | 2 | Blocked
# 44 | # 20 | mq_forked | 2 | Blocked
# 47 | # 20 | mq_forked | 2 | Blocked
# 50 | # 20 | mq_forked | 1 | Blocked
# 52 | # 20 | mq_forked | 1 | Blocked
# 40 | # 20 | mq_forked | 2 | Blocked
# 45 | # 20 | mq_forked | 4 | Blocked
# 38 | # 20 | mq_forked | 3 | Blocked
# 39 | # 20 | mq_forked | 2 | Blocked
```

Heap : 25.297 KiB used / 31.975 MiB free / 32.000 MiB total

Queue : [2, 1, 49]

CPUs : [0: 20]

```
[consumer 2 pid 47] pop msg 2, count = 1/4
[producer 1 pid 38] push msg 1001, count = 2/4
[consumer 3 pid 48] pop msg 1000, count = 1/4
[producer 0 pid 37] push msg 3, count = 2/4
[producer 2 pid 39] push msg 2000, count = 3/4
[consumer 0 pid 45] pop msg 1001, count = 2/4
[producer 3 pid 40] push msg 3000, count = 3/4
[consumer 4 pid 49] pop msg 3, count = 2/4
[consumer 5 pid 50] pop msg 2000, count = 1/4
[producer 4 pid 41] push msg 4000, count = 2/4
[consumer 1 pid 46] pop msg 3000, count = 1/4
[producer 5 pid 42] push msg 5000, count = 2/4
[producer 6 pid 43] push msg 6000, count = 3/4
[consumer 6 pid 51] pop msg 4000, count = 2/4
[producer 7 pid 44] push msg 7000, count = 3/4
[consumer 7 pid 52] pop msg 5000, count = 2/4
[consumer 2 pid 47] pop msg 6000, count = 1/4
[producer 1 pid 38] push msg 1002, count = 2/4
[consumer 3 pid 48] pop msg 7000, count = 1/4
[producer 0 pid 37] push msg 4, count = 2/4
[producer 2 pid 39] push msg 2001, count = 3/4
[consumer 0 pid 45] pop msg 1002, count = 2/4
```


[producer 3 pid 40] push msg 3001, count = 3/4
 [consumer 5 pid 50] pop msg 4, count = 2/4
 [consumer 4 pid 49] pop msg 2001, count = 1/4
 [producer 4 pid 41] push msg 4001, count = 2/4
 [consumer 1 pid 46] pop msg 3001, count = 1/4
 [producer 5 pid 42] push msg 5001, count = 2/4
 [producer 6 pid 43] push msg 6001, count = 3/4
 [consumer 6 pid 51] pop msg 4001, count = 2/4
 [producer 7 pid 44] push msg 7001, count = 3/4
 [consumer 7 pid 52] pop msg 5001, count = 2/4
 [consumer 2 pid 47] pop msg 6001, count = 1/4
 [producer 1 pid 38] push msg 1003, count = 2/4
 [consumer 3 pid 48] pop msg 7001, count = 1/4
 [producer 0 pid 37] push msg 5, count = 2/4
 [producer 2 pid 39] push msg 2002, count = 3/4
 [consumer 0 pid 45] pop msg 1003, count = 2/4
 [producer 3 pid 40] push msg 3002, count = 3/4
 [consumer 5 pid 50] pop msg 5, count = 2/4
 [consumer 4 pid 49] pop msg 2002, count = 1/4
 [producer 4 pid 41] push msg 4002, count = 2/4
 [consumer 1 pid 46] pop msg 3002, count = 1/4
 [producer 5 pid 42] push msg 5002, count = 2/4
 [producer 6 pid 43] push msg 6002, count = 3/4
 [consumer 6 pid 51] pop msg 4002, count = 2/4
 [producer 7 pid 44] push msg 7002, count = 3/4
 [consumer 7 pid 52] pop msg 5002, count = 2/4
 [consumer 2 pid 47] pop msg 6002, count = 1/4
 [producer 1 pid 38] push msg 1004, count = 2/4
 [consumer 3 pid 48] pop msg 7002, count = 1/4
 [producer 0 pid 37] push msg 6, count = 2/4
 [producer 2 pid 39] push msg 2003, count = 3/4
 [consumer 0 pid 45] pop msg 1004, count = 2/4
 [producer 3 pid 40] push msg 3003, count = 3/4
 [consumer 5 pid 50] pop msg 6, count = 2/4
 [consumer 4 pid 49] pop msg 2003, count = 1/4
 [producer 4 pid 41] push msg 4003, count = 2/4
 [consumer 1 pid 46] pop msg 3003, count = 1/4
 [producer 6 pid 43] push msg 6003, count = 2/4
 [producer 5 pid 42] push msg 5003, count = 3/4
 [consumer 6 pid 51] pop msg 4003, count = 2/4
 [producer 7 pid 44] push msg 7003, count = 3/4
 [consumer 7 pid 52] pop msg 6003, count = 2/4
 [consumer 2 pid 47] pop msg 5003, count = 1/4
 [producer 1 pid 38] push msg 1005, count = 2/4
 [consumer 3 pid 48] pop msg 7003, count = 1/4
 [producer 2 pid 39] push msg 2004, count = 2/4

```

[producer 0 pid 37] push msg 7, count = 3/4
[consumer 0 pid 45] pop msg 1005, count = 2/4
[producer 3 pid 40] push msg 3004, count = 3/4
[consumer 4 pid 49] pop msg 2004, count = 2/4
[consumer 5 pid 50] pop msg 7, count = 1/4
[producer 4 pid 41] push msg 4004, count = 2/4
[consumer 1 pid 46] pop msg 3004, count = 1/4
[producer 6 pid 43] push msg 6004, count = 2/4
[producer 5 pid 42] push msg 5004, count = 3/4
[consumer 6 pid 51] pop msg 4004, count = 2/4
[producer 7 pid 44] push msg 7004, count = 3/4
[consumer 7 pid 52] pop msg 6004, count = 2/4
[consumer 2 pid 47] pop msg 5004, count = 1/4
[consumer 3 pid 48] pop msg 7004, count = 0/4
[producer 1 pid 38] push msg 1006, count = 1/4
[producer 2 pid 39] push msg 2005, count = 2/4
[producer 0 pid 37] push msg 8, count = 3/4
[producer 3 pid 40] push msg 3005, count = 4/4
[consumer 0 pid 45] pop msg 1006, count = 3/4
[consumer 4 pid 49] pop msg 2005, count = 2/4
[consumer 5 pid 50] pop msg 8, count = 1/4
[consumer 1 pid 46] pop msg 3005, count = 0/4
[producer 4 pid 41] push msg 4005, count = 1/4
[producer 6 pid 43] push msg 6005, count = 2/4
[producer 5 pid 42] push msg 5005, count = 3/4
[producer 7 pid 44] push msg 7005, count = 4/4
[consumer 6 pid 51] pop msg 4005, count = 3/4
[consumer 7 pid 52] pop msg 6005, count = 2/4
[consumer 2 pid 47] pop msg 5005, count = 1/4
[consumer 3 pid 48] pop msg 7005, count = 0/4
[producer 1 pid 38] push msg 1007, count = 1/4
[producer 2 pid 39] push msg 2006, count = 2/4
[producer 0 pid 37] push msg 9, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#37 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 0 pid 45] pop msg 1007, count = 2/4
[producer 3 pid 40] push msg 3006, count = 3/4
[consumer 4 pid 49] pop msg 2006, count = 2/4
[consumer 5 pid 50] pop msg 9, count = 1/4
[producer 4 pid 41] push msg 4006, count = 2/4
[consumer 1 pid 46] pop msg 3006, count = 1/4
[producer 6 pid 43] push msg 6006, count = 2/4
[producer 5 pid 42] push msg 5006, count = 3/4
[consumer 6 pid 51] pop msg 4006, count = 2/4
[producer 7 pid 44] push msg 7006, count = 3/4

```

```

[consumer 7 pid 52] pop msg 6006, count = 2/4
[consumer 2 pid 47] pop msg 5006, count = 1/4
[producer 1 pid 38] push msg 1008, count = 2/4
[consumer 3 pid 48] pop msg 7006, count = 1/4
[producer 2 pid 39] push msg 2007, count = 2/4
[producer 3 pid 40] push msg 3007, count = 3/4
[consumer 0 pid 45] pop msg 1008, count = 2/4
[producer 4 pid 41] push msg 4007, count = 3/4
[consumer 4 pid 49] pop msg 2007, count = 2/4
[consumer 5 pid 50] pop msg 3007, count = 1/4
[producer 6 pid 43] push msg 6007, count = 2/4
[consumer 1 pid 46] pop msg 4007, count = 1/4
[producer 5 pid 42] push msg 5007, count = 2/4
[producer 7 pid 44] push msg 7007, count = 3/4
[consumer 6 pid 51] pop msg 6007, count = 2/4
[producer 1 pid 38] push msg 1009, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#38 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 7 pid 52] pop msg 5007, count = 2/4
[consumer 2 pid 47] pop msg 7007, count = 1/4
[producer 2 pid 39] push msg 2008, count = 2/4
[consumer 3 pid 48] pop msg 1009, count = 1/4
[producer 3 pid 40] push msg 3008, count = 2/4
[producer 4 pid 41] push msg 4008, count = 3/4
[consumer 0 pid 45] pop msg 2008, count = 2/4
[producer 6 pid 43] push msg 6008, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#45 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 4 pid 49] pop msg 3008, count = 2/4
[consumer 5 pid 50] pop msg 4008, count = 1/4
[producer 5 pid 42] push msg 5008, count = 2/4
[consumer 1 pid 46] pop msg 6008, count = 1/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#46 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[producer 7 pid 44] push msg 7008, count = 2/4
[producer 2 pid 39] push msg 2009, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#39 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 6 pid 51] pop msg 5008, count = 2/4
[producer 3 pid 40] push msg 3009, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#40 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0

```

```

[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 2 pid 47] pop msg 7008, count = 2/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#47 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 7 pid 52] pop msg 2009, count = 1/4
[producer 4 pid 41] push msg 4009, count = 2/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#41 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 3 pid 48] pop msg 3009, count = 1/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#48 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[producer 6 pid 43] push msg 6009, count = 2/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#43 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[producer 5 pid 42] push msg 5009, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#42 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[producer 7 pid 44] push msg 7009, count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#44 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 4 pid 49] pop msg 4009, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#49 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 5 pid 50] pop msg 6009, count = 2/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#50 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 6 pid 51] pop msg 5009, count = 1/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#51 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 7 pid 52] pop msg 7009, count = 0/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#52 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
mq capacity 4 finished, final queue count = 0
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x1001>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x1002>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x1003>

```

```
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x1001>1
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x1002>8
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x1003>0
```

3. dinner 运行结果

```
yyos> dinner
[TRACE] [crates/kernel/src/proc/process.rs:73] New process dinner#3 created.
[TRACE] [crates/elf/src/lib.rs:80] Loading ELF file... (user_access=true)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100000000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100001000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100004000) with flags PageTableFlags(PRESENT
| WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:52] Page Range: PageRange { start: Page[4KiB](0x3ffcffff000), end: Page[4KiB]
(0x3ffd00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:89] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0x111100001880,
  ),
  code_segment: SegmentSelector {
    index: 5,
    rpl: Ring3,
  },
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  stack_pointer: VirtAddr(
    0x3ffcfffff8,
  ),
  stack_segment: SegmentSelector {
    index: 6,
    rpl: Ring3,
  },
}
[TRACE] [crates/kernel/src/proc/manager.rs:291] New Process {
  pid: 3,
  name: "dinner",
  parent: Some(
    2,
  ),
}
```

```

status: Ready,
ticks_passed: 0,
children: Map {
  iter: Iter(
    [],
  ),
},
status: Ready,
context: StackFrame {
  stack_top: VirtAddr(
    0x3ffcffff8,
  ),
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  instruction_pointer: VirtAddr(
    0x111100001880,
  ),
  regs: Registers
  r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
  r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
  r9: 0x0000000000000000, r8: 0x0000000000000000, rdi: 0x0000000000000000,
  rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
  rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
},
vm: Some(
  ProcessVm {
    stack: Stack {
      top: 0x3ffd00000000,
      bot: 0x3ffcffff000,
    },
    memory_usage: "4 KiB",
    page_table: PageTable {
      addr: PhysFrame[4KiB](0x173000),
      flags: Cr3Flags(
        0x0,
      ),
    },
  },
),
}
[DEBUG] [crates/kernel/src/proc/mod.rs:169] Spawned process: dinner#3
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2000>1
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2001>1
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2002>1
Running 'dinner' (PID: 3)...

```

```

[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2003>1
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2004>1
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2010>4
dining philosophers start, mode = 0
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child dinner_forked#4 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child dinner_forked#5 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child dinner_forked#6 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child dinner_forked#7 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5, 6]
[P0 pid 4] thinking round 0
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child dinner_forked#8 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5, 6, 7]
[P0] got left chopstick 0
[P1 pid 5] thinking round 0
[P1] got right chopstick 2
[P2 pid 6] thinking round 0
[P3 pid 7] thinking round 0
[P3] got right chopstick 4
[P4 pid 8] thinking round 0
[P0] got right chopstick 1
[P0 pid 4] eating round 0, delay = 9
[P3] got left chopstick 3
[P3 pid 7] eating round 0, delay = 9
[P0] put chopsticks 0 and 1
[P1] got left chopstick 1
[P1 pid 5] eating round 0, delay = 9
[P3] put chopsticks 3 and 4
[P4] got left chopstick 4
[P4] got right chopstick 0
[P4 pid 8] eating round 0, delay = 9
[P0 pid 4] thinking round 1
[P1] put chopsticks 1 and 2
[P3 pid 7] thinking round 1
[P2] got left chopstick 2
[P2] got right chopstick 3
[P2 pid 6] eating round 0, delay = 9
[P4] put chopsticks 4 and 0
[P3] got right chopstick 4
[P0] got left chopstick 0
[P1 pid 5] thinking round 1
[P4 pid 8] thinking round 1
[P0] got right chopstick 1
[P0 pid 4] eating round 1, delay = 4

```

[P2] put chopsticks 2 and 3
[P3] got left chopstick 3
[P3 pid 7] eating round 1, delay = 4
[P1] got right chopstick 2
[P0] put chopsticks 0 and 1
[P3] put chopsticks 3 and 4
[P4] got left chopstick 4
[P0 pid 4] thinking round 2
[P3 pid 7] thinking round 2
[P0] got left chopstick 0
[P2 pid 6] thinking round 1
[P1] got left chopstick 1
[P1 pid 5] eating round 1, delay = 4
[P1] put chopsticks 1 and 2
[P0] got right chopstick 1
[P0 pid 4] eating round 2, delay = 9
[P2] got left chopstick 2
[P1 pid 5] thinking round 2
[P2] got right chopstick 3
[P2 pid 6] eating round 1, delay = 4
[P0] put chopsticks 0 and 1
[P4] got right chopstick 0
[P4 pid 8] eating round 1, delay = 4
[P2] put chopsticks 2 and 3
[P1] got right chopstick 2
[P4] put chopsticks 4 and 0
[P3] got right chopstick 4
[P1] got left chopstick 1
[P1 pid 5] eating round 2, delay = 9
[P2 pid 6] thinking round 2
[P4 pid 8] thinking round 2
[P0 pid 4] thinking round 3
[P3] got left chopstick 3
[P3 pid 7] eating round 2, delay = 9
[P1] put chopsticks 1 and 2
[P2] got left chopstick 2
[P0] got left chopstick 0
[P3] put chopsticks 3 and 4
[P4] got left chopstick 4
[P2] got right chopstick 3
[P2 pid 6] eating round 2, delay = 9
[P1 pid 5] thinking round 3
[P0] got right chopstick 1
[P0 pid 4] eating round 3, delay = 4
[P3 pid 7] thinking round 3
[P0] put chopsticks 0 and 1

[P2] put chopsticks 2 and 3
[P4] got right chopstick 0
[P4 pid 8] eating round 2, delay = 9
[P1] got right chopstick 2
[P0 pid 4] thinking round 4
[P2 pid 6] thinking round 3
[P3] got right chopstick 4
[P0] got left chopstick 0
[P4] put chopsticks 4 and 0
[P1] got left chopstick 1
[P1 pid 5] eating round 3, delay = 4
[P1] put chopsticks 1 and 2
[P0] got right chopstick 1
[P0 pid 4] eating round 4, delay = 9
[P2] got left chopstick 2
[P3] got left chopstick 3
[P3 pid 7] eating round 3, delay = 4
[P4 pid 8] thinking round 3
[P1 pid 5] thinking round 4
[P3] put chopsticks 3 and 4
[P4] got left chopstick 4
[P2] got right chopstick 3
[P2 pid 6] eating round 3, delay = 4
[P0] put chopsticks 0 and 1
[P3 pid 7] thinking round 4
[P2] put chopsticks 2 and 3
[P1] got right chopstick 2
[P4] got right chopstick 0
[P4 pid 8] eating round 3, delay = 4
[P2 pid 6] thinking round 4
[P1] got left chopstick 1
[P1 pid 5] eating round 4, delay = 9
[P0 pid 4] thinking round 5
[P4] put chopsticks 4 and 0
[P3] got right chopstick 4
[P0] got left chopstick 0
[P4 pid 8] thinking round 4
[P3] got left chopstick 3
[P3 pid 7] eating round 4, delay = 9
[P1] put chopsticks 1 and 2
[P2] got left chopstick 2
[P0] got right chopstick 1
[P0 pid 4] eating round 5, delay = 4
[P3] put chopsticks 3 and 4
[P0] put chopsticks 0 and 1
[P2] got right chopstick 3

```

[P2 pid 6] eating round 4, delay = 9
[P4] got left chopstick 4
[P0 pid 4] thinking round 6
[P0] got left chopstick 0
[P1 pid 5] thinking round 5
[P0] got right chopstick 1
[P0 pid 4] eating round 6, delay = 9
[P2] put chopsticks 2 and 3
[P3 pid 7] thinking round 5
[P1] got right chopstick 2
[P0] put chopsticks 0 and 1
[P4] got right chopstick 0
[P4 pid 8] eating round 4, delay = 9
[P2 pid 6] thinking round 5
[P1] got left chopstick 1
[P1 pid 5] eating round 5, delay = 4
[P0 pid 4] thinking round 7
[P1] put chopsticks 1 and 2
[P2] got left chopstick 2
[P4] put chopsticks 4 and 0
[P3] got right chopstick 4
[P0] got left chopstick 0
[P1 pid 5] thinking round 6
[P4 pid 8] thinking round 5
[P2] got right chopstick 3
[P2 pid 6] eating round 5, delay = 4
[P0] got right chopstick 1
[P0 pid 4] eating round 7, delay = 4
[P2] put chopsticks 2 and 3
[P1] got right chopstick 2
[P3] got left chopstick 3
[P3 pid 7] eating round 5, delay = 4
[P0] put chopsticks 0 and 1
[P0 pid 4] done
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner_forked#4 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner_forked killed with exit code 0
[P2 pid 6] thinking round 6
[P1] got left chopstick 1
[P1 pid 5] eating round 6, delay = 9
[P3] put chopsticks 3 and 4
[P4] got left chopstick 4
[P3 pid 7] thinking round 6
[P1] put chopsticks 1 and 2
[P4] got right chopstick 0
[P4 pid 8] eating round 5, delay = 4

```

```

[P2] got left chopstick 2
[P4] put chopsticks 4 and 0
[P2] got right chopstick 3
[P2 pid 6] eating round 6, delay = 9
[P3] got right chopstick 4
[P4 pid 8] thinking round 6
[P1 pid 5] thinking round 7
[P2] put chopsticks 2 and 3
[P1] got right chopstick 2
[P3] got left chopstick 3
[P3 pid 7] eating round 6, delay = 9
[P2 pid 6] thinking round 7
[P3] put chopsticks 3 and 4
[P1] got left chopstick 1
[P1 pid 5] eating round 7, delay = 4
[P4] got left chopstick 4
[P4] got right chopstick 0
[P4 pid 8] eating round 6, delay = 9
[P1] put chopsticks 1 and 2
[P1 pid 5] done
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner_forked#5 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner_forked killed with exit code 0
[P2] got left chopstick 2
[P3 pid 7] thinking round 7
[P2] got right chopstick 3
[P2 pid 6] eating round 7, delay = 4
[P4] put chopsticks 4 and 0
[P3] got right chopstick 4
[P2] put chopsticks 2 and 3
[P2 pid 6] done
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner_forked#6 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner_forked killed with exit code 0
[P4 pid 8] thinking round 7
[P3] got left chopstick 3
[P3 pid 7] eating round 7, delay = 4
[P3] put chopsticks 3 and 4
[P3 pid 7] done
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner_forked#7 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner_forked killed with exit code 0
[P4] got left chopstick 4
[P4] got right chopstick 0
[P4 pid 8] eating round 7, delay = 4
[P4] put chopsticks 4 and 0

```

```

[P4 pid 8] done
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner_forked#8 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2000>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2001>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2002>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2003>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2004>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2010>
dining philosophers finished
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner#3 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner killed with exit code 0
[TRACE] [crates/kernel/src/proc/paging.rs:30] Releasing page table at PhysFrame[4KiB](0x173000)
Process 3 exited with code 0.
yyos>

```

这个问题为了避免死锁，最多只允许四个哲学家同时进食，不难看出程序正常运行。

4. fish 运行结果

```

yyos> fish
[TRACE] [crates/kernel/src/proc/process.rs:73] New process fish#9 created.
[TRACE] [crates/elf/src/lib.rs:80] Loading ELF file... (user_access=true)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100000000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100001000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100004000) with flags PageTableFlags(PRESENT
| WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:52] Page Range: PageRange { start: Page[4KiB](0x3ff6ffff000), end: Page[4KiB]
(0x3ff700000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:89] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0x111100001730,
  ),
  code_segment: SegmentSelector {
    index: 5,
    rpl: Ring3,
  },
}

```

```

cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
),
stack_pointer: VirtAddr(
    0x3ff6ffffff8,
),
stack_segment: SegmentSelector {
    index: 6,
    rpl: Ring3,
},
}
[TRACE] [crates/kernel/src/proc/manager.rs:291] New Process {
    pid: 9,
    name: "fish",
    parent: Some(
        2,
    ),
    status: Ready,
    ticks_passed: 0,
    children: Map {
        iter: Iter(
            [],
        ),
    },
    status: Ready,
    context: StackFrame {
        stack_top: VirtAddr(
            0x3ff6ffffff8,
        ),
        cpu_flags: RFlags(
            IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
        ),
        instruction_pointer: VirtAddr(
            0x111100001730,
        ),
        regs: Registers
        r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
        r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
        r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi: 0x0000000000000000,
        rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
        rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
    },
}

```



```
[TRACE] [crates/kernel/src/proc/process.rs:211] Process fish killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process fish killed with exit code 0
[TRACE] [crates/kernel/src/proc/paging.rs:30] Releasing page table at PhysFrame[4KiB](0x173000)
Process 9 exited with code 0.
yyos>
```

八. 思考题

1. 1. 输入缓冲区使用 **Mutex** 保护时需要注意什么？

如果在串口输入中断和用户态读取之间使用同一个 **Mutex** 保护输入队列，需要避免在持锁期间发生可能再次获取同一把锁的中断。典型问题是：内核正在执行 **pop**，已经获得输入队列锁，但尚未释放；此时串口中断到来，中断处理程序尝试 **push** 新字符，也要获取同一把锁。由于当前 CPU 已经持有该锁，中断处理程序会自旋等待，而原来的 **pop** 又无法继续执行并释放锁，从而造成死锁。

解决方法是：在访问该缓冲区的临界区内关闭中断，保证持锁期间不会被同类中断打断；或者使用无锁队列，使中断上下文中的 **push** 不依赖可能被普通上下文持有的睡眠锁或自旋锁。若使用锁，还应保证临界区尽可能短，不在持锁期间执行阻塞或耗时操作。

2. 2. **fork** 复制内存时各页表之间是什么关系？

系统当前页表是 CPU 正在使用的页表，通常对应当前运行进程。进程页表描述该进程的虚拟地址空间。子进程页表由父进程页表派生而来，本实验中共享主要用户映射和内核映射，但为子进程单独安排栈空间。内核页表提供内核代码、数据和物理内存映射，用户进程页表通常由内核页表的高地址部分克隆或共享，使中断和系统调用进入内核后仍能访问内核空间。

复制内存时要注意：

1. 不能在正在使用的页表和目标页表之间造成地址别名冲突。
2. 子进程栈必须映射到独立区域，并复制父进程当前栈内容。
3. 修改子进程上下文中的 **rsp**，否则子进程仍会使用父进程栈地址。
4. 父子进程返回值不同，需要分别设置各自上下文中的 **rax**。
5. 若共享页表或物理页，需要使用引用计数管理生命周期，避免一个进程退出时释放仍被另一个进程使用的页表资源。

3.3. 为什么 fork 必须在堆分配前进行？

本实验的 `fork` 不实现完整地址空间复制，也没有实现写时复制。父子进程会共享堆和全局数据。如果在堆分配之后 `fork`，父子进程会共享同一个用户态堆分配器状态。之后任一进程执行 `alloc` 或 `dealloc` 都可能修改同一套堆元数据，导致重复分配、重复释放、链表损坏或悬垂指针等问题。

此外，Rust 的很多抽象会隐式使用堆，例如 `Vec`、`String`、格式化输出中的临时分配等。如果在这些分配之后 `fork`，父子进程共享的堆状态可能与各自执行路径不一致，出现难以定位的内存错误。因此实验要求在任何 Rust 堆分配前调用 `fork`。

4.4. Ordering 参数的含义

`Ordering` 用于描述原子操作与其他内存访问之间的顺序约束。常见取值如下：

- `Relaxed`：只保证该原子变量本身操作的原子性，不建立额外的跨线程顺序关系。
- `Acquire`：常用于加锁或读取同步标志，保证该操作之后的读写不会被重排到它之前。
- `Release`：常用于解锁或发布同步标志，保证该操作之前的读写不会被重排到它之后。
- `AcqRel`：同时具有 `Acquire` 和 `Release` 语义，常用于读-改-写操作。
- `SeqCst`：顺序一致性，是最强的内存顺序，所有线程看到的 `SeqCst` 原子操作顺序一致。

在本实验的自旋锁中，获取锁使用 `Acquire`，释放锁使用 `Release`。这足以保证进入临界区后能看到前一个持锁者释放前完成的写入。

5.5. 为什么 SpinLock 需要实现 Sync？Send 又是什么？

`Sync` 表示一个类型的共享引用 `&T` 可以安全地在线程之间共享。静态锁变量会被多个并发执行流通过共享引用访问，因此 `SpinLock` 需要实现 `Sync`。虽然 `SpinLock` 内部有可变状态，但该状态由 `AtomicBool` 以原子方式访问，满足并发安全要求，所以可以手动 `unsafe impl Sync for SpinLock`。

`Send` 表示一个类型的所有权可以安全地从一个线程转移到另一个线程。简单地说，`Send` 关注“值能不能被移动到别的线程”，`Sync` 关注“共享引用能不能被多个线程同时访问”。对于锁类型，通常更关心 `Sync`，因为多个线程共享的是同一个锁对象。

6.6. spin_loop 的 pause 和 hlt 有什么区别？为什么不能用 hlt？

`core::hint::spin_loop` 在 x86_64 上通常会生成 `pause` 指令。`pause` 是给处理器的提示，表示当前处于短暂自旋等待，可以降低流水线和超线程资源竞争，但 CPU 仍然继续执行当前线程。

`hlt` 则会让 CPU 进入停止状态，直到下一次外部中断到来。它通常用于内核空闲循环，而不是用户态锁等待。

自旋锁不能使用 `hlt`，原因是锁的释放不一定伴随硬件中断。如果等待者执行 `hlt` 后没有中断到来，它可能无法及时继续检查锁状态。用户态程序也不应该直接执行这类特权/低层电源管理语义的指令。因此自旋等待使用 `pause` 更合适。

九. 实验总结

本次实验将前几次实验中的进程、调度、系统调用和用户态程序串联起来。`fork` 的实现让我理解到，进程复制不只是创建一个 PCB，还必须正确处理上下文、返回值、父子关系和栈空间。阻塞与唤醒机制则体现了调度器的核心价值：等待资源时不应浪费 CPU，而应让出处理器给其他就绪进程。

在并发测试中，计数器展示了竞态条件的直观后果；消息队列展示了信号量对资源数量和临界区的组合控制；哲学家就餐问题展示了死锁、饥饿以及通过破坏环路等待解决死锁的思路；

`fish` 程序则验证了信号量不仅能保护资源，也能精确控制多个进程的执行顺序。

总体而言，本实验使我对“并发不是同时写代码，而是精确设计等待关系”有了更具体的认识。同步原语本身并不复杂，真正需要谨慎的是它们与调度、进程状态和共享资源生命周期之间的配合。



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验六：硬盘驱动与文件系统

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 理解块设备、分区表和文件系统之间的分层关系。
2. 掌握 MBR 分区表的格式，能够解析分区状态、类型、CHS 地址、起始 LBA 和扇区数量。
3. 理解 ATA PIO 工作方式，实现 28 位 LBA 下的磁盘识别与扇区读写。
4. 理解 FAT16 的 BPB、FAT 表、目录项、簇链和文件数据区，实现只读文件系统。
5. 将磁盘和文件系统接入内核，通过系统调用为 Shell 提供 `ls`、`open`、`read`、`close` 和 `cat` 功能。
6. 通过 `procfs`、`devfs`、`tmpfs` 等特殊文件系统理解 Linux“一切皆文件”的设计。

二. 实验环境

项目	配置
操作系统	YatSenOS / YYOS
开发语言	Rust (<code>no_std</code> 、 <code>no_main</code>)
目标架构	x86_64
引导环境	UEFI + OVMF
虚拟机	QEMU
磁盘接口	PATA，PIO 模式，28 位 LBA
文件系统	MBR + FAT16，只读实现
报告工具	Typst

三. 实验内容概述

本次实验从最底层的磁盘端口读写开始，逐层构造出可以被用户程序使用的文件访问接口。
整体调用关系如下：

```

Shell: ls / cat
|
用户态系统调用封装
|
内核 syscall dispatcher
|
Mount / FileSystem / FileHandle
|
FAT16: BPB、目录项、FAT 簇链
|
Partition: 分区内 LBA -> 磁盘 LBA
|
MBR 分区表
|
AtaDrive / AtaBus
|
ATA PIO 端口

```

实验中的存储 crate 定义了 `BlockDevice`、`PartitionTable`、`FileSystem`、`Read`、`FileHandle` 和 `Mount` 等抽象。内核中的 ATA 驱动只负责可靠地读写扇区，分区层负责地址平移，FAT16 层负责解释磁盘数据，系统调用层负责把这些能力安全地提供给用户程序。

四. 实验原理

1. MBR 分区表

MBR 位于磁盘第 0 个扇区，总长度为 512 字节。偏移 `0x1BE` 处开始存放 4 个主分区表项，每项 16 字节。一个表项包含：

偏移	长度	含义
<code>0x00</code>	1	状态， <code>0x80</code> 表示可启动
<code>0x01</code>	3	起始 CHS 地址
<code>0x04</code>	1	分区类型
<code>0x05</code>	3	结束 CHS 地址
<code>0x08</code>	4	分区起始 LBA，小端序
<code>0x0C</code>	4	分区扇区数量，小端序

CHS 地址中的扇区号占 6 位，柱面号占 10 位，因此不能直接按字节读取。以起始地址为例：

```
pub fn begin_sector(&self) -> u8 {
    self.data[0x02] & 0x3f
}

pub fn begin_cylinder(&self) -> u16 {
    ((self.data[0x02] as u16 & 0xc0) << 2)
    | self.data[0x03] as u16
}
```

后续实际磁盘访问采用 LBA，不依赖 CHS。分区是否存在也不能只通过 active 位判断，因为普通数据分区可以不是可启动分区。因此实现中使用“分区类型非零且扇区数量非零”判断有效表项。

2. 分区块设备

`Partition<T, B>` 本身也实现 `BlockDevice<B>`。对分区内第 `offset` 块的访问，需要转换为底层磁盘的：

$$\text{inner_offset} = \text{partition_start} + \text{offset}$$

同时必须检查 `offset < size`，并使用 `checked_add` 防止整数溢出。这样 FAT16 层只看到从 0 开始的逻辑分区，无须关心它位于整块磁盘的什么位置。

3. ATA PIO 与 28 位 LBA

ATA PIO 通过 I/O 端口与设备交互。主总线的常见基地址为 `0x1F0`，数据寄存器宽度为 16 位。发送一个 28 位 LBA 命令时：

1. 将扇区数量写入 `sector_count`。
2. 将 LBA 的低 24 位分别写入 `lba_low`、`lba_mid`、`lba_high`。
3. 将 LBA 的第 24 至 27 位写入 `drive` 寄存器低四位。
4. 在 `drive` 寄存器中设置 LBA 模式和主从盘选择。
5. 写入命令寄存器。
6. 轮询等待 `BUSY` 清零，并等待 `DATA_REQUEST_READY` 置位。

实现中的 `drive` 寄存器值为：

```
0xe0 | ((drive & 1) << 4) | (bytes[3] & 0x0f)
```

其中 `0xE0` 设置了固定高位和 LBA 模式，`drive` 选择主盘或从盘，最低四位保存 LBA 高四位。

ATA 每次通过 data 端口传输 16 位数据，因此读取一个 512 字节扇区需要读取 256 次 `u16`，并按小端序拆分为字节；写入过程与之相反。

4. FAT16 文件系统

FAT16 分区主要由保留区、FAT 表、根目录区和数据区构成：

```
[ BPB / Reserved ][ FAT 1 ][ FAT 2 ][ Root Directory ][ Data Clusters ]
```

BPB 位于分区第一个扇区，描述每扇区字节数、每簇扇区数、保留扇区数、FAT 数量、根目录项数量和每个 FAT 的扇区数等信息。

根目录占用扇区数为：

$$\text{root_dir_sectors} = \left\lceil \frac{\text{root_entries} \times 32}{\text{bytes_per_sector}} \right\rceil$$

第一个根目录扇区与第一个数据扇区分别为：

$$\text{root_start} = \text{reserved} + \text{fat_count} \times \text{sectors_per_fat}$$

$$\text{data_start} = \text{root_start} + \text{root_dir_sectors}$$

普通簇号从 2 开始，因此簇到扇区的转换公式为：

$$\text{sector} = (\text{cluster} - 2) \times \text{sectors_per_cluster} + \text{data_start}$$

FAT16 的每个 FAT 表项占 2 字节。通过当前簇号计算 FAT 表内字节偏移，可以读取下一个簇号。

`0xFFFF8..=0xFFFF` 表示簇链结束，`0xFFFF7` 表示坏簇，`0x0000` 表示空闲簇。

5. FAT16 目录项与文件读取

短文件名目录项大小为 32 字节，包含 8.3 文件名、属性、时间、首簇号和文件大小。本实验忽略长文件名项和卷标项，只解析标准短文件名。

读取目录时按目录占用的扇区遍历目录项：

- 首字节为 `0x00`：目录结束。
- 首字节为 `0xE5`：目录项已删除。
- 属性为 `LFN`：长文件名项，本实验跳过。
- 属性含 `VOLUME_ID`：卷标项，不作为普通文件返回。

读取文件时，需要同时考虑文件偏移、簇大小、扇区大小和用户缓冲区长度。每次复制的数据量为：

$$\min(\text{sector_remaining}, \text{request_remaining}, \text{file_remaining})$$

读完一个簇后查询 FAT 表进入下一个簇，直到缓冲区填满、文件结束或遇到簇链结束标记。

五. 实验过程与实现

1. 合并 storage 与 ATA 模块

我将实验六提供的增量代码合并到实验五工程中，并在 kernel 的 `Cargo.toml` 中加入 storage crate：

```
storage = { package = "yyos_storage", path = "../storage" }
```

同时在驱动模块中注册：

```
pub mod ata;  
pub mod filesystem;
```

2. 实现 MBR 分区表

`MbrTable::parse` 首先读取磁盘第 0 块，从 `0x1BE` 起每 16 字节解析一个分区项：

```
const PARTITION_TABLE_OFFSET: usize = 0x1be;  
const PARTITION_ENTRY_SIZE: usize = 16;  
  
for i in 0..4 {  
    let offset = PARTITION_TABLE_OFFSET + i * PARTITION_ENTRY_SIZE;  
    let entry = (&buffer[offset..offset + PARTITION_ENTRY_SIZE])  
        .try_into()  
        .map_err(|_| FsError::InvalidOperation)?;  
    partitions.push(MbrPartition::parse(entry));  
}
```

在 `MbrPartition` 中利用 `define_field!` 定义状态、磁头、分区类型、起始 LBA 和扇区数，并手工解析 CHS 中没有按字节对齐的扇区与柱面字段。

测试样例的解析结果为：

```

active      = true
begin head  = 1
begin sector = 1
begin cylinder = 0
partition type = 0x0b
end head    = 254
end sector  = 63
end cylinder = 764
begin LBA   = 63
total LBA   = 12289662

```

3. 实现 Partition 地址平移

分区块设备对越界访问返回 `FsError::InvalidOffset`，合法访问转换为底层磁盘 LBA：

```

let inner_offset = self
    .offset
    .checked_add(offset)
    .ok_or(FsError::InvalidOffset)?;
self.inner.read_block(inner_offset, block)

```

写操作采用相同的地址转换。虽然 FAT16 在本实验中只读，但完整实现分区写转发有助于保持 `BlockDevice` 抽象一致。

4. 实现 ATA 命令发送

`write_command` 将 28 位 LBA 写入寄存器，并轮询设备状态：

```

self.sector_count.write(1);
self.lba_low.write(bytes[0]);
self.lba_mid.write(bytes[1]);
self.lba_high.write(bytes[2]);
self.drive
    .write(0xe0 | ((drive & 1) << 4) | (bytes[3] & 0x0f));
self.command.write(cmd as u8);

```

发送命令后先判断空状态以识别不存在的设备，再等待 `BUSY` 清零，检查错误位，最后等待 DRQ 数据请求位。

5. 识别 ATA 磁盘

`identify_drive` 发送 `IdentifyDevice` 命令并读取 256 个 `u16`。根据 cylinder low/high 的签名区分 PATA、PATAPI、SATA 和 SATAPI。本实验只支持 PATA。

识别数据中的序列号、型号和最大 LBA 分别位于字节偏移 20、54 和 120。`AtaDrive` 只保存总线号、主从盘号、块数量、型号和序列号，实际端口由全局 `BUSES` 管理。

这种设计使 `AtaDrive` 可以轻量地实现 `Clone`，多个分区或文件系统对象可以持有同一个磁盘的逻辑句柄，而真实总线访问仍由 `Mutex<AtaBus>` 串行化。

6. 实现 PIO 扇区读写

读取数据时，每次从 data 端口读取一个 `u16`：

```
for chunk in buf.chunks_mut(2) {
    let bytes = self.read_data().to_le_bytes();
    chunk.copy_from_slice(&bytes[..chunk.len()]);
}
```

写入时将每两个字节组合成小端 `u16`。实现过程中曾出现 `bytes` 变量位于循环内部、`write_data` 位于循环外部的的问题，这会导致无法编译，也无法写出完整扇区。修正后每个 chunk 都在循环内写入 data 端口。

7. 实现 FAT16 BPB

使用 `define_field!` 完成 BPB 关键字段，包括：

- OEM 名称；
- 每扇区字节数、每簇扇区数；
- 保留扇区数、FAT 数量；
- 根目录项数量；
- 16 位与 32 位总扇区数；
- 每个 FAT 的扇区数；
- 隐藏扇区、卷标、系统标识；
- 末尾 `0xAA55` 签名。

`Fat16Bpb::new` 检查输入长度和尾部签名，避免把无效扇区解释为文件系统元数据。

8. 实现 DirEntry 与短文件名

`DirEntry::parse` 从 32 字节目录项中解析文件名、属性、创建/访问/修改时间、首簇和文件大小。`ShortFileName::parse` 将普通文件名转换为大写 8.3 格式，并检查空文件名、多个点、名称过长和非法字符。

例如 `hello.txt` 转换后：

```
basename = "HELLO "
extension = "TXT"
```

9. 实现 FAT 簇链和目录遍历

`next_cluster` 根据簇号定位 FAT 表项：

```
let offset = cluster.0 as usize * 2;
let sector = self.fat_start + offset / BLOCK_SIZE;
let index = offset % BLOCK_SIZE;
```

目录遍历会区分固定大小的 FAT16 根目录和由簇链组成的普通目录。路径查找使用 `/` 分割路径，逐级在当前目录中查找短文件名；若中间分量不是目录，则返回 `NotADirectory`。

10. 实现文件读取

`File` 保存当前偏移、当前簇、目录项和共享的 `Fat16Handle`。读取函数首先限制读取长度不超过文件剩余长度，然后根据当前偏移确定簇内扇区和扇区内偏移：

```
let cluster_offset = self.offset % cluster_size;
let sector_index = cluster_offset / BLOCK_SIZE;
let sector_offset = cluster_offset % BLOCK_SIZE;
```

读取扇区后，只复制当前请求需要的部分。恰好读完当前簇且文件尚未结束时，通过 FAT 表进入下一簇。这样既支持小于一个扇区的读取，也支持跨扇区、跨簇读取。

11. 将文件系统接入内核

内核初始化时打开第一个 ATA 磁盘，解析 MBR，取得第一个有效分区并挂载 FAT16：

```
let drive = AtaDrive::open(0, 0)
    .ok_or(FsError::DeviceError(DeviceError::UnknownDevice))?;
let table = MbrTable::parse(drive)?;
let part = table
    .partitions()?
    .into_iter()
    .next()
    .ok_or(FsError::InvalidOperation)?;

let fs = Fat16::new(part)?;
ROOTFS.call_once(|| Mount::new(Box::new(fs), "/".into()));
```

最初 `filesystem::init()` 没有加入内核启动流程，执行 `ls` 时 `ROOTFS.get().unwrap()` 会因 `None` 触发 `panic`。修复后在进程管理器初始化完成后调用 `filesystem::init()`，并把 `get_rootfs` 改为返回 `Option`。挂载失败只记录错误，`ls` 和 `open` 返回失败，不再终止内核。

12. 列出目录

添加 `ListDir` 系统调用。内核通过 `Mount::read_dir` 取得元数据迭代器，输出名称、大小和修改时间；目录名末尾添加 `/`：

NAME	SIZE	MODIFIED
EFI/	0.0 B	-
HELLO.TXT	38.0 B	2026-...

用户态 Shell 中：

```
fn cmd_list_dir(path: &str) {
    let path = if path.is_empty() { "/" } else { path };

    if !sys_list_dir(path) {
        println!("Failed to list directory '{}', path);
    }
}
```

13. 打开与读取文件

在进程资源表中增加 `Resource::File(FileHandle)`。 `open` 系统调用取得根文件系统中的文件句柄，并分配文件描述符； `read` 统一通过进程资源表分发； `close` 删除对应资源。

Shell 的 `cat` 遵循 `open-read-close`：

```

let fd = match sys_open(path) {
    Some(fd) => fd,
    None => {
        println!("cat: cannot open '{}'", path);
        return;
    }
};

loop {
    match sys_read(fd, &mut buf) {
        Some(0) => break,
        Some(count) => {
            sys_write(1, &buf[..count]);
        }
        None => break,
    }
}

sys_close(fd);

```

六. 实验结果

1. 单元测试

执行：

```
cargo test --package yyos_storage
```

结果：

```

running 4 tests
test fs::fat16::bpb::tests::test_fat16_bpb_1 ... ok
test fs::fat16::bpb::tests::test_fat16_bpb_2 ... ok
test fs::fat16::direntry::tests::test_dir_entry ... ok
test partition::mbr::entry::tests::partition_test ... ok

test result: ok. 4 passed; 0 failed

```

这说明 MBR 表项、两组 BPB 数据和 FAT16 目录项解析均通过测试。

2. 内核构建检查

执行：

```
cargo check --package yyos_kernel --lib
```

kernel 库目标通过类型检查。工程中仍有部分此前实验留下的未使用导入和未完成可选接口警告，但不影响实验六存储链路。

3. Shell 功能验证

在 FAT16 分区根目录放置 `hello.txt`，内容为：

```
Hello filesystem from 24312063!
```

启动系统后执行：

```
yyos> ls
NAME                SIZE MODIFIED
HELLO.TXT           32.0 B ...

yyos> cat /hello.txt
Hello filesystem from 24312063!
```

`ls` 能列出文件元信息，`cat` 能通过 `open-read-close` 读取并输出文件内容，说明 ATA 驱动、MBR、分区地址平移、FAT16 簇链和系统调用已经连通。

七. 探索 Linux 文件系统

本节按照实验任务直接回答 `procfs`、`devfs`、`tmpfs` 和 `chroot` 相关问题。

1. `procfs`

1.1. `/proc` 下的数字目录

`/proc/<pid>` 中的数字表示进程 ID。每个数字目录是内核为对应进程动态生成的视图，包含：

- `cmdline`：启动命令行；
- `environ`：环境变量；
- `status`、`stat`：进程状态与统计信息；
- `maps`、`smaps`：虚拟内存映射；
- `fd/`：文件描述符符号链接；

- `cwd`、`root`、`exe`：当前目录、根目录和可执行文件；
- `task/`：该进程的线程。

这些内容多数并不存储在磁盘上，而是读取时由内核根据当前状态生成。

1.2. `/proc/cpuinfo` 与 `/proc/meminfo`

`/proc/cpuinfo` 描述逻辑处理器，包括处理器编号、厂商、型号、主频、缓存、核心与线程拓扑、支持的指令集特性等。

`/proc/meminfo` 描述系统内存，包括总内存、空闲内存、可用内存、缓存、页缓存、交换空间、脏页、匿名页、Slab 和大页等。`free` 等工具会读取这些信息。

1.3. `/proc/loadavg` 与 `/proc/uptime`

`/proc/loadavg` 通常包含：

1. 最近 1、5、15 分钟的平均系统负载；
2. 当前可运行任务数/任务总数；
3. 最近创建进程的 PID。

负载并不等同于 CPU 利用率，它主要反映可运行或不可中断睡眠任务的数量。

`/proc/uptime` 包含两个以秒为单位的数值：系统启动后的运行时间，以及所有 CPU 空闲时间的累计值。

1.4. `/proc/interrupts`

该文件按中断号列出每个 CPU 处理该中断的次数、中断控制器类型和设备名称。可以观察时钟、键盘、网卡、磁盘等设备的中断分布，也可用于排查 IRQ 是否触发、是否集中在某个 CPU 等问题。

1.5. `/proc/self/status`

`self` 是指向当前读取进程自身 `/proc/<pid>` 的符号链接。`status` 以可读文本给出进程名、状态、PID/PPID、用户和组 ID、线程数、能力集合、CPU/内存亲和性、上下文切换次数以及 `VmSize`、`VmRSS` 等内存统计。

1.6. `/proc/self/smaps`

`smaps` 在 `maps` 的基础上对每个虚拟内存区域给出更细的统计，包括区域权限、映射文件、RSS、PSS、共享/私有的干净页和脏页、匿名页、交换空间、页大小等。它适合分析进程内存实际占用和共享情况，但读取开销比 `maps` 大。

1.7. `echo 1 > /proc/sys/net/ipv4/ip_forward`

该命令向 `procfs` 中的内核参数文件写入 `1`，启用 IPv4 转发，使 Linux 可以在不同网络接口之间转发 IPv4 数据包，常用于路由器、网关、NAT 和容器网络。

从系统调用角度看，Shell 依然使用普通的 `open`、`write`、`close`。VFS 将写操作转发给 `procfs` 对应节点，节点再修改内核网络参数。“一切皆文件”让应用可用统一接口操作配置、设备和内核状态，便于 Shell 组合、权限控制、重定向和脚本自动化，而不必为每个内核功能设计一套专用系统调用。

2. `devfs` / `devtmpfs`

2.1. 特殊设备文件

- `/dev/null`：读取立即得到 EOF，写入的数据被丢弃，常用于忽略输出。
- `/dev/zero`：读取返回无限的零字节，常用于初始化文件或匿名零页测试。
- `/dev/random`：提供内核随机数；现代 Linux 中初始化后通常与 `/dev/urandom` 具有相近的安全语义，但传统上可能在熵不足时阻塞。
- `/dev/urandom`：提供伪随机字节流，通常不阻塞，适合绝大多数密码学和一般随机数需求。

2.2. `/dev/kmsg`

`/dev/kmsg` 是用户空间访问内核日志缓冲区的字符设备。读取它可以看到内核输出的带序号、时间戳和日志级别的消息；`dmesg` 会通过相关接口读取内核日志。访问通常需要 `root` 或相应 `capability`，因为日志可能包含敏感的系统信息。

2.3. `/dev/sdX` 与 `/dev/sdX1`

`/dev/sdX` 表示整块 SCSI 风格块设备，例如 SATA、USB 存储或虚拟 SCSI 磁盘；`/dev/sdX1` 表示该磁盘上的第 1 个分区。前者包含分区表和全部扇区，后者只映射某个分区的 LBA 范围。

NVMe 系统中类似设备为 `/dev/nvme0n1`，其分区为 `/dev/nvme0n1p1`；virtio 磁盘可能是 `/dev/vda` 与 `/dev/vda1`。

2.4. `/dev/ttyX`、`/dev/loopX`、`/dev/srX`

- `/dev/ttyX`：虚拟控制台终端，例如 `tty1` 对应一个文本控制台。
- `/dev/loopX`：回环块设备，可把普通文件映射为块设备，用于挂载镜像文件。
- `/dev/srX`：SCSI CD/DVD 光驱设备，虚拟机挂载的 ISO 常表现为 `/dev/sr0`。

2.5. /dev/disk 中的符号链接

/dev/disk 常包含：

- `by-id`：按设备厂商、型号、序列号标识；
- `by-uuid`：按文件系统 UUID 标识；
- `by-label`：按文件系统卷标标识；
- `by-partuuid`：按分区 UUID 标识；
- `by-path`：按硬件连接路径标识。

这些符号链接最终指向 `/dev/sdX1`、`/dev/nvme...` 等设备。内核枚举顺序可能变化，`sda` 在下次启动时未必仍是同一磁盘；UUID、序列号或硬件路径更稳定，因此适合写入 `/etc/fstab` 和自动化脚本。

2.6. lsblk 输出解释

`lsblk` 以树状结构展示块设备及其父子关系。常见列包括：

- `NAME`：设备名；
- `MAJ:MIN`：主、次设备号；
- `RM`：是否可移除；
- `SIZE`：容量；
- `RO`：是否只读；
- `TYPE`：disk、part、loop、rom、lvm 等类型；
- `FSTYPE`：文件系统类型；
- `UUID`：文件系统 UUID；
- `MOUNTPOINTS`：挂载点。

磁盘节点下缩进的分区节点表示从属关系；如果分区上还有 LVM、加密或 RAID 映射，还会继续形成更深的设备树。

3. tmpfs

3.1. .pid 文件

PID 文件记录服务进程的 PID。程序启动时可原子地创建并锁定 PID 文件；若文件已被其他活动进程锁定，则拒绝再次启动。仅检查文件是否存在并不可靠，因为程序异常退出可能留下陈旧 PID 文件，因此通常还要配合文件锁或检查 PID 对应进程。

3.2. .lock 文件

锁文件用于协调多个进程对共享资源的访问。程序可以通过 `flock`、`fcntl` 文件锁或“排他创建文件”获得锁；只有持锁者可以操作资源，退出或关闭文件描述符后锁被释放。它常用于包管理器、串口设备、数据库和定时任务。

3.3. `.sock` / `.socket` 文件

这类文件通常是 Unix Domain Socket 的路径名。服务进程 `bind` 到该路径并监听，客户端通过同一路径连接，从而在本机进程间进行双向通信。相比 TCP，它不需要 IP 和端口，并可利用文件权限控制访问。

3.4. tmpfs 的作用

性能方面：数据主要保存在内存中，没有普通磁盘寻道和持久化写入开销，适合临时文件、共享内存、运行时状态和 socket。

安全方面：重启或卸载后内容消失，可减少临时敏感数据长期残留在磁盘上的风险；同时仍可使用 Unix 权限、挂载参数和命名空间进行隔离。但数据可能被交换到 swap，因此高安全场景仍需考虑加密和内存管理。

稳定性方面：运行时状态不污染持久磁盘，启动时总能得到干净环境；tmpfs 还可以设置容量上限，避免无限增长。不过它会占用内存和 swap，若缺少限制，大量写入仍可能造成内存压力。

4. chroot 前挂载 proc、sys 与 dev

命令：

```
mount proc /mnt/proc -t proc -o nosuid,noexec,nodev
mount sys /mnt/sys -t sysfs -o nosuid,noexec,nodev,ro
mount udev /mnt/dev -t devtmpfs -o mode=0755,nosuid
```

第一条把 procfs 挂载到新根目录，为 chroot 中的程序提供进程和内核状态；`nosuid`、`noexec`、`nodev` 限制其中内容被当作特权程序、可执行文件或设备节点使用。

第二条挂载 sysfs，使新系统能够看到设备模型、驱动、总线、电源和内核对象；示例中使用 `ro` 只读挂载，减少安装环境误修改内核设备属性的风险。

第三条挂载 devtmpfs，为新根目录提供控制台、终端、磁盘、随机数、null 等设备节点。实际安装还常绑定挂载 `/dev/pts`、`/dev/shm`、`/run`。

`chroot` 只改变进程看到的根目录，并不会自动创建这些虚拟文件系统。如果不挂载：

- `ps`、`top`、`free` 等依赖 `/proc` 的工具无法正常工作；
- `udev`、硬件探测和部分服务无法通过 `/sys` 获取设备信息；
- 缺少 `/dev/null`、`/dev/tty`、磁盘和伪终端，Shell、安装器和包管理器可能失败；

- DNS、终端会话、服务管理和设备配置能力会不完整。

八. 思考题

1. 1. storage crate 为什么使用条件 `no_std`？为什么 kernel 难以单元测试？

`#![cfg_attr(not(test), no_std)]` 表示正常编译 storage crate 时不链接标准库，只依赖 `core` 和 `alloc`，因此它可以运行在没有操作系统运行时的内核环境中；执行单元测试时则不启用 `no_std`，由 Rust 测试框架和宿主机标准库提供线程、输出和测试入口。

storage 中的 MBR、BPB、目录项等纯数据解析逻辑不依赖硬件，适合在宿主机进行单元测试。kernel 则较难测试，原因包括：

1. `no_std`、自定义入口、panic handler 和链接脚本与普通测试程序不同。
2. 端口 I/O、页表、中断、特权指令只能在 Ring 0 或虚拟机中执行。
3. 内核依赖 bootloader 提供内存映射、UEFI 表和已加载应用。
4. 全局中断、调度器和硬件状态使测试之间难以完全隔离。
5. 出错时可能直接 panic、三重故障或重启，无法使用普通测试框架收集结果。

因此合理做法是把可纯函数化的部分放入独立 crate 单元测试，把硬件和内核集成部分放入 QEMU 集成测试。

2. 2. MbrTable 泛型、Clone、PhantomData 与 Sized 的作用

`T: BlockDevice<B> + Clone` 表示 MBR 可以建立在任意块设备之上，例如真实 ATA 磁盘、内存磁盘或测试设备。解析分区后，每个 `Partition` 都需要持有同一个底层设备的逻辑句柄，因此需要克隆 `T`。

`B` 只出现在 trait 约束和方法签名中，没有直接存储在 `MbrTable` 字段里。`PhantomData<B>` 告诉编译器该结构在类型和生命周期语义上与 `B` 相关，也避免“未使用类型参数”错误。

`PartitionTable::parse` 返回 `Self`。trait 对象 `dyn PartitionTable` 的具体大小在编译期未知，不能直接按值返回，因此要求 `Self: Sized`，表示该方法只适用于大小已知的具体实现。

3. 3. AtaDrive 如何实现 Clone？分离 AtaBus 与 AtaDrive 的好处

`AtaDrive` 的字段是 `u8`、`u32` 和 `Box<str>`。整数可直接复制，`Box<str>` 实现了深拷贝，因此结构体可以派生 `Clone`。克隆的只是磁盘描述和总线索引，不是复制硬件。

`AtaBus` 持有真实端口对象并放在全局 `Mutex` 中，`AtaDrive` 只保存 bus/drive 编号和设备信息。这样设计的好处是：

- 端口访问集中管理，避免多个对象同时操作同一 ATA 总线；
- `AtaDrive` 轻量、可克隆，适合交给 MBR 和多个分区对象；
- 总线和设备职责分离，便于支持同一总线上的主盘与从盘；
- 上层只依赖 `BlockDevice`，不需要知道端口寄存器细节；
- 更容易使用内存块设备替换 ATA 驱动进行测试。

4. 4. 泛型、impl Trait 与 dyn Trait 的异同

4.1. 函数参数

`fn f<T: Foo>(f: T)` 使用显式泛型。调用者的具体类型在编译期确定，编译器会单态化，可在函数中引用类型参数 `T`，也能添加更多约束。

`fn f(f: impl Foo)` 也是静态分发，本质上是匿名泛型参数。写法简洁，但函数签名中不能直接命名该类型。每次调用仍只能传入一个具体类型。

`fn f(f: &dyn Foo)` 使用 trait object 和动态分发。参数由数据指针和虚表指针组成，可以在运行时接受不同具体类型，减少单态化代码膨胀，但有一次虚表调用开销，并受到 object safety 限制。

4.2. 结构体字段

`struct S<T: Foo> { f: T }` 的字段大小和具体类型在编译期已知，通常无堆分配和虚表开销，但 `S<A>` 与 `S<B>` 是不同类型。

`struct S { f: Box<dyn Foo> }` 在堆上存放具体对象，结构体本身大小固定，可以在运行时装入任意实现 `Foo` 的类型。实验中的 `Box<dyn BlockDevice<Block512>>` 和 `Box<dyn FileSystem>` 就利用了这种类型擦除。

5. 5. 硬链接、软链接与 Windows 快捷方式

硬链接是目录项直接指向同一个 inode。多个硬链接地位相同，共享文件内容和 inode 元数据；删除一个名字不会删除其他名字，只有链接计数归零且没有打开引用时才释放文件。硬链接通常不能跨文件系统，也通常禁止对目录创建。

软链接是一个独立 inode，其内容是目标路径。它可以跨文件系统，也可以指向目录；目标被删除或路径变化后会成为悬空链接。

Windows 快捷方式通常是 `.lnk` 普通文件，由 Shell 解释，除目标路径外还可保存图标、启动参数和工作目录。它更像桌面层元数据，不是所有文件 API 都会自动跟随。Linux 软链接由文件系统和 VFS 直接解析，对绝大多数程序透明。Windows 也存在 NTFS symbolic link、junction 和 hard link，它们才更接近 Linux 链接。

6.6. 日志文件系统与 FAT 的区别

FAT 这类非日志文件系统直接更新 FAT 表、目录项和数据。一次操作可能需要修改多个位置，如果写到一半断电，可能出现已分配簇没有目录项、目录项指向未完成簇链、重复分配或空间泄漏。恢复时通常需要 `fsck` 扫描较大范围的数据结构，耗时与文件系统容量和文件数量相关。

日志文件系统在修改主要结构前，先把事务意图或元数据更新写入日志，并通过提交记录确定事务是否完成。崩溃恢复时只需重放已提交事务、撤销未提交事务，能较快恢复一致状态。

日志实现需要事务、顺序保证、日志空间回收、校验和屏障等机制，代码更复杂，并增加额外写入。NTFS 使用日志保护关键元数据；一些 Linux 文件系统还支持 ordered、writeback 或 data journaling 等不同策略。日志通常保证文件系统结构一致，但不一定保证应用的最新数据已经持久化，应用仍需在适当位置调用 `fsync`。

九. 可选思考

1. 错误边界如何划分？

- 驱动层：报告超时、设备不存在、状态寄存器错误、读写失败和不支持的命令。
- 分区层：报告无效 MBR、越界 LBA、整数溢出和无有效分区。
- 文件系统层：报告 BPB/目录项损坏、坏簇、簇链循环、文件不存在、路径或类型错误。

最适合做面向用户的容错兜底的是文件系统/VFS 边界，因为它掌握操作语义，可以把底层错误转换为 `open`、`read`、`ls` 的失败；但底层各层仍必须尽早验证自身不变量，不能把越界和损坏数据继续向上传递。内核 syscall 层应保证错误不会演变为 panic。

2. FAT16 写入需要保证的一致性不变量

1. 一个已分配簇不能同时属于两个文件的簇链。

2. 目录项文件大小、首簇号和实际簇链必须一致。
3. FAT 链必须终止，不能形成意外环路或指向非法簇。
4. FAT 的多个副本应保持一致。
5. 删除文件时，目录项状态与簇释放不能产生长期悬挂或重复分配。
6. 扩展文件时，应先准备新簇，再以可恢复顺序连接簇链和更新目录项。

3. 教学实验优先采用哪种写回策略？

教学实验中我会先实现立即写回，因为语义直观、状态较少，便于验证每一步磁盘修改。待基本写功能稳定后，再引入小型日志或写回缓存。延迟写回虽然性能更好，但同时要求脏页管理、刷新时机、写入顺序和崩溃恢复，会显著扩大调试范围。

十. 实验总结

本次实验第一次让内核直接解释真实磁盘上的持久化数据。实现过程展示了操作系统存储栈清晰的层次：ATA 驱动只处理端口与扇区，MBR 将磁盘切分为分区，Partition 完成地址平移，FAT16 将簇链和目录项解释为文件，VFS 风格接口再把文件能力提供给系统调用和 Shell。

调试中最典型的问题是 ROOTFS 没有在启动阶段初始化。ls 从用户态一路进入内核后，最终对空的 `spin::Once` 执行 `unwrap`，导致整个系统 panic。这说明内核边界不能依赖“初始化一定成功”的假设。修复后，磁盘识别、分区解析和 FAT16 挂载都返回 `Result`，而系统调用对未挂载状态返回错误，系统即使无法访问磁盘也不会因此崩溃。

另一个重要认识是，磁盘上的结构必须严格按照规范解释。MBR 的 active 位不代表分区是否存在；FAT16 根目录与普通目录的存储方式不同；文件读取需要同时处理扇区边界、簇边界和文件长度。任何一个偏移或端序错误都会在更高层表现为完全不同的问题。

通过 Linux 特殊文件系统的探索，可以看到文件接口并不只用于持久化普通文件。内核状态、设备、进程信息、IPC 端点和运行时数据都可以映射为统一的路径与读写操作。这种统一抽象降低了用户程序和工具之间的组合成本，也是本实验中 `BlockDevice`、`FileSystem`、`Mount` 和 `Resource` 分层设计的现实对应。

十一. 参考资料

1. YatSenOS v2 Tutorial，实验六：硬盘驱动与文件系统，

<https://ysos.gzti.me/labs/0x06/tasks/>

2. OSDev Wiki, ATA PIO Mode ,
https://wiki.osdev.org/ATA_PIO_Mode
3. OSDev Wiki, FAT ,
<https://wiki.osdev.org/FAT>
4. Linux kernel documentation, procfs ,
<https://docs.kernel.org/filesystems/proc.html>
5. Linux manual pages: `proc(5)` 、 `random(4)` 、 `null(4)` 、 `tmpfs(5)` 、 `chroot(2)` °



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验七：内存管理与用户堆

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 完善物理帧分配器，使已经释放的物理帧能够被后续分配复用。
2. 理解进程虚拟内存中代码段、栈、堆和页表的所有权，正确释放退出进程占用的页面。
3. 统计进程虚拟内存和内核物理内存的使用情况，并在进程列表中显示。
4. 实现内核栈按需增长，理解页故障处理与页面权限之间的关系。
5. 实现 `brk` 系统调用、用户堆扩缩和用户态动态内存分配。

二. 实验环境

项目	配置
操作系统	YatSenOS / YYOS
开发语言	Rust (<code>no_std</code> 、 <code>no_main</code>)
目标架构	x86_64
引导环境	UEFI + OVMF
虚拟机	QEMU
构建工具	Cargo 、 Make
报告工具	Typst

三. 实验内容概述

本实验从“用户程序的内存释放”开始，完成了实验七后续必做任务。实现后的内存管理路径如下：


```

Bootloader
|-- 加载内核 ELF，记录映射页范围
|-- 仅映射内核栈初始页面
`-- BootInfo(kernel_pages, memory_map, ...)
    |
Kernel    |-- BootInfoFrameAllocator：新帧 + 回收帧
           |-- ProcessVm：代码段 + 栈 + 堆 + 页表上下文
           |-- Page Fault：用户栈或内核栈按需增长
           `-- sys_brk：调整用户堆边界
    |
User library    `-- brk_alloc 全局分配器

```

物理帧分配器负责“帧是否可再次使用”，`ProcessVm` 负责“一个进程拥有哪些虚拟内存区域”，页表上下文则通过 `Arc` 表达 `fork` 后父子进程共享地址空间的生命周期。

四. 实验原理

1. 帧回收

Bootloader 提供的可用物理帧迭代器是惰性的，适合顺序分配，却不能把已经取出的帧重新放回原迭代器。因此分配器增加 `Vec<PhysFrame>` 作为回收栈：

```

pub struct BootInfoFrameAllocator {
    size: usize,
    frames: BootInfoFrameIter,
    used: usize,
    recycled: Vec<PhysFrame>,
}

```

分配时优先 `pop` 回收帧，回收集合为空时再从启动内存图迭代器中获取新帧；释放时将帧 `push` 到回收集合。`used` 统计从启动内存图中取出的帧数，当前仍在使用的帧数可表示为：

$$\text{live_frames} = \text{used_frames} - \text{recycled_frames}$$

使用 `Vec` 时每个记录约占 8 字节，空间效率低于位图，但实现简单，适合本实验规模。

2. 进程虚拟内存的所有权

用户进程的内存占用由三部分组成：ELF 映射、用户栈和用户堆。本实验忽略页表本身的统计，但在进程退出时仍会释放页表页面。

ELF 加载函数返回实际映射的页范围，`ProcessVm` 保存这些范围并累计字节数。栈和堆各自记录当前范围，并提供 `memory_usage` 与 `clean_up`。因此：

$$\text{process_memory} = \text{elf_memory} + \text{stack_memory} + \text{heap_memory}$$

`fork` 后页表上下文由 `Arc` 共享。若每个 `ProcessVm` 都立即解除代码段和堆映射，就会重复释放相同物理帧。因此清理时先释放进程私有栈，仅当 `Arc::strong_count(page_table) == 1` 时，才由最后一个地址空间持有者清理共享堆、ELF 页面和低级页表。最顶层 P4 页由页表根对象的 `Drop` 释放。

3. 内核内存统计

Bootloader 加载内核 ELF 时记录所有映射页范围，并通过 `BootInfo` 传给内核。内核 `ProcessVm` 以这些范围初始化代码段统计，再加上内核栈得到内核进程的虚拟内存用量。

进程列表同时读取帧分配器状态，输出已使用、已回收、总帧数和物理内存占用比例。统计反映的是当前由分配器管理的可用物理区域，不包括固件保留区域。

4. 内核栈自动增长

内核栈在配置中保留 1 MiB 虚拟地址范围，但启动时只映射顶部 8 页，即 32 KiB。访问尚未映射、但仍位于预留栈范围内的地址时触发页故障，统一的栈增长逻辑分配并映射缺失页面。

用户栈页面使用 `PRESENT | WRITABLE | USER_ACCESSIBLE`，内核栈只使用 `PRESENT | WRITABLE`。若错误地给内核栈设置 `USER_ACCESSIBLE`，会破坏内核与用户地址空间的权限边界。

5. brk 与用户堆

`brk` 调整进程数据段末端地址。调用参数为 0 时查询当前 `break`；参数非 0 时尝试设置新 `break`。用户请求的 `break` 可以不是页对齐地址，但页面映射必须按 4 KiB 对齐，因此实现分别维护：

- 精确 `break`：用于系统调用返回值和字节级内存统计；
- 对齐后的映射末端：用于决定需要新增或解除哪些页面。

扩展堆时逐页分配物理帧并映射；失败时回滚本次已经映射的页面。缩小堆时解除不再覆盖的整页映射并回收物理帧。用户库通过 `sys_brk` 封装系统调用，并启用 `brk_alloc` 作为全局分配器。

五. 实验过程与实现

1. 实现帧分配与回收

在 `BootInfoFrameAllocator` 中加入 `recycled` 集合和统计接口。分配顺序为“回收帧优先，新帧兜底”，释放时还使用调试断言检查同一帧未被重复加入回收集合。

这种计数语义使运行时可以直接显示：

```
Frames : 281 used, 0 recycled, 3183 total
```

当进程退出后，释放的页面进入 `recycled`；后续创建进程时优先复用这些帧，不再单调消耗启动内存图中的新帧。

2. 记录并释放 ELF 映射

修改 ELF 模块，使 `load_elf` 返回各可加载段实际覆盖的 `PageRangeInclusive`。

`ProcessVm::load_elf_code` 保存范围并累计页数。退出时遍历范围：

1. 使用 `mapper` 解除虚拟页映射；
2. 获得原物理帧并交给帧回收器；
3. 刷新对应 TLB 项；
4. 最后清理不再使用的下级页表。

零长度 ELF 段不会产生页范围，避免页数计算下溢。

3. 完善 ProcessVm 生命周期

`ProcessVm` 的 `Drop` 调用统一的 `clean_up`，确保正常退出、加载失败或所有者提前返回时都能释放资源。清理顺序为私有栈、共享地址空间内容、低级页表、顶级页表根对象。

原有页表根析构中递归释放用户页的逻辑被移除，避免它与 `ProcessVm` 的区域清理重复释放同一物理帧。页表根析构现在只负责自己明确拥有的 P4 帧。

4. 添加内存显示

`Process` 的格式化实现和 `print_process_list` 增加 Memory 列，显示每个进程的 ELF、栈和堆总占用。进程列表末尾增加物理内存与帧状态，例如：

```
shell  47.99 K  Running
kernel 32.29 M  Ready
Memory : 1.10 MiB / 12.43 MiB (8.83%)
Frames : 281 used, 0 recycled, 3183 total
```

5. 传递内核页范围并启用内核栈增长

启动程序保存加载内核 ELF 时得到的映射范围，在 `BootInfo` 中新增 `kernel_pages`。内核进程据此初始化 `ProcessVm` 的代码区域。

`boot.conf` 将内核栈虚拟范围设为 1 MiB，并指定启动时只映射 8 页。内核页故障处理不再直接终止，而是先交给当前进程的 `ProcessVm` 判断故障地址是否位于可增长的栈范围内。

6. 实现 `brk` 完整调用链

系统调用链依次完成以下修改：

```
用户程序
-> yslib::sys_brk
-> Syscall::Brk = 12
-> syscall dispatcher
-> service::sys_brk
-> proc::brk
-> ProcessInner::brk
-> ProcessVm::brk
-> Heap::brk
```

内核入口使用 `VirtAddr::try_new` 拒绝非规范地址，并以 `usize::MAX` 表示失败。堆实现检查地址是否位于预设用户堆区间，扩展时具有失败回滚，缩小时回收整页。

用户库默认特性由 `kernel_alloc` 切换到 `brk_alloc`。同时删除旧的 `allocator.rs`，解决它与 `allocator/mod.rs` 同时存在导致的 Rust 模块冲突，并修正工作区依赖中的包名为实际的 `yslib`。

六. 测试与结果

1. 静态检查与构建

完成以下检查，均通过：

```
cargo check -p yyos_kernel --lib --offline
cargo check -p yslib --offline
cargo check -p yyos_boot --lib --offline
cargo check -p yyos_elf --offline
make target/x86_64-unknown-yyos/release
make target/x86_64-unknown-none/release/yyos_kernel
make target/x86_64-unknown-uefi/release/yyos_boot.efi
make build
git diff --check
```

用户程序 `shell` 、 `hello` 、 `dinner` 、 `fork` 、 `counter` 、 `fish` 、 `factorial` 和 `mq` 均能为自定义用户态目标成功构建。相关实验目录中不再存在未完成的 `FIXME/TODO` 。

2. QEMU 运行测试

系统可以通过 UEFI 正常启动。日志显示内核栈总预留大小为 1 MiB，初始映射 8 页；Shell 启动时 `brk` 分配器成功初始化。

执行 `stat` 能看到进程内存和物理帧统计。执行 `hello` 后，用户堆发生如下扩展：

```
Adjust heap break:
0x2000000000000 -> 0x200000001ff8
mapped end:
0x2000000000000 -> 0x200000002000
```

`hello` 正常输出并以 233 退出。退出日志出现顶级页表释放记录，Shell 正确收到退出码，系统未发生 panic、重复释放或页故障循环。QEMU 最终仅由测试命令的超时机制终止。

七. 问题与解决

1. 共享地址空间导致重复释放

最初若同时在 `ProcessVm` 清理和页表根析构中遍历用户页面，退出时会对同一帧执行两次回收。解决方法是明确所有权：`ProcessVm` 负责代码、栈、堆和下级页表，页表根对象只负责 P4；共享部分只由最后一个 `Arc` 持有者清理。

2. 用户栈与内核栈权限不同

直接复用用户栈增长函数会给内核栈加上 `USER_ACCESSIBLE`。为 `Stack` 增加 `user_access` 属性后，映射标志由栈类型决定，既复用了增长逻辑，也保持了页级权限隔离。

3. break 与页面边界不一致

`brk` 的接口按字节工作，而页表按页面工作。若只保存对齐地址，会向用户返回错误的 `break`；若完全按字节解除映射，又会错误释放仍被堆覆盖的页面。因此实现保存精确 `break`，并仅根据向上对齐后的边界增删页面。

4. 构建配置中的历史冲突

工程中同时存在 `allocator.rs` 与 `allocator/mod.rs`，`Rust` 无法确定模块来源；工作区又将实际包 `yslib` 写成了 `yylib`。删除过时模块文件并修正 `package` 名后，用户态自定义目标可以正常编译。

八. 思考题

1. 删除正在运行的 ELF 文件会发生什么

`Linux` 中运行程序的地址空间持有可执行文件对应 `inode` 和映射的引用。`unlink` 只删除目录项；只要仍有进程引用该 `inode`，文件数据不会立即回收，进程也可以继续运行。尚未调入的文件页仍可通过已有映射和页缓存读取。当最后一个文件描述符、映射和 `inode` 引用都释放后，存储空间才真正回收。访问不属于有效映射的地址仍会触发 `SIGSEGV`。

2. 为什么 `Arc::strong_count` 是关联函数

`Arc<T>` 实现 `Deref<Target = T>`。如果 `strong_count` 设计成普通方法，可能与 `T` 自身的同名方法在自动解引用中产生语义混淆。写成 `Arc::strong_count(&value)` 可以明确表示查询的是 `Arc` 控制块，而不是内部对象的方法，也不会占用 `T` 的方法命名空间。

3. 内核栈最少映射多少页

不存在对所有内核都固定的最小值。初始页面必须支撑从进入内核到页故障处理器、帧分配器、`GDT/IDT` 和日志设施都可用之前的最大栈深度。本实现中 8 页即 32 KiB 可以稳定启动。若

初始映射过小，内核可能在页故障处理能力尚未建立时再次缺页，进一步引发双重故障甚至三重故障重启。

4. mmap、munmap 与 mprotect

`mmap` 创建文件或匿名内存映射，`munmap` 删除映射，`mprotect` 修改映射页面的读、写、执行权限。下面的 Linux 示例把文件映射为共享可写内存并显式同步：

```
int fd = open("data.bin", O_RDWR);
ftruncate(fd, 4096);
char *p = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
               MAP_SHARED, fd, 0);
memcpy(p, "hello", 5);
msync(p, 4096, MS_SYNC);
mprotect(p, 4096, PROT_READ);
munmap(p, 4096);
close(fd);
```

对 `MAP_SHARED` 映射的写入会先修改内存页，随后由 `msync`、解除映射或内核回写机制同步到文件；`MAP_PRIVATE` 则使用写时复制，修改不会回写原文件。

九. 总结

本实验将上一阶段“能够分配页面”的实现扩展为具有完整生命周期的内存管理：物理帧可以回收复用，进程退出会释放 ELF、栈、堆和页表，内核栈可以按需增长，用户程序可以通过 `brk` 使用动态堆，同时系统能够观察进程和物理内存占用。

实现中最关键的是明确所有权边界。只有当每类页面都有唯一、可追踪的释放责任时，`fork`、进程退出和错误回滚才能同时正确。最终构建与 QEMU 实测均通过，实验任务要求的相关占位实现已经补全。

十. 参考资料

1. YatSenOS 实验七任务：<https://ysos.gzti.me/labs/0x07/tasks/>
2. Rust `alloc::sync::Arc` 文档。
3. x86_64 页表与页故障处理相关文档。
4. Linux `mmap(2)`、`munmap(2)`、`mprotect(2)` 手册。



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验八：VGA 图形输出与键盘输入

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 理解 UEFI Graphics Output Protocol (GOP) 及线性 framebuffer 的工作方式。
2. 在 Bootloader 中获取显示模式、分辨率、stride、像素格式和显存地址，并将其传递给内核。
3. 使用 `embedded-graphics` 抽象实现基本图元、字符绘制和 Shell 图形终端。
4. 将 framebuffer 输出接入内核日志及用户程序标准输出，同时保留串口调试能力。
5. 实现 PS/2 键盘中断，将 GUI 键盘与串口输入汇入同一 Shell 输入队列。
6. 完成上下分屏和动态时钟图形加分项。

二. 实验环境

项目	配置
操作系统	YatSenOS / YYOS
开发语言	Rust (<code>no_std</code> 、 <code>no_main</code>)
目标架构	x86_64
引导环境	UEFI + OVMF
虚拟机	QEMU 8.2.2
图形接口	UEFI GOP，线性 framebuffer
图形库	<code>embedded-graphics 0.8.2</code>
键盘库	<code>pc-keyboard 0.8.0</code>
实测模式	1280×800，BGR，stride 1280
报告工具	Typst

三. 实验内容概述

本实验在原有串口控制台的基础上加入图形显示和键盘输入。完整数据通路如下：

UEFI GOP

```
-> Bootloader 获取 framebuffer 元数据
-> BootInfo::frame_buffer
-> FrameBufferConsole
|-- DrawTarget<Rgb888>
|-- embedded-graphics 字符和图元
|-- ANSI 清屏及日志颜色
`-- 上半屏动态时钟 / 下半屏 Shell

串口 RX IRQ4 ----\
      +-> INPUT_BUF<ArrayQueue<u8>> -> sys_read -> Shell
PS/2 IRQ1 -----/

内核日志 / sys_write
-> print_internal
|-- UART16550
`-- FrameBufferConsole
```

这种设计没有修改用户程序的标准 I/O 接口。Shell 仍然通过 `stdin`、`stdout` 和系统调用工作，显示设备与输入设备的合并完全由内核完成。

四. 实验原理

1. UEFI Graphics Output Protocol

UEFI GOP 提供当前显示模式和 framebuffer。Bootloader 使用：

```
let handle = boot::get_handle_for_protocol::<GraphicsOutput>()?;
let mut gop = boot::open_protocol_exclusive::<GraphicsOutput>(handle)?;
let mode = gop.current_mode_info();
let mut frame_buffer = gop.frame_buffer();
```

本实验需要保存以下数据：

字段	作用
<code>address</code>	framebuffer 物理基地址
<code>size</code>	显存可访问字节数，用于边界检查
<code>width</code> / <code>height</code>	屏幕可见分辨率
<code>stride</code>	每条扫描线实际像素数，可能大于 width
<code>pixel_format</code>	RGB、BGR 或不支持的格式

像素地址不能简单使用 `y * width + x`，必须使用：

$$\text{offset} = (y \times \text{stride} + x) \times \text{bytes_per_pixel}$$

实测 QEMU GOP 模式为 1280×800、BGR、每像素 4 字节，framebuffer 物理地址为 `0x80000000`，大小为 `0x3e8000`。

2. framebuffer 地址映射

Bootloader 已经将物理地址空间线性映射到高半区。GOP 提供的是显存物理地址，因此内核访问地址为：

$$\text{virtual_address} = \text{framebuffer_address} + \text{physical_memory_offset}$$

显示驱动直接使用同一个 `BootInfo` 中的物理偏移，避免依赖全局地址模块的初始化顺序。所有显存读写使用 `read_volatile` 和 `write_volatile`，防止编译器消除具有设备副作用的访问。

3. embedded-graphics DrawTarget

`FrameBufferConsole` 实现：

```
impl OriginDimensions for FrameBufferConsole { ... }
impl DrawTarget for FrameBufferConsole {
    type Color = Rgb888;
    type Error = Infallible;
    ...
}
```

`draw_iter` 将 `Rgb888` 转换成 GOP 的 RGB/BGR 字节顺序，然后写入显存。实现 `DrawTarget` 后，字符、矩形、圆和直线都可以复用 `embedded-graphics` 的统一绘制接口。

字符渲染使用 `FONT_8X13` 单色字体。中文等字体中不存在的字符显示为 `?`，避免错误解析 UTF-8 字节。当前终端每个字符占 8×13 像素，行距为 15 像素。

4. 图形终端

屏幕被划分为两个区域：

- 上半屏：深蓝色图形仪表区，绘制标题、圆形表盘和动态秒针；
- 下半屏：深色 Shell 终端区，支持字符、换行、回车、退格、滚屏和清屏。

终端实现了实验所需的常用 ANSI 控制序列：

- `ESC[2J`：清除下半屏终端；
- `ESC[H`：光标回到终端左上角；

- `ESC[31m` 至 `ESC[36m` 、 `ESC[90m` ：日志颜色；
- `ESC[0m` ：恢复默认颜色。

当光标到达终端底部时，驱动按一行高度把 framebuffer 内容向上复制，再清除最后一行。Shell 的 `clear` 命令只清理下半屏，上方图形区继续保留。

5. 输出复用与并发

内核 `print_internal` 在关闭中断的临界区内依次写入 UART 和 framebuffer，因此内核日志、`sys_write`、Shell 及所有用户程序都会同时输出到两个设备。

framebuffer 由 `spin::Mutex` 保护。时钟中断更新图形时使用 `try_lock`：若普通输出正在持锁，就跳过当前动画帧，避免在中断上下文中等待锁形成死锁。串口仍然保留，图形驱动失效时也可以继续调试系统。

6. PS/2 键盘中断

PS/2 键盘使用 IRQ1，对应 IDT 向量 `0x21`。中断处理过程为：

1. 从 I/O 端口 `0x60` 读取扫描码；
2. 使用 `pc-keyboard` 的 `ScancodeSet1` 和 `Us104Key` 布局解析按键；
3. 将可表示为 ASCII 的 `DecodedKey::Unicode` 写入输入队列；
4. 向 Local APIC 发送 EOI。

键盘和串口接收中断都调用已有的 `drivers::input::push_key`。因此 `Resource::Console(Stdin)` 无须区分输入来源，用户态 `stdin().read_line()` 也不需要修改。

五. 实验过程与实现

1. 扩展 BootInfo

在 boot crate 中定义与 UEFI 生命周期无关的纯数据结构：

```
pub struct FrameBufferInfo {
    pub address: u64,
    pub size: usize,
    pub width: usize,
    pub height: usize,
    pub stride: usize,
    pub pixel_format: FrameBufferPixelFormat,
}
```

Bootloader 在退出 Boot Services 前获取 GOP 信息，并把 `Option<FrameBufferInfo>` 写入 `BootInfo`。若环境不提供 GOP，内核仍可依靠串口启动。

2. 实现图形驱动

新增 `drivers/framebuffer.rs`，实现以下能力：

1. RGB/BGR 像素转换和显存边界检查；
2. `embedded-graphics` 的 `DrawTarget<Rgb888>`；
3. `FONT_8X13` 字符绘制；
4. 换行、回车、退格和自动滚屏；
5. ANSI 清屏、光标复位和日志颜色；
6. 上下分屏；
7. 圆、直线、矩形等基本图形绘制；
8. 时钟中断驱动的动态秒针。

显示初始化在日志系统之前完成。日志初始化后会输出 `framebuffer` 模式确认信息，实测为：

```
Framebuffer Initialized: 1280x800, stride 1280, Bgr
```

3. 接入日志和标准输出

原有 `print_internal` 只调用 UART 的 `write_fmt`。修改后同时调用：

```
if let Some(mut serial) = get_serial() {
    serial.write_fmt(args).unwrap();
}
framebuffer::write_fmt(args);
```

用户态 `stdout` 经过 `sys_write` 到达内核 `Resource::Console`，最终同样进入 `print!`，所以无需修改 Shell 的普通输出逻辑。Shell 原有的 ANSI 清屏命令也可以直接由图形终端解释。

4. 实现键盘 IRQ

新增 `interrupt/keyboard.rs`，注册 IDT 向量 `0x21`，并在 IOAPIC 中启用 IRQ1：

```
keyboard::register_idt(&mut idt);  
enable_irq(1, 0);
```

扫描码解码后写入和串口相同的 `ArrayQueue<u8>`。回车、退格、Shift 组合和常见符号由 `pc-keyboard` 状态机处理，Shell 继续使用原来的逐字节读取及回显逻辑。

5. 图形加分项

屏幕上半部分使用 `embedded-graphics` 绘制标题、表盘和秒针；下半部分专用于 Shell。时钟中断每 64 tick 尝试刷新一次秒针，降低在中断上下文中重绘大块区域的成本。

本实现没有额外创建用户态后台进程，而是把动态仪表作为低频内核显示任务。它同样可以在 Shell 运行时持续更新，并通过 `try_lock` 与终端输出安全共存。

6. QEMU 启动方式

默认 `make run` 改为保留 GUI，并把串口绑定到宿主终端：

```
QEMU_OUTPUT := -serial stdio
```

同时增加 `make run-headless`，需要纯串口模式时仍可使用 `-nographic`。

六. 测试与结果

1. 编译检查

以下命令均通过：

```
cargo check -p yyos_boot --lib --offline  
cargo check -p yyos_kernel --lib --offline  
make target/x86_64-unknown-uefi/release/yyos_boot.efi  
make target/x86_64-unknown-none/release/yyos_kernel  
make build  
git diff --check
```

普通宿主目标直接检查 bootloader 二进制会触发该工程已有的 `no_std` `panic-unwind` 限制，因此 Bootloader 以真实的 `x86_64-unknown-uefi` 目标构建为准。

2. 启动与显示验证

QEMU 启动日志显示：

```
GOP framebuffer: 1280x800, stride 1280,  
format Bgr, address 0x80000000, size 0x3e8000  
Framebuffer Initialized: 1280x800, stride 1280, Bgr  
Enable IOApic: IRQ=1, CPU=0  
YYOS initialized.
```

系统随后正常加载 Shell，未发生页故障、死锁或重复中断。

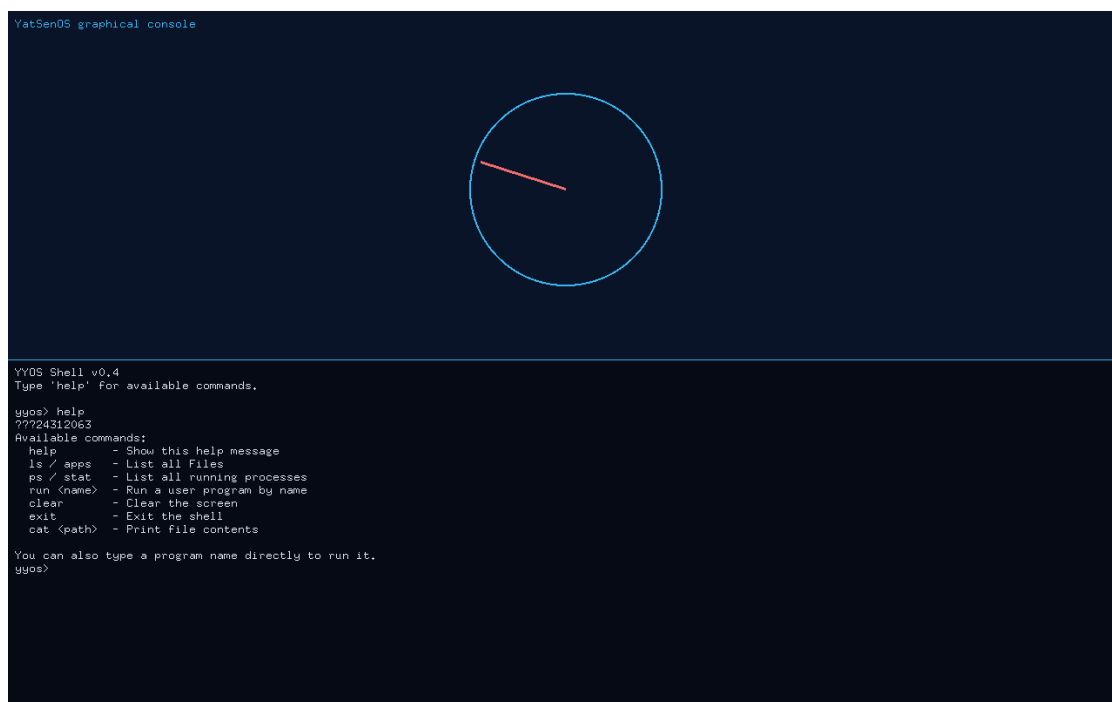


图 1 QEMU framebuffer 实际输出：上半屏动态图形区与下半屏 Shell

截图分辨率为 1280×800，检测到上下区域不同背景色以及文本、边框和时钟指针颜色，说明图形和字符确实由 framebuffer 驱动绘制，而不是来自宿主串口终端。

3. 键盘交互验证

测试通过 QEMU monitor 向 PS/2 键盘发送：

```
sendkey h-e-l-p-ret
```

串口和 VGA Shell 均显示输入回显，Shell 成功执行 `help` 并输出完整帮助菜单，随后返回 `yyos>` 提示符。这同时验证了：

1. IRQ1 正确路由到内核；
2. 扫描码状态机能够解析字符和回车；
3. 键盘输入进入共享 `INPUT_BUF`；
4. 用户态 `sys_read` 和 Shell 命令循环工作正常；
5. 输出能够同时到达 UART 与 framebuffer。

七. 问题与解决

1. framebuffer 初始化顺序

第一版显示驱动调用全局 `physical_to_virtual`，但 framebuffer 初始化早于物理地址模块，运行时会因为偏移尚未注册而 panic。最终改为将 `BootInfo` 中的 framebuffer 信息和物理偏移一起传入显示驱动，从接口上消除初始化顺序依赖。

2. stride 与可见宽度

framebuffer 每行的物理像素数不一定等于屏幕宽度。如果使用 width 计算地址，在某些模式下会出现斜行和越界。实现始终使用 `stride` 计算偏移，使用 width/height 进行可见区域裁剪。

3. 中断上下文锁竞争

时钟中断若阻塞等待 framebuffer 锁，而持锁代码又依赖中断或调度继续执行，会造成死锁。动画刷新使用 `try_lock`，锁被占用时丢弃一帧。图形动画允许丢帧，但输入输出不能因此停顿。

4. ANSI 控制序列

串口终端原先由宿主解释 ANSI 序列，framebuffer 只会看到普通字节。为兼容已有 Shell 和彩色 logger，驱动加入了小型状态机，仅实现工程实际使用的清屏、光标和颜色命令，避免引入完整终端模拟器的复杂度。

5. GUI 与串口同时使用

`-nographic` 会把显示设备重定向到终端，不适合作为 GUI 键盘验收入口。默认运行参数改成 `-serial stdio`，保留 QEMU 图形窗口，同时仍能在终端查看串口日志；另保留 `run-headless` 兼容自动化测试。

八. 思考与改进

1. 当前字体只覆盖 ASCII。后续可加入点阵中文字库或按需加载 PSF 字体。
2. 当前终端只实现实验所需 ANSI 子集。可以继续支持光标移动、前景/背景 256 色和局部擦除。
3. 当前动画由时钟中断直接绘制。更完整的系统应由显示服务或后台内核线程消费 tick 事件，在普通进程上下文重绘。
4. 当前绘制直接写显存，复杂界面可能闪烁。可以增加内存后备缓冲和 dirty rectangle，只刷新变化区域。
5. 当前键盘布局固定为 US 104 键。可将布局、Caps Lock 指示灯和扩展按键处理抽象为输入子系统。

九. 总结

本实验完成了从 UEFI GOP 到内核图形终端的完整链路。Bootloader 将 framebuffer 元数据传入内核，内核基于 `embedded-graphics` 实现像素、图元和字符绘制，并让日志及用户程序标准输出同时到达串口和 VGA。PS/2 键盘中断与串口共享输入队列，使 QEMU GUI 可以直接操作原有 Shell。

在必做内容之外，系统还实现了上下分屏和持续更新的图形时钟。最终真实 UEFI 目标构建、完整镜像构建、QEMU 启动、framebuffer 截图和键盘命令测试均通过。

十. 参考资料

1. YatSenOS 实验八任务：<https://ysos.gzti.me/labs/0x08/tasks/>。
2. UEFI Specification：Graphics Output Protocol。
3. `uefi 0.36.1` crate 文档。
4. `embedded-graphics 0.8.2` crate 文档。
5. `pc-keyboard 0.8.0` crate 文档。
6. OSDev Wiki：PS/2 Keyboard、VGA Hardware、Drawing in a Linear Framebuffer。